

SubRep:
Compositional, Cross-Paradigm
Goal Pursuit for AGI
- *Preliminary draft* -

Ben Goertzel

September 1, 2025

Abstract

We present *SubRep*, a unified framework for compositional goal pursuit that integrates recursive subgoal discovery with planner-facing admission tests and a learnable motive geometry. The core idea is to evaluate options on a data-driven *motive slice* and admit only those that robustly improve the planner’s backed-up value on that slice. We formalize two complementary admission rules: *Cone-Dominant Subtasks* (CDS), which require uniform improvement over a cone W of plausible motive weights, and a more permissive *Pareto-Dominant* rule (PDS), which requires componentwise improvement on a finite MDN-induced cover W_{ref} with budgeted and coverage variants. A *Motive Decomposition Network* (MDN) co-learns the motive geometry (cone or cover) from the same residuals used to train option models, progressively tightening tests and lowering certificate cost as learning proceeds.

We show how predictive coding (PC) networks can implement CD-S/PDS locally without global backprop and why PC’s precision-weighted, factorized credit assignment is a good fit for subgoal decomposition. We also give a modality-neutral interface that lets logical macros and evolutionary programs export compatible expectation-model summaries $(\hat{r}, \hat{n})$, so neural, logical, and evolutionary generators are screened and composed by the same algebra.

Simple two-room case studies are given to illustrate when CDS beats single-weight reward-respecting options, when PDS admits complementary skills a cone would reject, how PC outperforms continual backprop for admission, and how risk-sensitive objectives (e.g., entropic risk) change preferred options.

A category-theoretic formalization is given, yielding a residuated quantale for planning, *weakest-sufficient* decomposition via residuation, and clean monotonicity properties under MDN tightening/refinement. Building on this, we introduce a novel formalism aimed at explaining how neural, logical and evolutionary methods may cooperate effectively to achieve subgoal learning: the *Evo-Adhesive Smooth Atlas* (EASA), a tri-modal recipe that glues smooth neural charts, logical guards, and evolutionary mixtures into planner-admissible decompositions with sufficiency and approximation guarantees.

We explain how to add a *conservative attention* layer (a streamlined, budget-conserving variant of PRIMUS RFS) to the framework, that allocates compute to small bundles of certified options according to their robust margins/gaps.

We provide theoretical results showing join-safety and certification, MDN monotonicity, convergence under two-timescale learning, sample-complexity and approximation bounds, and a logarithmic admitted-library growth analysis. These results indicate that SubRep offers a

principled, cross-paradigm path from motives to options to execution, with auditable certificates and scalable implementation hooks.

As one potential example of scalable realization of these ideas, we sketch an *EASA-Waveformer* architecture that embeds MDN, CD-S/PDS gates, and option heads inside a Waveformer-style transformer, and we map the design to the MORK metagraph substrate for cross-paradigm execution.

In-depth practical experimentation will be the next major step.

Contents

1	Introduction	7
2	Subgoal Decomposition via Cone-Dominance and Pareto-Dominance	9
2.1	Background: Sutton’s reward-respecting subtasks and STOMP	10
2.2	From Reward-Respecting to Cone-Dominant Subtasks (CDS)	14
2.3	Pareto-Based Admission: A Weaker Alternative to Cone-Dominance	20
2.4	What we mean by a ”decision transformer”	25
2.5	Practical tractability and principled covers for CDS/PDS . .	27
2.6	MDN co-learning: explicit losses, regularization, and pathology control	29
3	Synergies between CDS/PDS and Predictive Coding	31
3.1	Brief Review of Predictive Coding for Reinforcement Learning	32
3.2	Why Predictive Coding is a Good Execution Engine for CDS/PDS	37
3.3	Interleaving PC, MDN, and CDS/PDS Throughout the Hierarchy	42
3.4	Continual Backprop vs. Predictive Coding	47
4	Integration with Reasoning, Evolution and Hyperon	50
4.1	MetaMo and weakness-driven learning (WDL)	51
4.2	Beyond standard RL: logic and evolutionary learning	55
5	Tiny two-room illustrations	59
5.1	Basic Two-room example	60
5.2	A simple case where a cone-dominant option beats the best single-weight choice	65
5.3	A two-room variant where explicit logical inference finds the right goal decomposition quickly	68
5.4	A two-room example where quantified logical reasoning outperforms gradient search	69
5.5	A small RL domain where logic + evolution + gradients strictly dominate all pairs	71
5.6	Two-room case where Pareto admission outperforms cone-dominance	73

5.7	Two-room example: PC vs. continual backprop (CBP) for subgoal admission	76
5.8	MDN co-learning in action	79
5.9	Two-room example under entropic risk: when risk aversion prefers a detour	82
6	Automated Motive Decomposition: Just-So Stories	84
6.1	How MDN motives might emerge under CDS/PDS co-learning: Robotics and Research-Assistant Use-Cases	84
6.2	How an MDN might learn subgoals for “moderated pace of goal self-modification”	88
7	Theoretical Properties of Subgoal Decomposition	91
7.1	Basic Properties and Consistency Results	91
7.2	Convergence of MDN-CDS Co-Learning	93
7.3	Sample Complexity Analysis	94
7.4	Approximation Theory	95
7.5	Library Growth Analysis	96
7.6	Cross-Paradigm Unification	97
7.7	Implications and Discussion	97
8	Categorical Framework for Subgoal Decomposition	98
8.1	The Planning Category and Its Quantale Structure	98
8.2	Categorical Characterization of Optimal Decompositions . . .	100
8.3	Connection to Polynomial Functors and Compositional Systems	105
9	Combining Neural, Logical, and Evolutionary Methods: Theory and Practice	109
9.1	Formal EASA Definition and Ideal Properties	110
9.2	When EASA Structure Provably Emerges	111
9.3	Why These Conditions Are Too Strong	112
9.4	Approximate EASA in Practice	113
9.5	Concrete Example: Robot Package Delivery	115
9.6	Theoretical Implications of Approximate EASA	118
9.7	Practical Guidelines	118
9.8	Conclusion	119
9.9	Realistic Compositional Weakness Under Approximate EASA	119

10 Economic Attention Allocation for Resource-Aware Subgoal Selection	123
10.1 From RFS to Conservative Attention: A Simplified Approach	124
10.2 Simplified Conservative Attention for Subgoal Selection . . .	126
10.3 Theoretical Properties of Conservative Attention	130
11 Implementing CDS/PDS with a Co-Learned MDN in a Waveformer Driven by Conservative Attention	133
11.1 Transformers and Waveformers: a short primer	133
11.2 From motives to options inside a Waveformer	134
11.3 Execution-time control: conservative attention in the loop . .	135
11.4 Computing m_o without hiding cost (CDS support function) .	136
11.5 Maintaining an ε -Pareto cover (PDS)	136
11.6 Weakness regularization and origin of the λ 's	137
11.7 Semantics vs implementation: deconfounding the layers . . .	137
11.8 How A1-A3 Improve Transformer/Waveformer Attention . . .	137
11.9 Local PC updates versus backpropagation in a transformer .	139
11.10 Baseline maintenance: alternatives when $O(1)$ is unrealistic .	139
11.11 Computational feasibility with concrete budgets	140
11.12 Robust certificates	141
11.13 Complexity summary	142
11.14 Formal Analysis of Attention Pathologies	143
11.15 Theoretical Analysis: Conservative Attention and Transformer Training	146
11.16 MORK: why and how to map the design	148
11.17 A conservative-attention-first MVP and upgrades	151
12 Conclusion	151

1 Introduction

Reinforcement learning (RL) is a powerful tool for helping AI systems choose actions with promise to help achieve their goals in observed contexts. However, standard RL approaches struggle with various aspects of real-world action-selection, including integration of subgoals on multiple time-scales and, more generally, complexity of interaction of diverse subgoals. In practice to make RL work on real-world action-selection problems in highly complex realistic environments can sometimes be infeasible given practical time and compute constraints.

Introducing temporal abstraction into reinforcement learning frameworks has some promise to overcome these issues, providing faster planning and better sample efficiency by discovering and reusing options with their own termination. Sutton et al [1] have introduced "reward-respecting subtasks" as one way to realize this notion, focusing discovery by preserving the main cumulant and biasing termination toward occasionally high-value states, thereby producing models that often win the planner's per-state maximization. However, this approach also has fairly clear limitations, in cases of real agents facing uncertain, multi-objective, and shifting priorities. In such regimes it seems clearly preferable to look at decomposing top-level rewards or other goals or subgoals into collections of sub-motives, and then admit options that are *robustly* useful across an adaptively constructed set of plausible motives, not just tuned to one weight at a time.

Our aim here is to address this challenge in a robust, general and scalable way. Toward this end we introduce a framework called **SubRep**, which recursively decomposes rewards and goals into subgoals in a geometric way that reflects the underlying compositional structure of the problems involved, and supports efficient computation via a mix of neural gradient descent (backprop and or predictive-coding), explicit logical inference and evolutionary learning, supporting addition of other mechanisms as well.

From reward respect to robust, learned motive slices. We propose here *SubRep*, which turns temporal abstraction into *representation learning* over a learned motive geometry. Two admission rules anchor the framework:

- *Cone-Dominant Subtasks* (CDS), which admit an option only if it beats the baseline *uniformly* for all weights in a motive cone W ; and
- a *Pareto-Dominant* rule (PDS), which admits an option if its stacked backed-up values dominate the baseline on an *MDN-selected finite*

cover W_{ref} , optionally allowing small budgeted regressions while insisting on significant gains elsewhere. Reward-respecting subtasks arise as a strict special case of CDS on a two-ray cone, while PDS often admits diverse, complementary options earlier than CDS.

A *Motive Decomposition Network* (MDN) co-learns the motive slice from the same residuals that train option models, with light disentanglement regularization (orthogonality, sparsity, total correlation, anchors) and stability priors. As the MDN tightens W or refines W_{ref} , certificates get cheaper and admissions sharpen, with monotonic persistence of prior admissions.

Modality-neutral integration and predictive coding. SubRep is agnostic to how options are built: any generator that exports an expectation model $(\hat{r}, \hat{n})$ can be screened by CDS/PDS and safely joined into planning. We detail neural implementations with predictive coding (PC), showing how the max (planner join) and inf/Pareto gates are realized *locally* via error nodes and precisions, avoiding global backprop and providing natural hooks for motive disentanglement. We also show how probabilistic logic and evolutionary program learning export compatible $(\hat{r}, \hat{n})$ summaries and benefit from the same planner-facing certificates.

Theoretical guarantees. Formally, we prove *join-safety* of admitted options: under both CDS and PDS, composing any finite family of admitted generators by joins and sequencing preserves improvement on the learned motive slice, so certificates are stable under planner composition. We show *monotone persistence* under MDN tightening/refinement: when the cone W is narrowed or the finite cover W_{ref} is refined, previously admitted options remain admitted (margins weakly increase), and certificate cost decreases. A coupled two-timescale analysis establishes almost-sure convergence of SubRep’s admission dynamics: fast option/expectation updates track slowly varying MDN geometry, yielding stationary admitted libraries under standard stochastic approximation conditions. We give *sample-complexity and approximation* bounds that scale with cone width and cover size (e.g., with $\text{diam}(W)$ and $|W_{\text{ref}}|$), and *robustness* results showing certificates remain valid under bounded model error and MDN uncertainty via margin cushions. On the algebraic side, we endow the planning operators with a *residuated quantale* structure; residuation yields *weakest-sufficient* prefixes/suffixes for plan specifications and ensures order consistency of CDS/PDS predicates. We bound the growth of the admitted library by a logarithmic law under

mild separability, keeping planning tractable, and we show that the conservative attention allocator is budget-conserving and nonincreasing in a regret surrogate when robust margins are unbiased. Finally, we extend all guarantees to risk-sensitive objectives (e.g., entropic risk, CVaR), and give *EASA* sufficiency/approximation results: glueing smooth neural charts, logical guards, and evolutionary mixtures preserves SubRep certificates on the same motive slice.

A categorical backbone and EASA. We recast planning as a residuated quantale of decision transformers with joins (choice) and composition (sequencing), enabling *weakest-sufficient* prefixes/suffixes via residuation and establishing join-safety of admitted generators on a motive slice. From this view we introduce *EASA* (Evo-Adhesive Smooth Atlas): complex goals can be executed by glueing smooth neural charts (contractive patches), logical guards (clean case splits), and small evolutionary mixtures, with CDS/PDS certificates on an MDN-learned slice. We give sufficiency, approximation, and continuity results, and derive qualitative *compositional weakness* bounds.

Scalable implementation and economic control. To make the pipeline practical, we embed MDN, option heads, and CDS/PDS gates into a Waveformer-style transformer and map the design to the MORK metagraph for efficient cross-paradigm execution. On top of certified options we add a *conservative attention* layer: a single-stage, budget-conserving allocator (a streamlined variant of PRIMUS RFS) that distributes runtime to admitted options in proportion to their robust margins/gaps and measured costs, maintaining strict conservation across hierarchy and modalities.

2 Subgoal Decomposition via Cone-Dominance and Pareto-Dominance

We begin our journey into the integration of subgoal decomposition and representation learning with the excellent work of Sutton et al. [1], who propose a route to making *temporal abstraction* practical inside model-based RL by learning and planning with *options* (policies with stopping rules) whose models are abstract in time, not only in state. The central difficulty confronted here is option discovery: the space of possible options is enormous, and many discovery methods (e.g., bottleneck shortest-paths, eigenoptions) pick subtasks that ignore the main task’s reward, which can

make the resulting option models unhelpful for planning. The paper proposes *reward-respecting subtasks*: subtasks whose cumulant is the original reward and whose stopping rule and stopping value bias termination toward states that are sometimes of high value on the main task. These options yield models that more often “win the max” in value-iteration updates and thus accelerate planning.

In this section we frame this challenge and preview the STOMP progression (SubTask, Option, Model, Planning) in terms of which reward-respecting subtasks are conceived. We then present a broader and more powerful way of realizing STOMP, in which reward-respect is replaced with concepts of cone-dominance or Pareto-dominance across multiple motives – which can be exercised in the context of a fixed set of motives, or (more interestingly) in the context of co-learning with a neural network or other module decomposing goals into motive-sets. The latter approach is potentially especially powerful as it effectively fuses subgoal decomposition with representation learning.

Sutton et al are working strictly within a standard RL framework. The ideas presented here are meaningful and workable within this framework but not restricted to it, and are applicable to systems with multiple top-level goals which may or may not take the form of expected reward maximization.

2.1 Background: Sutton’s reward-respecting subtasks and STOMP

The STOMP framework explicitly describes a pipeline for a reward-seeking agent to follow: (i) **define subtasks** as General Value Functions (GVFs); (ii) **solve them** to get options; (iii) **learn option models** (reward and expected next-state features); (iv) **plan** with those models via approximate value iteration (AVI). The contribution is not a new planning algorithm but a subtask class that better serves the needs of AVI when it chooses among action/option models by a per-state maximization. Although most simply presented sequentially, all four steps can be executed online and off-policy (and that is how the learning algorithms are instantiated). Figure 1 visually compares planning progress in a two-room gridworld when adding a reward-respecting hallway option versus a shortest-path option; the former speeds planning substantially.

2.1.1 GVFs: a unifying language for subtasks

Formalization of these STOMP steps is done via GVFs, which generalize value functions in three ways: they (i) sum an arbitrary *cumulant* c , not just reward; (ii) are allowed to *stop* stochastically according to β ; and (iii) add a *stopping value* z at the stopping time K . Formally,

$$v_{\pi,\beta}^{c,z}(s) = \mathbb{E}_{\pi,\beta} \left[\sum_{j=1}^K \gamma^{j-1} c(S_j) + \gamma^{K-1} z(S_K) \mid S_0 = s \right],$$

with termination handled as a special case. Subtasks fix (c, z) and search over options (π, β) to maximize this value. This single template captures the main task ($c = R$, no stopping), shortest-path subtasks ($c = -1$, $z = 0$ at a subgoal), and many intrinsic-motivation schemes.

2.1.2 Reward-respecting subtasks and feature attainment

Sutton et al’s proposed family — *reward-respecting subtasks of feature attainment* — keeps the cumulant equal to the true reward ($c = R$) while shaping *stopping values* z_i to prefer termination when a particular feature x_i is high in a state that sometimes has high main-task value. With a linear value estimate $\hat{v}(s) = w^\top x(s)$, define a *bonus weight* $\bar{w}_i$ (one of feature i ’s historically high weights) and set

$$z_i(s) = w^\top x(s) - w_i x_i(s) + \bar{w}_i x_i(s),$$

so stopping in states with large x_i gets a *stopping bonus* $(\bar{w}_i - w_i)x_i(s)$. Intuition: during periods when feature i truly matters (i.e., w_i is high), the pre-learned option that *already* steers to “high- x_i , sometimes-high-value” states will become decisively useful for planning. The paper argues that it is only worth building such a subtask when w_i varies over time; otherwise planning has nothing to gain.

2.1.3 Option learning: a compact off-policy actor–critic via a generic TD error

All learning updates in the paper share a generic TD target

$$\delta(c, z, v, v', \beta) = c + \beta z + \gamma(1 - \beta)v' - v$$

and a single UpdateWeights&Traces (UWT) routine that accumulates eligibilities and applies $\alpha\delta$ corrections. By instantiating (c, z, β) appropriately,

the same machinery yields (i) on-policy TD(λ) for the main task, (ii) an off-policy actor-critic for each subtask (with importance sampling), and (iii) later, model learning. In the two-room illustration, one-hot state features ($d = 72$) and a softmax policy over one-hot state-action features are trained from a uniformly random behavior policy μ ; the subtask’s value error shrinks and the learned option’s start-state value approaches its optimal subtask value.

2.1.4 Model learning: expectation models that match AVI

Each action/option $o = (\pi_o, \beta_o)$ has an *ideal* reward model

$$r(s, o) = \mathbb{E} \left[\sum_{t=1}^K \gamma^{t-1} R_t \mid S_0 = s \right]$$

and an *ideal* transition “distribution” $p(s'|s, o) = \sum_{t \geq 1} \gamma^t \Pr[K = t, S_t = s' \mid S_0 = s]$, whose discount matches the AVI backup. Because the planner uses a *linear* value $v(x) = w^\top x$, it suffices to learn an *expectation model* in feature space: $\hat{r}(x, o) \approx r(s, o)$ and $\hat{n}(x, o) \approx \mathbb{E}[\gamma^K x(S_{t+K}) \mid x_t = x, o]$. The paper uses linear forms $\hat{r}(x, o) = w_r^{(o)\top} x$ and $\hat{n}(x, o) = W^{(o)}x$ and learns each row with TD targets. Off-policy learning drives model error down, and with $\sim 20k$ steps of random experience planning is already near-optimal in the two-room case.

2.1.5 Planning: approximate value iteration with “backed-up” values

Planning is asynchronous AVI over the feature space:

$$w \leftarrow w + \alpha \left(\max_{o \in O(x)} [\hat{r}(x, o) + \hat{v}(\hat{n}(x, o); w)] - \hat{v}(x; w) \right) \nabla_w \hat{v}(x; w),$$

so the *backed-up value* $\hat{r} + \hat{v} \circ \hat{n}$ is the per-option look-ahead target. Computing this target is what the x-axis (“planning look-ahead operations”) counts in the paper’s figures. An option is *useful* to the planner insofar as its model tends to take the max in these updates — hence the design goal: options that both produce cumulative reward during execution *and* land the agent in states that sometimes have high value, justifying the stopping bonus.

2.1.6 Empirical contrasts: two-room and four-room

The value of these ideas is illustrated in Sutton et al [1] via simple, small-scale examples, e.g.

Two-room. Start left, goal right, gray band of -1 per step; $\gamma = 0.99$, with optimal start value ≈ 0.843 . Adding a reward-respecting hallway option accelerates planning dramatically compared to actions only, while adding a shortest-path hallway option actually slows planning relative to actions-only — likely because its trajectories traverse sustained negative reward and thus seldom belong to good plans.

Four-room. With stochastic actions (intended direction w.p. $2/3$), the authors define four reward-respecting hallway-attainment subtasks, learn their options, model them, and plan. Planning with reward-respecting options is fastest; then shortest-path, eigenoptions, option-critic, and actions-only (the latter eventually catches up but is sensitive to estimation bias).

2.1.7 Stopping bonuses matter

Varying the bonus weight $\bar{w}_i$ shifts the trade-off between reward accumulation and feature attainment. In the two-room grid, $\bar{w}_h = 1$ yields a round-about hallway option that avoids the gray band; very large $\bar{w}_h$ (e.g., 100) collapses toward shortest-path behavior; small $\bar{w}_h$ (e.g., 0.1) reduces the emphasis on the feature in the second room. Planning performance remains robust and best for intermediate “reasonable” bonuses, while extremes can approximate the less helpful shortest-path option.

2.1.8 Why reward-respecting helps planning (and limits)

By construction, reward-respecting subtasks optimize the *same* cumulant as the main task and terminate in occasionally high-value states, so their models tend to increase the backed-up target $\hat{r} + \hat{v} \circ \hat{n}$ in those states — exactly what AVI’s maximization seeks. This explains the consistent planning gain observed. However, the approach also has significant limitations including:

- search-control matters
- the option-critic’s on-policy nature can produce models accurate only on visited regions
- while expectation models pair well with linear values, richer models may be needed for highly non-linear approximations.

These limitations can all be addressed in various ways; here rather than patching the reward-respecting idea we will opt to generalize it significantly.

2.1.9 Minimal recipe (from the paper) to reproduce the pipeline

A quick re-cap of the methodology from [1] would look like:

1. **Specify subtasks as GVFs** with $c = R$ and z_i given by the bonus-weight swap; pick features x_i whose weights w_i vary over time.
2. **Learn options off-policy** with the unified actor-critic update using the generic TD error, from behavior policy μ .
3. **Learn expectation models** for actions and options with TD targets for $\hat{r}$ and each row of $\hat{n}$.
4. **Plan by AVI** using backed-up values $\hat{r} + \hat{v}(\hat{n})$, counting each evaluation as one look-ahead operation.

The key point evidently underlying these steps is: When subtasks preserve the main reward and bias termination toward occasionally high-value states, their learned option models systematically increase AVI’s backed-up targets where it matters, causing these options to be selected more often in the planning max and thereby accelerating convergence – an effect supported by a single, compact off-policy learning framework for options and models.

2.2 From Reward-Respecting to Cone-Dominant Subtasks (CDS)

1

We now explain how to generalize the ”reward-respecting” concept to something a bit broader, which we call cone-dominant subtasks (CDS). Later on we will extend this even further into a Pareto-dominance based variant (PDS), but for a first presentation, we’ll stick with CDS. These variations work on a set of motives rather than an original goal, so they are most interesting when paired with a neural network or other module that decomposes a goal into motives (an MDN or Motive Decomposition Network), which

¹While we sometimes use the language of reward here for convenience and continuity with the RL literature, in fact statements in this section carry over verbatim to monotone, translation-equivariant risk functionals (e.g., entropic risk, CVaR, or whatever) by replacing the backed-up value $B_o(x; w)$ with its risk-evaluated version $B_o^\rho(x; w)$ and applying CDS/PDS to B_o^ρ . As we be clear later when we formalize these notions categorially, the approach presented here is extremely general and not at all restricted to expected reward maximization.

can then be co-trained with networks enacting cone-dominance or pareto-dominance.

We work in the same ?expectation?model + linear slice? setting as before, but we do not assume that the agent?s top-level objective is expected reward. Let J be any evaluation functional on future histories (expected discounted reward is a special case within the GVF view). Each action/option o exports an expectation summary $(\hat{r}, \hat{n})$, where $\hat{r}(x, o)$ denotes the immediate *increment to J* (a generic cumulant/utility term) and $\hat{n}(x, o)$ is the expected successor-feature summary of termination. For a weight vector w drawn from a motive set W (potentially induced by an MDN), the planner uses the backed-up score

$$B_o(x; w) := \hat{r}(x, o) + w^\top \hat{n}(x, o).$$

Cone-Dominant Subtasks (CDS) admit o precisely when $B_o - B_{\text{base}}$ is non-negative uniformly over W ; we will also give a Pareto variant (PDS) that requires improvement on a finite MDN-induced cover W_{ref} . This framing covers reward maximization but equally accommodates costs, constraints, risk measures, or multi-criteria utilities without changing the algebra.

Motive decomposition via a co-learned MDN. While the reward-respecting approach works with a single fixed weight, in CDS (and PDS) the idea is that the overall objective may be expressed as a blend of one or more top-level *motives* (externally supplied utility, auxiliary drives, risk, etc.), each treated as a component of the objective. Each motive can, in turn, be decomposed into sub-aspects (e.g., correctness, groundedness, safety, novelty). Parts of this decomposition may be specified at setup time (by humans or other AI components), but in many cases much or all of it should be learned by a Motive Decomposition Network (MDN): a neural (or other adaptive) module that maps state/context to a structured, low-dimensional set of sub-aspect coordinates and associated weights, subject to optional constraints, principles or anchors. The MDN is co-trained with the CDS machinery that learns $(\hat{r}, \hat{n})$ and performs planning, so that decomposition adapts to the same data stream and planning losses. Where appropriate, internal regularization (e.g., sparsity, orthogonality/independence penalties, group or disentangling terms) biases learned representations toward factors that respect the decomposition of reward into motives and motives into sub-aspects, improving alignment between the MDN?s sub-aspects and the option models that CDS selects.

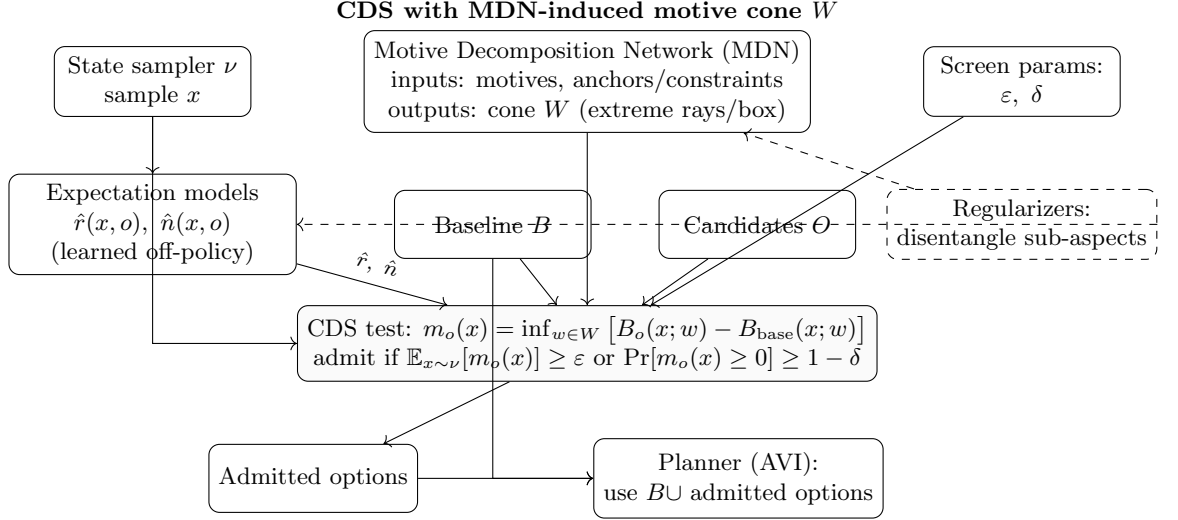


Figure 1: Cone-dominant screening (CDS). The MDN factorizes motives and induces a cone W of plausible weights. For each candidate o , the CDS block computes worst-case margin $m_o(x)$ against the baseline B across all $w \in W$ and admits options passing the distributional screen. Admitted options augment B for AVI. Dashed arrows indicate co-training/regularization that biases learned representations to respect the motive/sub-aspect decomposition.

2.2.1 Motive cones and the CDS objective

Meta-level motives (externally supplied utility, auxiliary drives, risk, etc.) are represented by linear evaluators $v_w(\cdot) = w^\top(\cdot)$. Let $W \subset \mathbb{R}^d$ be a closed convex cone of plausible weights (e.g., the positive hull of currently active motives, or a box around the present main-task weight). In the MDN-enhanced setting, top-level motives yield evaluators whose effective weights w may be *induced* by an MDN factorization (e.g., $w = Az$ for sub-aspects z produced by the MDN under given anchors/constraints); we then take W to be the positive hull of the admissible w produced by the MDN together with any setup-time anchors.

Given a baseline set of generators B (typically the primitive actions and any already-selected options), define the baseline backed-up value

$$B_{\text{base}}(x; w) := \max_{b \in B} (\hat{r}(x, b) + w^\top \hat{n}(x, b)).$$

Definition (statewise cone dominance). An option o is W -cone domi-

nant at x with margin $\varepsilon \geq 0$ if

$$m_o(x) := \inf_{w \in W} (B_o(x; w) - B_{\text{base}}(x; w)) \geq \varepsilon.$$

Intuitively, o beats the best thing you can already do, for every motive in W .

Distributional form. For a planning/update state distribution ν , require $\mathbb{E}_{x \sim \nu}[m_o(x)] \geq \varepsilon$ or $\Pr_{x \sim \nu}[m_o(x) \geq 0] \geq 1 - \delta$. When W is MDN-induced, the same tests apply while sampling w either from the MDN's current admissible cone (e.g., extreme rays/box bounds implied by anchors) or via stochastic draws over its sub-aspect outputs mapped into w .

2.2.2 Dual-cone view and efficient evaluation

Write each option's backup vector $b_o(x) := (\hat{r}(x, o), \hat{n}(x, o)) \in \mathbb{R}^{1+d}$ and each evaluator as $\xi_w := (1, w) \in \mathbb{R}^{1+d}$. Then

$$B_o(x; w) = \langle b_o(x), \xi_w \rangle.$$

Let $K := \text{cone}\{\xi_w : w \in W\}$ and $K^* := \{y : \langle y, \xi \rangle \geq 0, \forall \xi \in K\}$. If $b_o(x) - b_{\text{base}}(x) \in K^*$, then $m_o(x) \geq 0$. Thus CDS is a dual-cone dominance test on backup vectors. This holds unchanged when W is the cone induced by an MDN factorization; only the description of W changes.

2.2.3 Quantale formulation and monotonicity

For deriving theoretical properties of CDS later on, we will benefit from presenting CDS in the context of a quantale structure (with an appropriate commutative algebraic structure).

Let $V = \mathbb{R}^X$ with pointwise order, and K be the set of monotone W -continuous maps $T : V \rightarrow V$ under join (pointwise sup) and product (composition). Each option induces T_o with $[T_o v](x) = \hat{r}(x, o) + v(\hat{n}(x, o))$. AVI's Bellman-like operator is $B_G := \bigvee_{g \in G} T_g$ for a generator set G .

Proposition (restricted dominance). Fix a cone W . If o satisfies $m_o(x) \geq 0$ for all x in the planner's update support, then for all $w \in W$,

$$[B_{B \cup \{o\}} \hat{v}(\cdot; w)](x) \geq [B_B \hat{v}(\cdot; w)](x),$$

hence the Kleene sequence for AVI with $B \cup \{o\}$ dominates that with B when evaluated on $\{\hat{v}(\cdot; w) : w \in W\}$.

2.2.4 Reward-respecting as a special case of CDS

Sutton’s feature-attainment subtask keeps the cumulant equal to the true reward and changes only the stopping value by swapping weight w_i for an optimistic bonus $\bar{w}_i$. Let $w^{(0)}$ be the current main-task weight and $w^{(i)}$ the ”bonus-swapped” weight ($w_i^{(i)} := \bar{w}_i$, $w_j^{(i)} := w_j^{(0)} \forall j \neq i$). Take the motive set

$$W := \text{cone}\{w^{(0)}, w^{(i)}\}.$$

Any option o that solves the reward-respecting GVF for feature i typically satisfies $m_o(x) \geq 0$ for a large measure of x , because o jointly accumulates externally supplied utility and terminates in states where feature i is high and value is occasionally high. Thus reward-respecting corresponds to CDS on a two-ray cone and is a strict special case.

2.2.5 Robust margins and certificates (model error)

For any $w \in W \subseteq \{w : \|w\|_1 \leq kWk_{\max,1}\}$,

$$B_o(x; w) - B_{\text{base}}(x; w) \geq \hat{B}_o(x; w) - \hat{B}_{\text{base}}(x; w) - \varepsilon_r^\Delta(x, o) - kWk_{\max,1} \varepsilon_n^\Delta(x, o),$$

where ε^Δ collects worst-case differences versus all $b \in B$. Using confidence radii for ε gives high-probability screening. When W is MDN-induced, MDN uncertainty (e.g., over sub-aspect posteriors or anchors) can be folded into these radii or into conservative box/ellipsoid bounds that define the admissible cone used in the same certificate.

2.2.6 A practical screening algorithm (CDS-STOMP)

Inputs: candidate options O , baseline B , cone W , state sampler ν , thresholds ε, δ .

Step 1: Learn models. Off-policy, learn $(\hat{r}, \hat{n})$ for $B \cup O$ with TD-style updates. *If using an MDN, co-train it here as part of the same loop: learn the motive-to-sub-aspect decomposition (with any anchors/constraints) while applying regularizers that promote disentangled sub-aspects aligned with the option models that CDS admits.*

Step 2: Estimate robust margins. Draw $x \sim \nu$; for each $o \in O$ compute robust margin; aggregate by $\bar{m}_o := \mathbb{E}_\nu[\cdot]$ or $\Pr_\nu[\cdot]$. *If W is MDN-induced, compute margins either against the MDN’s current extreme rays/box bounds or by Monte Carlo draws of w from the MDN and taking empirical infima.*

Step 3: Select. Keep $\{o : \bar{m}_o \geq \varepsilon\}$ or top- k by $\bar{m}_o$.

Step 4: Plan. Run AVI with $O(x) \leftarrow B \cup$ selected options.

2.2.7 Computational notes

- Cost: For a box W , computing a statewise margin is $O(d \cdot |B|)$ per (x, o) .
- Search-control: Use a mixture of replayed interesting states and a uniform cover of the feature support to stabilize ν .
- Warm starts: Maintain running sign estimates of $\Delta\hat{n}$ to avoid re-computing the infimum unnecessarily.
- *MDN regularization and stability*: lightweight penalties (e.g., $L_1/L_{2,1}$ sparsity, orthogonality/total-correlation terms, or small KL priors to anchors) help keep the learned sub-aspects disentangled and stable as CDS updates the option library; these sit alongside the usual model and value residuals without changing the CDS test.

2.2.8 Relations to existing paradigms

2

Robust control enforces worst-case performance over model sets; CDS enforces worst-case *planner advantage* over motive weights. Successor features and generalized policy improvement guarantee transfer across *linear mixtures of evaluation features*; CDS applies the analogous idea to option models, selecting those whose $\hat{r} + \hat{v} \circ \hat{n}$ wins uniformly over W , irrespective of whether $\hat{r}$ encodes reward, cost, constraint slack, risk, or any other cumulant induced by the chosen functional J .

2.2.9 Worked micro-example (one-state certificate)

Suppose from x two generators are available: a baseline b and a candidate o . Let $\Delta\hat{r} := \hat{r}(x, o) - \hat{r}(x, b) = +2.0$. Let $\Delta\hat{n}$ be nonzero only on feature h with $\Delta\hat{n}_h = -0.04$. With $\gamma = 0.99$, this matches a "shorter vs. longer" trade-off in two-room where the candidate avoids a few -1 steps but is four steps longer. With $W = \{w : 0 \leq w_h \leq 80\}$, the box-cone infimum is attained at $w_h^{\max} = 80$:

$$m_o(x) \geq 2.0 + (-0.04) \times 80 = -1.2.$$

²With a risk functional ρ in place of expectation, CDS/PDS relate directly to risk-sensitive RL (exponential-utility and coherent-risk formulations): the dominance tests simply compare risk-evaluated backed-up values componentwise.

If the -1 band is three steps wide so that $\Delta R \approx 2.88$, then

$$m_o(x) \geq 2.88 + (-0.04) \times 80 = -0.32,$$

and the exact LP typically yields a nonnegative exact margin; this mirrors why the roundabout, reward-respecting hallway option wins the planner’s max more often.

2.2.10 Take-home

CDS keeps the core of Sutton et al’s method – linear values, expectation models, and AVI – unchanged, and adds a robust, computable selection rule for options: keep those that raise the backed-up value against your whole cone of plausible motives. *When motives are exposed as top-level components and further factorized by a co-learned MDN into sub-aspects, CDS uses the MDN-induced cone W and the regularized, disentangled representations to admit options that robustly improve planning across that structured space of motives.*

2.3 Pareto-Based Admission: A Weaker Alternative to Cone-Dominance

Now we extend the CDS perspective a little further, relaxing cone-dominant screening (which demands uniform improvement across an entire motive cone W) to a *Pareto*-based rule over a finite cover of the cone. The details become slightly intricate, but the idea is simple: keep a candidate generator if, when we look at a small set of representative weights, its backed-up value improves at least one representative without making the others unacceptably worse. We also include budgeted/coverage variants that permit small regressions on some objectives while still insisting on meaningful gains on others. The result is a more permissive admission rule that often admits diverse options earlier, while preserving clean algebraic properties (join safety, stability under composition) and a practical certification routine.

Operationally, the approach we take is to stack per-weight backed-up values into a vector and compare vectors by the componentwise (Pareto) order. A candidate passes if its vector dominates the baseline vector (with optional budgets). We then recast this in a product-quantale view so the same join/composition facts used for cone-dominance carry over. Finally, we give error-aware certificates and a simple screening recipe, and note cases where Pareto admission outperforms cone-dominance in practice (e.g., when different weights favor different options across a broad cone).

up values at state x :

$$B_o(x) = (B_o(x; w^{(1)}), \dots, B_o(x; w^{(K)})) \in \mathbb{R}^K, \quad B_o(x; w) = \hat{r}_o(x) + w^\top \hat{n}_o(x).$$

Let $B_{\text{base}}(x) = (\max_{b \in B} B_b(x; w^{(k)}))_{k=1}^K$ be the baseline vector (componentwise max over admitted generators B). Use the product (Pareto) order on $\mathbb{R}^K$: for vectors u, v , write $u \leq_P v$ iff $u_k \leq v_k$ for all k , and $u <_P v$ iff $u \leq_P v$ and $u_j < v_j$ for some j .

Definition (Pareto-dominant subtask, PDS). A candidate o is Pareto-dominant at x if $B_o(x) \geq_P B_{\text{base}}(x)$ and $B_o(x) \neq B_{\text{base}}(x)$. Statewise admission requires the condition on a region Ω (pointwise or in expectation over a sampler ν).

Epsilon/budgeted relaxation (weaker than CDS). To avoid brittleness and allow slight degradations, admit o if for some $(\varepsilon, \rho) \geq 0$,

$$B_o(x) \geq_P B_{\text{base}}(x) - \rho \mathbf{1} \quad \text{and} \quad \exists k : B_o(x; w^{(k)}) \geq B_{\text{base}}(x; w^{(k)}) + \varepsilon,$$

for $x \in \Omega$ (pointwise or in expectation). The parameter ρ is a tolerated downside budget per weight; ε is a significance margin. Setting $(\varepsilon, \rho) = (0, 0)$ recovers strict PDS; CDS over W_{ref} implies strict PDS, but not conversely.

Coverage relaxation (fractional Pareto). Instead of a single improving component, one may require improvement on at least a fraction $q \in (0, 1]$ of the weights:

$$|\{k : B_o(x; w^{(k)}) \geq B_{\text{base}}(x; w^{(k)}) + \varepsilon\}| \geq qK, \quad B_o(x; w^{(k)}) \geq B_{\text{base}}(x; w^{(k)}) - \rho, \quad \forall k.$$

2.3.2 Quantale formulation in product order

Let $V = \{v : X \rightarrow \mathbb{R}\}$ and V^K be the K -fold product with product order. Define for each generator o a componentwise transformer $T_o : V^K \rightarrow V^K$ by

$$[T_o v]_k(x) = \hat{r}_o(x) + v_k(\hat{n}_o(x)), \quad k = 1, \dots, K.$$

Let $B_G = \bigvee_{g \in G} T_g$ be the componentwise join (pointwise max per k). Then $(K^K, \otimes, \bigvee)$ with $\otimes$ given by componentwise composition is a unital quantale.

Proposition (PDS as dominance in the product quantale). At a fixed $v = v_{W_{\text{ref}}} = (v_{w^{(1)}}, \dots, v_{w^{(K)}}) \in V^K$, strict PDS at x is equivalent to

$$T_o v(x) \geq_P B_B v(x) \quad \text{with strict inequality in some component.}$$

Epsilon/budgeted PDS is equivalent to $T_o v \geq_P B_B v - \rho \mathbf{1}$ and strict improvement by ε in at least one component.

2.3.3 Compositional stability and joins

The following mirror the CDS properties but in product order.

Join safety. For any extra set J ,

$$B_{B \cup \{o\} \cup J} v = (B_{B \cup J} \vee T_o) v \geq_P B_{B \cup J} v.$$

Thus adding o cannot worsen any component of the planner's backed-up target.

Suffix stability with budget. Let $U \in K^K$ act componentwise as above (same $\hat{r}, \hat{n}$ shared across components). If $T_o v \geq_P B_B v - \rho \mathbf{1}$ on the cover, then

$$(U \otimes T_o) v = U(T_o v) \geq_P U(B_B v - \rho \mathbf{1}) = U(B_B v) - \rho \mathbf{1},$$

by translation-equivariance w.r.t. constants; strict improvement in at least one component propagates by monotonicity.

Prefix stability under slice invariance. Assume cover/slice invariance: for each k and $V \in K$ there exists k' with $V v_{w(k)} = v_{w(k')}$. If $T_o v \geq_P B_B v - \rho \mathbf{1}$ on the cover, then

$$(T_o \otimes V) v \geq_P (B_B \otimes V) v - \rho \mathbf{1}.$$

2.3.4 Residuation and weakest-sufficient multiobjective decomposition

Because K^K is a product quantale (componentwise order, join, and composition), residuals exist componentwise:

$$S/B = (S_k/B_k)_{k=1}^K, \quad A \setminus S = (A_k \setminus S_k)_{k=1}^K.$$

If a multiobjective spec S assigns per-weight lower bounds on Ω , the weakest prefix $A^* = S/B$ achieves S when composed with the suffix B , componentwise minimal in the product order. In a hybrid architecture, LO constructs S and admissible guards; EO searches skeletons approximating A^* ; GO fits $(\hat{r}, \hat{n})$ per component.

2.3.5 Certificates with estimation error

Suppose a module exports $(\hat{r}_o, \hat{n}_o)$ with uniform envelopes

$$|\hat{r}_o - r_o| \leq \varepsilon_r, \quad \|\hat{n}_o - n_o\|_\infty \leq \varepsilon_n.$$

Then for each component

$$B_o(x; w^{(k)}) \geq \hat{B}_o(x; w^{(k)}) - \varepsilon_r - \|w^{(k)}\|_1 \varepsilon_n,$$

and the vector inequality becomes

$$B_o(x) \geq_P \hat{B}_o(x) - \delta(x), \quad \delta_k(x) = \varepsilon_r + \|w^{(k)}\|_1 \varepsilon_n.$$

When the weight cover is MDN-induced, uncertainty from the MDN (e.g., dispersion of sub-aspect posteriors or anchor variability) can be folded into δ (per-component slack) or handled by a conservative choice of W_{ref} (e.g., extreme rays/ellipsoids), yielding the same budgeted certificate forms. A conservative PDS certificate is therefore

$$\hat{B}_o(x) \geq_P B_{\text{base}}(x) + \varepsilon \mathbf{1} + \delta(x),$$

or, for budgeted PDS, replace the right-hand side by $B_{\text{base}}(x) - \rho \mathbf{1} + \varepsilon \mathbf{e}_J$ with $\mathbf{e}_J$ the indicator of improved components.

2.3.6 Practically screening and keeping the library small

- **PDS screen.** For $x \sim \nu$ and $k = 1..K$, compute $\Delta_k(x, o) = \hat{B}_o(x; w^{(k)}) - B_{\text{base}}(x; w^{(k)})$. Admit o if the budgeted/coverage criterion holds on Ω .
- **Compression.** Maintain a Pareto cover of admitted generators: discard any o whose vector $B_o(x)$ is dominated by a convex combination of others on a cover of Ω ; or impose ε -nondominance and a cap on library size.
- **Scheduling.** Use *adaptive* W_{ref} : emphasize weights seen in MetaMo’s mixture or as proposed by the MDN’s current sub-aspect-to-weight mapping; expand W_{ref} if instability is detected. Co-train the MDN in the same loop as $(\hat{r}, \hat{n})$ and planning so that the decomposition and the option models remain aligned.

2.3.7 Certificates without circularity: calibration and data-splitting

Digging deeper into the devilish details of these certificates, it might seem at first there is some circularity involved here. The certificates that account for model uncertainty depend on knowing bounds on the modeling errors. But in practice, these bounds would typically be estimated from the same data used to train the models, creating potential circularity in the guarantees. However this is easily bypassed via appropriate implementation strategies.

Split data and shadow models. To avoid circularity, we maintain a small *calibration buffer* disjoint from the data used to fit $(\hat{r}, \hat{n})$ and the MDN. Per option o we estimate envelopes $\varepsilon_r, \varepsilon_n$ on this buffer (or via cross-fitting), and we compute certificate margins with these held-out slacks only.

Distribution-free calibration (conformal margins). Let $\hat{m}_o(x)$ be the empirical CDS margin computed on the active set/covers. We form conformalized margins

$$m_o^{\text{conf}}(x) := \hat{m}_o(x) - q_{1-\delta}(\{\hat{m}_o(x_i) - m_o(x_i)\}_{x_i \in \text{calib}}),$$

where $q_{1-\delta}$ is the $(1 - \delta)$ empirical quantile of residuals on the calibration buffer. Then $\Pr[m_o^{\text{conf}}(X) \geq 0] \geq 1 - \delta$ holds marginally without distributional assumptions. The same componentwise construction applies to PDS.

Online updates with precision nodes (PC-friendly). In predictive-coding implementations (like we will review below), per-head precisions act as on-line variance estimators. We can map these to slacks $(\varepsilon_r, \varepsilon_n)$ via self-normalized bounds, and still reserve a small, periodically refreshed calibration buffer to keep guarantees independent of the training residuals.

Summary of guarantees. By (i) clean train/calibration separation or cross-fitting, (ii) conservative conversion of variances to slacks, and (iii) conformal back-off on margins, the resulting certificates are not circular: they explicitly *do not reuse* the same samples for both fitting and validating bounds, and they degrade gracefully under drift by periodic recalibration.

2.4 What we mean by a "decision transformer"

We have casually used the term "transformer" above and will use it again in the following; to avoid terminological confusion, it seems a good time now to explicitly lay out what we are talking about.

In this paper a *decision transformer* (sometimes just "transformer") is the mathematical object that a subgoal (an option, macro, rule, or program) exposes to the planner. It is not the Transformer neural architecture (nor a giant robot that can turn into a car or planet!); rather, it is the *effect* of a decision on future value, packaged as a function that turns a downstream value function into a new value function that already accounts for taking that decision now.

Think of it like an adaptor with a simple API:

- **input:** a downstream value function v saying how good states are *after* the subgoal finishes;
- **internal:** the subgoal’s expectation model (its expected immediate payoff and where it tends to land);
- **output:** a new value function $x \mapsto$ “payoff if I run this subgoal from x and then evaluate v where I land”.

Because every subgoal provides this same interface, the planner can (i) compare many subgoals (take a max/join), and (ii) chain them (compose), without caring how they were built (neural controller, logical macro, evolutionary program). CDS/PDS simply compare these outputs under motive weights.

2.4.1 Formal definition (single-slice CDS setting)

Let X be the state-feature space and $V = \mathbb{R}^X$ the set of value functions ordered pointwise. Fix a linear value slice $v_w(x) = w^\top \phi(x)$ with features ϕ and motive weight w . A *decision transformer* induced by a subgoal/option o is the monotone endomap

$$T_o : V \rightarrow V, \quad [T_o v](x) := \hat{r}(x, o) + v(\hat{n}(x, o)),$$

where $(\hat{r}, \hat{n})$ is the subgoal’s expectation model on the slice: $\hat{r}(x, o)$ is expected immediate payoff (cumulant) while $\hat{n}(x, o)$ is the expected successor feature-state (termination summary). Evaluated on the slice v_w , the backed-up value is

$$[T_o v_w](x) = \hat{r}(x, o) + w^\top \hat{n}(x, o) \equiv B_o(x; w).$$

A set of subgoals G induces the planner join $B_G = \bigvee_{g \in G} T_g$, where $(B_G v)(x) = \max_{g \in G} [T_g v](x)$. Sequential execution corresponds to composition: running o_1 then o_2 is $T_{o_2} \circ T_{o_1}$. The identity I represents “do nothing” (immediate stop).

Properties used in this paper.

- *Monotonicity:* $v \leq u \Rightarrow T_o v \leq T_o u$ (pointwise), hence joins and compositions form a residuated quantale on the generated family $K \subseteq V^V$.
- *CDS screening:* given a baseline set $B \subseteq K$, the cone-dominant margin at x is

$$m_o(x) := \inf_{w \in W} \left([T_o v_w](x) - [B_B v_w](x) \right),$$

and o is admitted if $m_o(x) \geq 0$ in distribution (or with high probability) on the update sampler.

- *Weakest decomposition*: residuation S/B and $A \setminus S$ (when they exist) give weakest prefix/suffix transformers consistent with a plan target S .

2.4.2 Formal definition (vectorized PDS setting)

For a finite cover $W_{\text{ref}} = \{w^{(k)}\}_{k=1}^K$ (MDN-induced), define the product space V^K with componentwise order and the componentwise transformer

$$[T_o v]_k(x) := \hat{r}(x, o) + v_k(\hat{n}(x, o)), \quad k = 1, \dots, K.$$

The vector of backed-up values for o at x is $B_o(x) = (B_o(x; w^{(1)}), \dots, B_o(x; w^{(K)}))$. PDS admission checks componentwise Pareto dominance (possibly with budgets/coverage). Composition and joins are taken componentwise, yielding the same algebraic structure in V^K .

2.4.3 Relation to implementation (neural, logical, evolutionary)

As we will elaborate in later sections, the definition of decision transformer is agnostic to how o is built, and can be applied in the case of e.g. logical or evolutionary methods as well as neural methods.

2.4.4 Why we call it a "transformer"

It *transforms* any downstream value v into a new upstream value $T_o v$ by inserting the subgoal's one-step model. This unifies "what a decision does to value" for all modalities, makes comparisons (joins) and chaining (composition) precise, and is exactly what CDS/PDS and residuation operate on.

2.5 Practical tractability and principled covers for CDS/PDS

The algorithms described above are schematic rather than fully detailed, and it may be worth clarifying how and why they can be realized in a computationally tractable way.

Normalized cones and support functions (why inf is well-defined and cheap in common cases). For CDS we evaluate, at a state x and candidate o with baseline B ,

$$m_o(x; W) := \inf_{w \in W} \Delta r(x, o) + w^\top \Delta n(x, o), \quad \Delta r = \hat{r}(x, o) - \hat{r}_{\text{base}}(x), \quad \Delta n = \hat{n}(x, o) - \hat{n}_{\text{base}}(x).$$

To avoid the trivial $-\infty$ on unbounded cones, we work with a *normalized base* of the cone,

$$\mathcal{B}(W) := \{w \in W : \alpha^\top w = 1, Gw \leq h\},$$

where α selects a scale (e.g., $\mathbf{1}$ on nonnegative cones). Then the CDS margin reduces to a support-function evaluation

$$m_o(x; W) = \Delta r(x, o) + \inf_{w \in \mathcal{B}(W)} w^\top \Delta n(x, o) = \Delta r(x, o) - h_{\mathcal{B}(W)}(-\Delta n(x, o)).$$

This form gives concrete, tractable special cases:

- **Box (orthant-slice):** $\mathcal{B}(W) = \{w \geq 0, \mathbf{1}^\top w = 1, \ell \leq w \leq u\}$ yields the closed-form $m_o = \Delta r - \sum_i (-\Delta n_i)_+^\downarrow \cdot \tilde{u}_i$ after sorting and saturating bounds (linear-time in d).
- **Polyhedral cones:** $\mathcal{B}(W) = \{Aw \leq b, Cw = c\}$ gives an LP $\min \Delta n^\top w$ s.t. $Aw \leq b, Cw = c$. Interior-point solvers are $O(d^3)$ per solve but *warm-start well* across nearby states; in practice we reuse the last active set and see near-constant amortized time.
- **Second-order/ellipsoidal slices:** $\mathcal{B}(W) = \{w : \|Q^{1/2}(w - \mu)\|_2 \leq 1, \alpha^\top w = 1\}$ gives an SOCP with small dimension.

Anytime evaluation and active rays (when to stop early). To make things practically tractable, we can use a three-tier gate per candidate o :

1. *Cached rays test:* evaluate $\Delta r + w_j^\top \Delta n$ on a small cache $\{w_j\}_{j=1}^J$ (extreme rays seen before). If the minimum exceeds the threshold by a margin κ , admit immediately; if it falls below by κ , reject.
2. *Cutting-plane refinement:* if inconclusive, solve the normalized LP/-SOCP once to get w^* , add w^* to the cache, and re-test. Stop when improvement $< \epsilon$ or a fixed budget of cuts is reached.
3. *Projected subgradient (borderline only):* run T steps of $w \leftarrow \Pi_{\mathcal{B}(W)}(w - \eta \Delta n)$; the best iterate gives an ϵ -accurate lower bound with $O(1/\sqrt{T})$ error decay. We use this only when the margin is close to zero.

This policy ensures exactness for polyhedral slices (via LP) and certified ϵ -accuracy for general convex slices, while keeping most evaluations in the cheap cached tier.

Principled W_{ref} (PDS covers) with error control. For PDS we choose a finite cover $W_{\text{ref}} \subset \mathcal{B}(W)$. Because $B_o(x; w)$ is *linear* in w , if W_{ref} is an ϵ -net of $\mathcal{B}(W)$ under a norm $\|\cdot\|$, then for any o ,

$$\left| \inf_{w \in \mathcal{B}(W)} (\Delta r + w^\top \Delta n) - \min_{w \in W_{\text{ref}}} (\Delta r + w^\top \Delta n) \right| \leq \epsilon \|\Delta n\|_*,$$

where $\|\cdot\|_*$ is the dual norm. We operationalize this with an MDN-driven, violation-guided routine:

1. Initialize W_{ref} with MDN anchors/extreme rays (or k -center points in the MDN metric).
2. During screening, whenever an LP/SOCP (or subgradient) finds a w^* that beats all W_{ref} by more than $\epsilon \|\Delta n\|_*$, add w^* to W_{ref} .
3. Periodically compress W_{ref} by greedy max-min or by removing Pareto-redundant points.

This yields *adaptive PDS* with a tunable error ϵ and small, data-dependent $|W_{\text{ref}}|$.

Optional evolutionary solver (fits the same gate). As an alternative to LP/SOCP, an evolutionary algorithm or EDA can be used to minimize $w^\top \Delta n$ over $\mathcal{B}(W)$ by sampling normalized weights and focusing mass on low values; its best sample acts as the new w^* for the cutting-plane cache. Because CDS/PDS gates are local and streaming, such evolutionary refinement interleaves cleanly with PC-style local learning.

2.6 MDN co-learning: explicit losses, regularization, and pathology control

It may also be worth further clarifying how the co-learning of the Motive Decomposition Network can be done in a tractable and generally non-pathological way.

What the MDN outputs. We parameterize the motive geometry by θ : either a normalized polyhedral base $\mathcal{B}(W_\theta) = \{w : A_\theta w \leq b_\theta, \alpha^\top w = 1\}$ (CDS) or a finite cover $W_{\text{ref}, \theta} = \{w_\theta^{(k)}\}_{k=1}^K$ (PDS), together with latent sub-aspects z (anchors).

How CDS/PDS residuals train the MDN. For CDS we minimize a hinge on the robust margin (approximated by the active set $\mathcal{S}$ of rays/cuts found so far):

$$\mathcal{L}_{\text{CDS}}(\theta) = \mathbb{E}_{(x,o)} \rho\left(\varepsilon - \min_{w \in \mathcal{S}_\theta(x,o)} (\Delta r(x,o) + w^\top \Delta n(x,o))\right),$$

with $\rho(u) = \log(1 + e^u)$ or $(u)_+$. For PDS we use a budgeted/coverage hinge over the MDN cover:

$$\mathcal{L}_{\text{PDS}}(\theta) = \mathbb{E}_{(x,o)} \max\left\{ \max_k \rho(\rho_\downarrow - \Delta_k), \varepsilon_\uparrow - \max_k \Delta_k \right\},$$

where $\Delta_k = \Delta r + w_\theta^{(k)\top} \Delta n$, $\rho_\downarrow$ is the downside budget, and $\varepsilon_\uparrow$ the required improvement. Gradients flow through the selected w (the minimizer or the argmax k). In the polyhedral CDS case, the minimizer lies on a face/vertex; its subgradient w.r.t. θ is obtained by implicit differentiation of the active constraints.

Regularization (and why). We include:

- **Disentanglement:** a total-correlation penalty on z and orthogonality of anchor directions encourages near-independent sub-aspects, reducing collinearity in w so that margins are identifiable and stable.
- **Anchor KLS (optional):** weak pulls toward human/AI-provided anchors encode prior semantics without hard constraints.
- **Stability/smoothness:** a temporal TV penalty $\|\mathcal{B}(W_{\theta_{t+1}}) - \mathcal{B}(W_{\theta_t})\|$ (or Hausdorff distance on covers) prevents thrashing and enforces the monotone tightening/refinement assumptions used in the theory.
- **Weakness/complexity:** a volume (or log-determinant) penalty on $\mathcal{B}(W_\theta)$ biases toward the weakest adequate motive slice, discouraging degenerate, overly wide cones that admit everything.

Together these terms avoid pathological decompositions by (i) keeping sub-aspects disentangled and identifiable, (ii) anchoring semantics when available, (iii) enforcing slow, monotone geometry drift, and (iv) preferring minimal slices consistent with observed improvements.

Training loop (coherent with the gate). At each step: update option heads $(\hat{r}, \hat{n})$; run the CDS/PDS gate with the cached rays/cover; collect residuals and active w ; backprop $\mathcal{L}_{\text{CDS/PDS}} + \lambda_{\text{reg}} \mathcal{R}$ into θ ; occasionally refresh the cache by one exact solve (LP/SOCP) per borderline option. This couples MDN learning exactly to the admission criterion it must support.

3 Synergies between CDS/PDS and Predictive Coding

The overall CDS/PDS framework, like the reward-respecting framework that inspired it, is fairly independent of what learning mechanism is used to realize it. The default approach given modern AI trends would be to use backpropagation-trained deep neural nets for the purpose, however this is far from the only alternative. We will explore here several other options – including non-neural approaches such as logical inference or evolutionary learning; and, in the current section, neural learning using predictive coding (PC).

We begin with a compact review of PC for RL, then move into an execution design and then into a deeper, layerwise integration. The overall aim is to make the max/inf primitives required by CDS/PDS local and streaming, and to turn the motive sets used by those screens into *learned* objects via a co-trained Motive Decomposition Network (MDN). This lets the geometry of motives (the cone W or the weight cover W_{ref}) co-evolve with the option library and the world model, instead of being a fixed external input.

We present PC for RL as a hierarchical generative model with local error nodes, precision-weighted updates, and action selection cast as inference. We discuss two specific active-PC approaches along these lines: Rao et al’s hypernetworked, hierarchical variant that generates lower-level transition and policy parameters (APC) [6], and Ororbia et al’s backprop-free dual-circuit variant that couples a world-model PC loop with an actor PC loop modulated by instrumental and epistemic signals (ActPC) [7]. These architectures help us define the exact interfaces we will use for integrating PC with CDS/PDS: a reward/value head $\hat{r}(x, o)$, successor-feature head $\hat{n}(x, o)$, candidate controllers o , and precision knobs that can themselves be treated as motives.

We then argue that PC is a good *execution engine* for CDS/PDS and sketches the concrete loop. The PC graph already computes a soft per-state max over backed-up values, and it can realize the robust inf over motives either in closed form (box cones) or by evaluating a few MDN-suggested extreme rays. The MDN is co-trained with the same residual stream that trains $(\hat{r}, \hat{n})$, so it learns a disentangled factorization of motives and induces W or W_{ref} that remain aligned with the evolving option models. We spell out the local residuals for CDS margins and PDS Pareto gaps, show how they attach to PC error nodes, and list practical disentanglement pressures (orthogonality, sparsity, total correlation, anchors, precision shaping).

We also note why – under streaming, nonstationary motives – PC’s local, precision-weighted credit assignment can outperform conventional backprop for this use case.

Finally, we show how to push the integration deeper by *interleaving* PC, MDN, and CDS/PDS throughout the hierarchy. Lower PC layers define internal rewards from surprise reduction in selected downstream errors; small, layer-local MDNs decompose those internal rewards into sub-aspects; and layer-local CDS/PDS screens admit latent-space options that measurably help. We describe one-step, all-layers execution, cross-layer coupling (e.g., APC-style hypernets), safeguards (precision gating by MDN confidence, anti-collapse shaping), and library compression. The result is a modular, anytime stack in which new skills surface bottom-up as soon as they reduce prediction error in the right directions, and are certified top-down by CDS or PDS with explicit, auditable certificates.

3.1 Brief Review of Predictive Coding for Reinforcement Learning

Predictive coding (PC) casts perception, learning, and action as approximate Bayesian inference in a generative model, implemented by local message passing on prediction errors. In RL, a PC agent maintains a predictive world model and selects actions to minimize a composite objective that balances exploitation (instrumental value) with exploration (epistemic value). This subsection reviews PC mechanics for RL and then summarizes two recent “active PC” formulations, using a common notation that will make it easy to interface with CDS/PDS in the next subsection.

Common notation. Let $o_t \in \mathcal{O}$ be observations, $s_t \in \mathbb{R}^{d_s}$ latent states, and $a_t \in \mathcal{A}$ actions. A time-local generative model is

$p_\theta(o_t | s_t) p_\theta(s_{t+1} | s_t, a_t)$, with recognition variables updated by PC messages to minimize F_t .

Each layer has activities x_ℓ with predictions $\hat{x}_\ell = f_\ell(W_\ell x_{\ell-1})$ and errors $\varepsilon_\ell = x_\ell - \hat{x}_\ell$. In continuous time (or inner-loop iterations), state/action/feature nodes descend a free-energy-like objective $F = \sum_\ell \frac{1}{2} \|\varepsilon_\ell\|^2$ (plus priors and control terms), while synapses update via local Hebbian terms such as $\Delta W_\ell \propto \varepsilon_\ell x_{\ell-1}^\top$. In control, one augments the model with optimality variables or priors so that maximizing entropy-regularized return is equivalent to posterior inference (planning-as-inference); active-inference variants minimize an expected-free-energy objective that decomposes into instrumental (cost) and epistemic (information-gain) parts.

Design patterns for PC in RL. The core approach we assume for PC-based RL is:

1. Predictive world model: learn $p_\theta(s_{t+1} \mid s_t, a_t)$ and $p_\theta(o_t \mid s_t)$ from prediction errors; value/reward heads attach to s_t .
2. Actions as controllable priors: gradient flows on action nodes implement policy improvement.
3. Precision (temperature) scheduling: precisions on error nodes modulate exploration/exploitation and stabilize learning.

Within this framework, one can combine amortized inference (fast recognition nets) with short PC correction loops for streaming, partially observed control.

We next briefly review two specific approaches to PC-for-RL from the recent literature.

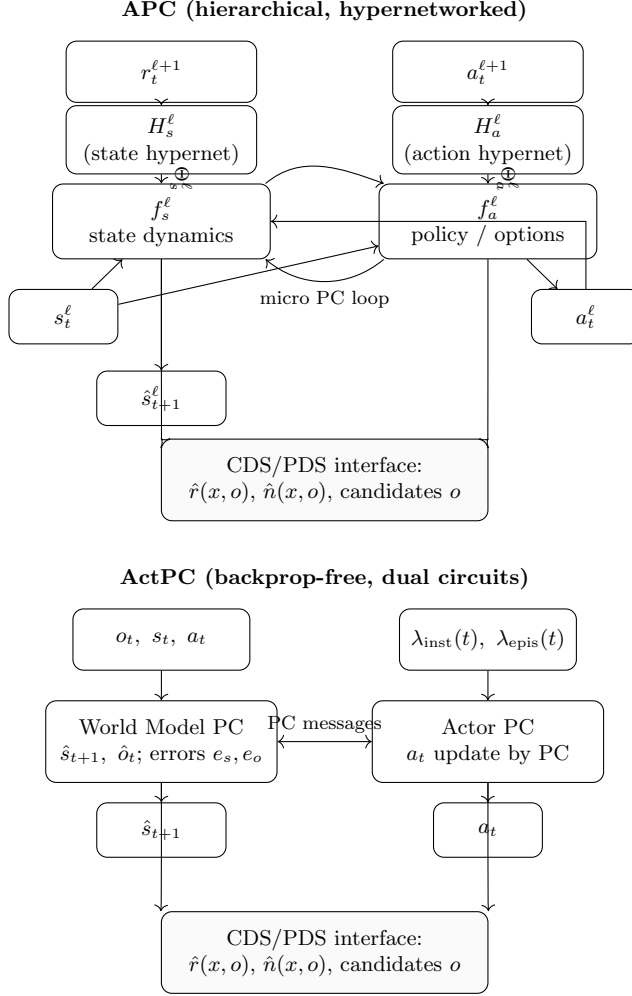


Figure 3: Two active predictive-coding (PC) formulations in a common RL-oriented view. Top: APC uses top-level embeddings to *generate* lower-level transition and policy parameters via hypernets, with micro (execution) and macro (abstraction) timescales. Bottom: ActPC couples a world-model PC circuit with an actor PC circuit, modulated by instrumental (λ_{inst}) and epistemic (λ_{epis}) signals. In both, the CDS/PDS interface consumes a reward/value head $\hat{r}(x, o)$, successor features $\hat{n}(x, o)$, and candidate options o .

Rao’s Active Predictive Coding (APC) in our notation. Rao et al [6] outline an approach we will abbreviate APC, which introduces a hierarchical, *state-action factorization* across abstraction levels $\ell = 1, \dots, L$. At level $\ell + 1$, a *state embedding* $r_t^{\ell+1}$ and an *action embedding* $a_t^{\ell+1}$ serve as hyper-conditions that *generate the parameters* of the lower-level transition and policy functions via hypernetworks:

$$\begin{aligned}\Theta_s^\ell &\leftarrow H_s^\ell(r_t^{\ell+1}), & \Theta_a^\ell &\leftarrow H_a^\ell(a_t^{\ell+1}), \\ s_{t+1}^\ell &\sim f_s^\ell(\cdot; \Theta_s^\ell)(s_t^\ell, a_t^\ell), & a_t^\ell &\sim f_a^\ell(\cdot; \Theta_a^\ell)(s_t^\ell).\end{aligned}$$

Here f_s^ℓ (task-invariant state dynamics) and f_a^ℓ (task-dependent policy/options) are typically RNNs at the lower level, with horizontal PC message passing between state and action subnetworks at the same level. A higher level runs for macro-steps; within each macro-step the lower level runs T_1 micro-steps before handing control back up. Learning couples *self-supervised* prediction losses for f_s^ℓ with *RL losses* (or optimality priors) for f_a^ℓ ; the hypernets H_s^ℓ, H_a^ℓ enable *compositional re-use* of lower-level skills and part-whole structure (in vision) under a single predictive-coding umbrella.

The key takeaways for RL from this approach may be:

- explicit separation of task-invariant state dynamics from task-dependent policy modules
- hierarchical planning by generating option/policy parameters via hypernetworks
- a two-timescale PC loop (micro for execution, macro for abstraction).

These will map cleanly to CDS/PDS admission over options produced at level ℓ , as we will show a little later.

Ororbia’s Active Predicting Coding (ActPC) in our notation. Ororbia et al [7] give a slightly different approach to PC-for-RL which we will abbreviate ActPC. Act PC builds a backpropagation-free active-inference agent entirely from predictive-coding circuits (“neural generative coding”), via instantiating two interacting PC modules: a *world model* (transition+observation) and an *actor/policy circuit*. At each step, local PC updates balance two internally generated scalars: an *instrumental* signal (drive to reduce expected cost / increase reward) and an *epistemic* signal (drive to reduce uncertainty

via information gain). Denote these transient modulators by $\lambda_{\text{inst}}(t)$ and $\lambda_{\text{epis}}(t)$. A simple composite loss is

$$\mathcal{J}_{\text{ActPC}} = \sum_t \left[\lambda_m \underbrace{\|o_t - \hat{o}_t\|^2 + \alpha \|s_{t+1} - \hat{s}_{t+1}\|^2}_{\text{model prediction errors}} + \lambda_{\text{inst}}(t) \underbrace{\ell_{\text{cost}}(o_t, s_t, a_t)}_{\text{instrumental}} - \lambda_{\text{epis}}(t) \underbrace{\mathcal{I}_t}_{\text{epistemic proxy}} \right],$$

where $\mathcal{I}_t$ is an info-gain proxy (e.g., change-in-belief or ensemble variance). Crucially, *both* world-model and actor adapt via local PC-style updates, with no global backprop-through-time, enabling streaming online learning and robust behavior under sparse extrinsic rewards. Practically, ActPC agents report competitive control on robotic benchmarks while using purely local learning rules; this suggests a convenient route to plug predictive world-models and option-like controllers into our CDS/PDS admission machinery.

A unified active-PC control objective. Both APC and ActPC can be framed as minimizing a single-step free-energy surrogate that mixes model fit, instrumental value, and epistemic value, with hierarchical structure optional:

$$\min_{\theta, \phi, \pi} \mathbb{E} \left[\underbrace{-\log p_{\theta}(o_t | s_t) - \log p_{\theta}(s_{t+1} | s_t, a_t)}_{\text{predictive coding errors}} \right] + \underbrace{\beta \mathbb{E}[\ell_{\text{cost}}(o_{t+1}, s_{t+1})]}_{\text{instrumental}} - \underbrace{\eta \mathbb{E}[\text{IG}(s_{t+1}; o_{t+1} | s_t, a_t)]}_{\text{epistemic}},$$

subject to action updates $a_t \leftarrow \arg \min F_t$ (or a stochastic policy π) implemented as PC flows on action nodes. APC contributes the *hypernetworked, hierarchical* parameterization (state and action modules at multiple levels), while ActPC contributes a *local-learning, backprop-free* instantiation with explicit instrumental/epistemic modulators.

Interfaces we will use for CDS/PDS What PC gives us, in an RL context, is:

- a predictive transition head producing successor-state features that we will map to $\hat{n}(x, o)$
- a reward/value head producing $\hat{r}(x, o)$ (or a cost head)
- policy or option modules f_a^{ℓ} at various levels that can be proposed as candidates o
- optional precision/modulation knobs (instrumental/epistemic) that can serve as motives. In the next subsection, we will show how to admit such options with CDS or PDS and how to co-train an MDN that decomposes motives while PC learns the world model and controllers.

3.2 Why Predictive Coding is a Good Execution Engine for CDS/PDS

Now we draw the connection between PC-for-RL and the CDS/PDS approach to subgoal decomposition, arguing that the combination is not only feasible and natural but highly synergetic.

CDS/PDS admission hinges on two local planning primitives: (i) a per-state max over backed-up values across the current library of generators; and (ii) a robust test over motives (either a worst-case check across a cone W for CDS, or a componentwise Pareto screen over a finite cover W_{ref} for PDS). Predictive coding (PC) naturally implements both with local gates and error nodes, so the same residuals that already drive model learning and AVI can drive option admission. Adding a co-learned Motive Decomposition Network (MDN) simply turns the motive sets into learnable objects: the PC stack learns $(\hat{r}, \hat{n})$ and value, while the MDN learns a structured factorization of motives and induces W or W_{ref} that stay aligned with the evolving option library. PC’s locality of learning appears to have strong potential to enable this sort of co-learning.

3.2.1 PC execution sketch for CDS/PDS with a co-learned MDN

To implement CDS in a PC style we may proceed roughly as follows.

Nodes. Per feature-state x , the PC graph contains: (a) a value readout $\hat{v}(x; w) = w^\top x$ with value error $\varepsilon_v(x) = \text{softmax}_o B_o(x; w) - \hat{v}(x; w)$ implementing the AVI step; (b) per-option model nodes for $\hat{r}(x, o)$ and $\hat{n}(x, o)$ with standard TD targets; (c) backed-up nodes $B_o(x; w)$ feeding the softmax gate; (d) an *MDN head* that outputs a structured decomposition of motives into sub-aspects and induces either a cone W (extreme rays/box bounds) for CDS or a finite cover W_{ref} for PDS. The MDN may be lightly regularized (sparsity, orthogonality/independence or total-correlation penalties, anchor KLS) so that learned sub-aspects remain disentangled and stable as options evolve.

CDS margin residual (with MDN-induced W). For each candidate o ,

$$m_o(x) := \inf_{w \in W} (B_o(x; w) - B_{\text{base}}(x; w)), \quad \varepsilon_o^{\text{CDS}}(x) = \rho(\varepsilon_{\text{margin}} - m_o(x)),$$

with $\rho(u)$ a smooth rectifier. For box cones the infimum is local via componentwise extreme weights chosen by the sign of $\Delta \hat{n}(x, o)$; for general cones, evaluate along a small MDN-provided set of extreme rays. The CDS resid-

uals directly nudge $(\hat{r}, \hat{n})$ and (indirectly, via ε_v) the value readout to make m_o nonnegative on the update support.

PDS vector residuals (with MDN-induced W_{ref}). For PDS, stack $B_o(x; w^{(k)})$ over $w^{(k)} \in W_{\text{ref}}$ and form per-component gaps $\Delta_k(x, o) = B_o(x; w^{(k)}) - B_{\text{base}}(x; w^{(k)})$. Use a Pareto/budgeted residual that requires an improvement of at least ε on some components while penalizing violations of a downside budget ρ on any component (coverage- q variants are analogous). These vector residuals are local to the B_o and baseline nodes and back-propagate only through the option models that created them.

Local updates (all Hebbian). With precisions Π ,

$$\Delta w = \eta_v \Pi_v \varepsilon_v x, \quad \Delta w_r^{(o)} = \eta_r \Pi_{r,o} \varepsilon_{r,o} x, \quad \Delta W_n^{(o)} = \eta_n \Pi_{n,o} \varepsilon_{n,o} x^\top,$$

plus the CDS/PDS contributions $\Delta\theta \propto \Pi_{c,o} \varepsilon_o^{\text{CDS/PDS}} \partial m_o / \partial \theta$ (CDS) or $\partial \Delta_k / \partial \theta$ (PDS), all computed where the residuals are formed. The MDN parameters receive gradients from an MDN loss (anchors/regularizers) *and* from CDS/PDS residuals through the induced W or W_{ref} , aligning the motive geometry with the evolving option models.

Anytime online loop (co-training).

1. Clamp x ; read current w , $(\hat{r}, \hat{n})$ and MDN outputs (W or W_{ref}).
2. For each o , compute $B_o(x; w)$; softmax to get the planner target; update $\hat{v}$ via ε_v .
3. Form CDS or PDS residuals using the MDN-induced motive set; update $(\hat{r}, \hat{n})$, the MDN, and (indirectly) w .
4. Aggregate margins/gaps over the update sampler; admit any o passing the CDS/PDS screen; plan with AVI on $B \cup$ admitted options.

All steps are local; no global backprop-through-time or replay buffers are required, and adding options does not force retraining a monolithic model.

3.2.2 Anytime online loop: exact gradients and implementable updates

Danskin subgradients for the CDS margin. For a candidate o and baseline B at state x ,

$$m_o(x; W_\theta) := \inf_{w \in W_\theta} \Delta \hat{r}(x, o) + w^\top \Delta \hat{n}(x, o),$$

with W_θ induced by the MDN parameters θ . By Danskin’s theorem, if $w^\star \in \arg \min_{w \in W_\theta}(\cdot)$ then

$$\frac{\partial m_o}{\partial \Delta \hat{r}} = 1, \quad \frac{\partial m_o}{\partial \Delta \hat{n}} = -w^\star, \quad \frac{\partial m_o}{\partial \theta} \in \partial_\theta(w^\star(\theta)^\top \Delta \hat{n}),$$

so the CDS hinge $\rho(\varepsilon_{\text{margin}} - m_o)$ backpropagates via $w^\star$.

Practical MDN parameterizations (for $\partial m/\partial \theta$). *Ray-simplex:* the MDN emits normalized rays $\{r_k(\theta)\}$; $W_\theta = \text{cone}(\{r_k\})$ intersected with a scale plane, yielding

$$m_o = \Delta \hat{r} + \min_k r_k(\theta)^\top \Delta \hat{n}, \quad \frac{\partial m_o}{\partial \theta} = (J_\theta r_{k^\star})^\top \Delta \hat{n},$$

with $k^\star \in \arg \min_k(\cdot)$. *Polyhedral base:* $W_\theta = \{w : A_\theta w \leq b_\theta, \alpha^\top w = 1\}$; a single LP gives $w^\star$, and implicit differentiation of the KKT system yields $\partial m_o/\partial \theta$. In practice we either unroll a few primal-dual steps or use an implicit-diff LP layer.

PDS gradients. For $W_{\text{ref},\theta} = \{w^{(k)}(\theta)\}$ and $\Delta_k = \Delta \hat{r} + w^{(k)}(\theta)^\top \Delta \hat{n}$,

$$\frac{\partial \Delta_k}{\partial \Delta \hat{n}} = w^{(k)}(\theta), \quad \frac{\partial \Delta_k}{\partial \theta} = (J_\theta w^{(k)}(\theta))^\top \Delta \hat{n},$$

passed through the budget/coverage hinge.

Cache, cuts, and LP/SOCP only for borderlines. We keep a small cache of active rays $\{w_j\}$; most m_o calls reduce to $\min_j(\Delta \hat{r} + w_j^\top \Delta \hat{n})$. If inconclusive, we solve one LP/SOCP to find $w^\star$, add it to the cache, and continue; this keeps the loop anytime and inexpensive.

3.2.3 Why the MDN belongs in the execution loop

A learned motive geometry solves a moving-target problem. As $(\hat{r}, \hat{n})$ and the library B evolve, the set of weights that matter for robust admission also changes. By co-training the MDN with the same residual stream that shapes options, we: (i) keep W or W_{ref} tight around currently plausible motives; (ii) bias the internal representations toward disentangled sub-aspects that match the factors the planner actually trades off; and (iii) make the robust tests cheaper (few extreme rays/components) and sharper (less conservatism). In short, PC provides the local machinery to *realize* CDS/PDS; the MDN provides a learnable, adaptive *domain* over which the tests are applied.

3.2.4 Notes on stability and compute

Softmax temperature τ and precisions Π control gate sharpness and learning rates; annealing them mitigates flip-flopping and drift. Certificates that account for model and MDN uncertainty are straightforward (inflate margins by error radii or enlarge W_{ref}), and join safety/composition stability continue to hold for admitted options. Practically, this yields an anytime, modular, low-latency execution engine for CDS/PDS that remains robust under nonstationarity and streaming updates.

Sufficient conditions and practical tuning. Use two-timescale steps $\eta_{\text{heads}} \gg \eta_{\text{MDN}}$ with per-epoch dwell time before changing W . Ensure PC charts are contractive (free-energy F is Lyapunov), and update geometries monotonically (tighten cones or refine covers) so prior admissions persist. Schedule τ in piecewise-constant drops only after buffers stabilize; update precisions via clipped moving inverses of residual variances to avoid runaway gains. Limit cross-layer coupling by small coefficients and add temporal TV penalties on W/W_{ref} to damp oscillations.

3.2.5 Disentanglement objectives and mechanisms (PC + MDN)

To explicitly encourage disentanglement of motives and their sub-aspects in the shared representation x and in the MDN outputs:

- **Orthogonality and decorrelation.** Penalize cross-covariance between sub-aspect channels (e.g., $\|C - \text{diag}(C)\|_F^2$ for C the batch covariance of MDN sub-aspects), and add orthogonality constraints on the mapping A that maps sub-aspects z to weights $w = Az$.
- **Sparsity and group sparsity.** Use L_1 or $L_{2,1}$ penalties on sub-aspect activations and on the columns of A so that each feature contributes to few sub-aspects and each sub-aspect loads on a small, coherent subset of features.
- **Total-correlation (TC) penalty.** Add a TC term on the sub-aspect vector (e.g., via a density-ratio estimator) to reduce higher-order dependencies among sub-aspects.
- **Anchor and simplex priors.** Provide anchor directions for a subset of interpretable sub-aspects and regularize MDN outputs toward a simplex (Dirichlet/stick-breaking) so that learned sub-aspects form near-orthogonal rays that cleanly induce extreme rays of W .

- **Precision shaping.** Assign separate error nodes and precisions per sub-aspect motive; increase precision when a sub-aspect is active and decrease it when inactive. This yields selective pressure to represent factors independently, because off-axis coupling produces extra prediction error.
- **Temporal slowness and controllability.** Encourage slowness (penalize $\|\Delta z_t\|^2$) and controllability (reward sub-aspects that predictably change with actions). This separates slowly varying context from action-sensitive factors.

These penalties live naturally as local additions to the PC objective (they attach to specific error nodes and parameters), and they also stabilize the MDN: disentangled sub-aspects produce sharper, more stable W or W_{ref} , reducing the number of extreme rays or reference weights needed for reliable CDS/PDS screening.

3.2.6 Why PC can outperform backprop here

PC offers three advantages over conventional backprop for CDS/PDS with a co-learned MDN:

1. **Local, factorized credit assignment.** Each motive/sub-aspect has its own error channel and precision. Updates flow where the corresponding residuals arise, reducing gradient interference across motives and improving stability under streaming, nonstationary objectives. Backprop aggregates all losses into one global gradient, which tends to entangle factors and amplifies interference between concurrently learned options and motives.
2. **Easy injection of structured priors and constraints.** Orthogonality, sparsity, TC, anchors, and precision shaping are inserted as local error terms or priors in PC; they do not require bespoke gradient surgery or large auxiliary networks. This makes disentanglement objectives inexpensive and consistently enforced during online learning.
3. **Adaptive precision = robust gating.** Precision-weighted errors act like local second-order preconditioning and reliability gating. When MDN-induced sub-aspects are uncertain, their low precision down-weights their influence on admission; as they stabilize, precision increases and CDS/PDS decisions sharpen. Backprop typically needs carefully tuned loss weights or meta-optimizers to achieve the same effect.

4. **Two-time-scale inference for moving targets.** PC’s inner-loop inference rapidly rebalances representations as the option library and motives shift, while outer-loop synaptic updates consolidate. This reduces the need for replay and mitigates catastrophic interference relative to one-shot backprop updates.
5. **Modularity and compositionality.** Because PC attaches residuals to specific modules (world-model heads, option models, MDN), improvements in one module need not be backpropagated through the entire stack to be useful for admission. This modularity is exactly what CDS/PDS needs to keep the option library compact and the screens sharp.

In short, PC makes it straightforward to *separate* and *stabilize* the factors that CDS/PDS care about (motive sub-aspects, option-specific backups) via local, precision-weighted error flows and explicit disentanglement pressures. Co-training the MDN in this regime yields a motive geometry that is both interpretable and computationally efficient for robust screening, which would likely be difficult to obtain with standard backprop trained end-to-end on a single composite loss.

3.3 Interleaving PC, MDN, and CDS/PDS Throughout the Hierarchy

Now things get more interesting: We try to go further and sketch a deeper integration in which lower PC layers generate *internal rewards* from their own prediction-error geometry, and MDN modules decompose these internal rewards into sub-aspects that directly drive local CDS/PDS screens. In effect, the architecture becomes a stack of PC blocks, each paired with a small MDN that turns layer-specific error reductions into structured motives, with layer-local options admitted by CDS or PDS. The MDNs themselves can be trained by PC-style local residuals, yielding a fully local, streaming scheme.

3.3.1 Nontrivial internal rewards and hierarchical convergence

Layer-wise internal rewards should not become trivial loops. We therefore: (i) use potential-based shaping tied to the parent layer (e.g., $r_{\text{int}}^\ell = \psi^\ell(x_t) - \gamma\psi^\ell(x_{t+1})$ with $\psi^\ell = -F^\ell$) so improvements reflect genuine surprise reduction; (ii) redeem internal rewards against certified parent-layer gains via a running budget so local credit pays out only when it helps the

parent evaluator; and (iii) gate admissions at layer ℓ on the parent slice using CDS/PDS, so internal rewards act as hints, not standalone objectives. Under contractive charts and monotone geometry updates, this yields a qualitative convergence invariant: admitted micro-skills do not degrade the parent evaluator and their cumulative internal payout is bounded by measured parent improvements.

Common setup and notation. Let a PC hierarchy have layers $\ell = 1, \dots, L$ with activities x_t^ℓ and prediction errors $\varepsilon_t^\ell = x_t^\ell - \hat{x}_t^\ell$, precisions Π^ℓ , and per-layer free-energy contributions

$$F_t^\ell = \frac{1}{2} \|\varepsilon_t^\ell\|_{\Pi^\ell}^2 = \frac{1}{2} (\varepsilon_t^\ell)^\top \Pi^\ell \varepsilon_t^\ell.$$

Let S^ℓ denote the set of error nodes that ℓ is responsible to reduce (typically children and lateral peers). Define the *internal layer reward* as a shaped reduction in downstream free-energy:

$$r_t^\ell := \underbrace{\alpha_\ell (F_t^{\text{ref},\ell} - F_{t+1}^{\text{ref},\ell})}_{\text{surprise reduction}} - \underbrace{\beta_\ell \Delta F_t^\ell}_{\text{regularity cost}} - \underbrace{\gamma_\ell \|x_{t+1}^\ell - x_t^\ell\|^2}_{\text{stability}},$$

where $F^{\text{ref},\ell}$ aggregates a chosen subset of lower-level errors (“what this layer tries to help”); the weights $(\alpha_\ell, \beta_\ell, \gamma_\ell)$ shape myopic vs. farsighted trade-offs. Using $\Delta F_t^\ell := F_{t+1}^\ell - F_t^\ell$ penalizes degenerate “predict nothing” solutions.

Layer-local MDN motives. At each layer, an MDN maps layer context to a compact motive decomposition:

$$z_t^\ell = \text{MDN}^\ell(x_t^\ell, \text{ctx}_t^\ell), \quad w_t^\ell = A^\ell z_t^\ell, \quad W^\ell = \text{cone}(\mathcal{E}^\ell)$$

where $z_t^\ell \in \mathbb{R}^{k_\ell}$ are sub-aspects, A^ℓ is a learnable mixing, and $\mathcal{E}^\ell$ denotes the set of extreme rays (from anchors/box bounds or learned). For CDS we use the cone W^ℓ ; for PDS we use a finite cover $W_{\text{ref}}^\ell = \{w^{\ell,(1)}, \dots, w^{\ell,(K_\ell)}\}$ sampled from the MDN.

Layer-local option models. Each layer has a small library B^ℓ and a candidate pool O^ℓ of *options* that act in the latent space (e.g., macro-transitions, controllable priors, or hypernetwork parameter switches). As in the top-level CDS/PDS:

$$\hat{r}^\ell(x^\ell, o) \approx \mathbb{E}[r_{t+1}^\ell \mid x_t^\ell, o], \quad \hat{n}^\ell(x^\ell, o) \approx \mathbb{E}[x_{t+1}^\ell \mid x_t^\ell, o].$$

Define the layer- ℓ backup

$$B_o^\ell(x^\ell; w^\ell) = \hat{r}^\ell(x^\ell, o) + (w^\ell)^\top \hat{n}^\ell(x^\ell, o).$$

3.3.2 Layerwise CDS/PDS screens

CDS at layer ℓ . Using the MDN-induced cone W^ℓ ,

$$m_o^\ell(x^\ell) := \inf_{w \in W^\ell} (B_o^\ell(x^\ell; w) - B_{\text{base}}^\ell(x^\ell; w)),$$

admit o if $\mathbb{E}_{x^\ell \sim \nu^\ell}[m_o^\ell(x^\ell)] \geq \varepsilon_\ell$ or $\Pr[m_o^\ell(x^\ell) \geq 0] \geq 1 - \delta_\ell$.

PDS at layer ℓ . With W_{ref}^ℓ , stack $B_o^\ell(x^\ell; w^{\ell, (k)})$ over k and apply a Pareto/budgeted screen with $(\varepsilon_\ell, \rho_\ell, q_\ell)$.

3.3.3 Interleaved execution (one step, all layers)

1. **PC inference.** Run the PC inner loop to update $(x_t^1, \dots, x_t^L)$ and a_t .
2. **Internal rewards.** For each ℓ , compute r_t^ℓ from chosen error sets S^ℓ and the shaped reduction in $F^{\text{ref}, \ell}$.
3. **MDN updates.** For each ℓ , produce z_t^ℓ and W^ℓ or W_{ref}^ℓ ; apply disentanglement penalties (below) and PC residuals to train MDN $^\ell$ locally.
4. **Layer-local CDS/PDS.** For each candidate $o \in O^\ell$, evaluate CDS or PDS using the MDN-induced motive set; admit passing options into B^ℓ .
5. **Planner updates.** Update layer values with a softmax over B_o^ℓ (or propagate to a top-level value), and step online Hebbian parameter updates to $(\hat{r}^\ell, \hat{n}^\ell)$ and MDN $^\ell$.

3.3.4 Cross-layer coupling and safeguards

- **Top-down hypernetworks.** Higher layers can parameterize lower-layer option families via hypernets (e.g., APC-style), while lower-layer PDS can surface novel skills upward.
- **Precision gating.** Couple precisions Π^ℓ to MDN confidence so uncertain motives contribute less to admission, avoiding spurious internal reward loops.
- **Anti-collapse shaping.** Include reconstruction/forecast terms and slowness/controllability priors in r_t^ℓ so internal rewards cannot be maximized by trivial constant predictions.
- **Library compression.** Maintain ε -Pareto covers per layer to keep $|B^\ell|$ small.

3.3.5 Disentanglement at every level (PC + MDN)

To keep motives and sub-aspects cleanly factorized across layers, use local penalties aligned with PC residuals:

- **Orthogonality/decorrelation.** Penalize off-diagonal covariance in z^ℓ and encourage A^ℓ to have near-orthonormal columns.
- **Sparsity/group sparsity.** L_1 and $L_{2,1}$ on z^ℓ and on columns of A^ℓ so that each sub-aspect corresponds to a small set of features and each feature loads on few sub-aspects.
- **Total correlation (TC).** Add a TC penalty on z^ℓ (estimated by density-ratio or classifier-based bounds) to break higher-order dependencies.
- **Anchors/simplex priors.** Seed interpretable rays and regularize z^ℓ to a simplex (Dirichlet/stick-breaking), making extreme rays of W^ℓ explicit.
- **Precision shaping by factor activity.** Give each sub-aspect its own error node and precision; misalignment across factors creates extra error, pushing toward disentanglement by construction.
- **Slowness and controllability.** Penalize $\|z_{t+1}^\ell - z_t^\ell\|^2$ while rewarding predictable, action-conditional changes in selected sub-aspects.

Because these are local error terms, they integrate seamlessly with PC updates and directly stabilize MDN $^\ell$ outputs, tightening the cones W^ℓ and covers W_{ref}^ℓ used by CDS/PDS.

3.3.6 Why this interleaving is useful

Layerwise internal rewards let lower levels autonomously surface skills that reduce downstream surprise, while MDNs turn those skills into structured motives that CDS/PDS can screen reliably. This creates a bidirectional pipeline: top-down hypernets shape what skills are useful; bottom-up PDS/CDS certifies which skills measurably help under current motives.

3.3.7 Why PC should outperform backprop here

Conceptually, it appears PC confers advantages that are amplified by this interleaving:

1. **Local, motive-specific credit.** Internal rewards and MDN sub-aspects each attach to their own error nodes, so gradients do not have to traverse the entire stack to influence the responsible module. This reduces gradient interference that is common in end-to-end backprop with many auxiliary heads.
2. **Adaptive precision as reliability gating.** Precision-weighted errors downweight uncertain MDN factors or noisy layer rewards, acting like online trust-region scaling. Backprop typically needs elaborate loss reweighting or meta-optimizers to emulate this.
3. **Two-time-scale adaptation.** PC’s inner inference loop rebalances representations instantly when internal rewards shift, while synaptic updates consolidate more slowly. This mitigates nonstationarity introduced by layerwise option admission better than one-shot gradient steps.
4. **Built-in disentanglement hooks.** Orthogonality, sparsity, TC, anchors, and precision shaping are all implemented as local PC residuals, so they continuously regularize the representations that MDNs see. In backprop, these require global losses that often conflict and entangle.
5. **Modular compositionality.** Options, MDNs, and world-model heads receive exactly the residuals that prove their usefulness (CDS/PDS margins or Pareto gaps). Improvements propagate through the planner without retraining a monolith, enabling faster skill discovery and safer admission.

Learning disentangled representations is particularly natural in this setting: each sub-aspect z_i^ℓ has its own precision-weighted error path and appears explicitly as an extreme ray of W^ℓ . Co-training MDN^ℓ inside PC aligns those rays with factors that actually reduce downstream surprise and pass CDS/PDS screens. End-to-end backprop, by contrast, tends to blend these factors into a single composite gradient, making it harder to keep motives and sub-aspects separable and stable during streaming learning.

3.3.8 Implementation tips and default settings

- **Inner CDS solver tiers.** Use cached rays first; fall back to at most one LP/SOCP per borderline decision; cache 8–16 active rays per context and compress periodically.

- **Adaptive PDS cover.** Initialize with $K \in [8, 16]$ MDN anchors; add any new w^* that beats the current cover by more than a tolerance; prune with greedy max-min to keep $K \in [8, 24]$.
- **Step-size hierarchy.** Choose $\eta_{\text{heads}} \gg \eta_{\text{MDN}}$; halve on plateaus; schedule softmax temperature drops only after admission buffers stabilize.
- **Precision caps.** Update precisions with a clipped moving inverse-variance of residuals and caps $(\Pi_{\min}, \Pi_{\max})$ to avoid runaway gains.
- **Shaping choices.** Set $A\psi^\ell = -F^\ell$ for potential shaping; redeem internal rewards against measured parent improvements using a small dual step.
- **Monotone geometry.** Tighten cones (CDS) or refine covers (PDS) with hysteresis to preserve earlier admissions and reduce flip-flop.

3.4 Continual Backprop vs. Predictive Coding

Having outlined how to integrate CDS/PDS tightly with PC to form a novel RL approach, we now pause to compare the approach to another innovative approach from the literature: continual backprop as proposed in [10].

Continual backprop in brief. The continual backprop (CBP) idea is motivated by the way vanilla backprop learns well at the start but often *loses plasticity* during extended tracking: performance degrades as distributions and tasks drift, with growing weight magnitudes, saturation, and gradient interference. CBP addresses this by *continually injecting random features* and replacing low-utility units via a generate-and-test mechanism, which restores diversity and keeps adaptation going while maintaining constant memory. Empirically, CBP stabilizes performance on long semi-stationary streams (Bit-Flipping; Permuted MNIST) and in nonstationary RL (Slippery Ant), whereas plain backprop deteriorates over time; details include the utility-based tester, maturity thresholds, and PPO integration (CPPO). These observations motivate methods that maintain plasticity without relying on a large, global composite gradient.

3.4.1 Comparison: continual backprop vs. predictive coding

How does CBP compare with PC-for-RTL?

Credit assignment and interference. Backprop aggregates many objectives (reward heads, auxiliary losses, option losses, motive penalties) into a *single* global gradient. Under drift, this induces interference among loosely related signals; controlling the relative weights is delicate and often non-stationary. PC keeps *separate error channels and precisions* for each objective (e.g., CDS margins, PDS gaps, model-fit errors, MDN anchors), so updates flow only where a residual is generated; precisions act as reliability gates. This factorization, arguably, reduces interference and eliminates much of the loss-weight tuning.

Plasticity maintenance. CBP explicitly re-injects randomness to restore plasticity, replacing low-utility features with fresh ones from the initial distribution; this stabilizes long-run adaptation in both supervised streams and RL and improves over backprop baselines. PC maintains plasticity *implicitly* via two mechanisms: (i) inner-loop inference quickly rebalances latent activities when streams drift; and (ii) precision shaping downweights unreliable channels (e.g., shaky motives) and upweights consistent ones. The end result, we suggest, is continual adaptation without global feature replacement and with less dependence on initialization.

Nonstationarity and moving motives. When the blend of motives changes (new safety constraints, new risk budgets), backprop must propagate those changes through deep stacks before option admission reflects them. In PC, the motive geometry from the MDN enters locally at the CD-S/PDS gates; changing W or W_{ref} immediately changes the residuals that drive model and policy updates, with precisions damping oscillations.

Subgoal admission and certificates. Backprop can implement CD-S/PDS, but the robust inf/Pareto tests typically sit at the *top* of a stack, so lower layers only feel their influence through a global loss. PC implements these tests as *local gates* on backed-up nodes $B_o(x; w) = \hat{r}(x, o) + w^\top \hat{n}(x, o)$, producing explicit margin/gap residuals that directly push the parameters that created them. This yields sharper certificates and lower-variance credit assignment.

Disentanglement and interpretable motives. Disentangling motives and sub-aspects (so that $w = Az$ with near-orthogonal columns of A and minimally dependent components of z) is awkward to enforce with global backprop. In PC, orthogonality, sparsity, total-correlation penalties, anchor

priors, and precision shaping are simply extra local error terms; they continuously regularize the sub-aspects the MDN emits and the features that feed them.

Compute and robustness. CBP preserves backprop’s complexity class by trading occasional feature resets for constant memory, and it wins strongly over plain backprop in long, drifting regimes. PC replaces global backprop passes with local updates and small inner inference loops; robust screening is cheap for box W (closed form) and moderate for general W (few extreme rays), and PDS works with small K -covers. In practice both CBP and PC scale, but PC’s *locality* matches the CDS/PDS gates we need and reduces sensitivity to replay and to loss weighting.

3.4.2 Why predictive coding should work better for subgoal decomposition (CDS/PDS)

The ultimate result of these differences between CBP and PC, we believe, will be a marked superiority of performance for PC on subgoal decomposition for sizeable and difficult RL problems.

Local realization of cone- and Pareto-dominance. CDS requires worst-case improvement across a cone W ; PDS requires componentwise dominance on W_{ref} . In PC, these are computed exactly where the backed-up values are formed, producing residuals

$$m_o(x) = \inf_{w \in W} (B_o(x; w) - B_{\text{base}}(x; w)), \quad \Delta_k(x, o) = B_o(x; w^{(k)}) - B_{\text{base}}(x; w^{(k)}),$$

which then drive only the parameters that produced B_o and $\hat{r}, \hat{n}$. Backprop, by contrast, must move signal through a shared loss that mixes many unrelated terms.

Co-learned MDN for motive geometry. A co-trained MDN emits sub-aspects z and weights $w = Az$, inducing W (CDS) or W_{ref} (PDS). In PC, the MDN is trained by local residuals tied to CDS/PDS violations and to disentanglement penalties (orthogonality, sparsity, total correlation, anchors), with precision gating by MDN confidence. This keeps W tight around what the planner *actually* trades off and makes CDS/PDS tests both cheaper (fewer extreme rays/components) and sharper (less conservatism).

Disentanglement improves subgoal quality. Subgoals that pass CDS/PDS should advance *distinct* aspects of performance. PC’s local penalties make those aspects independent in the representation feeding the MDN (and thus in W or W_{ref}), so admitted options tend to specialize cleanly. In backprop, composite gradients tend to entangle such factors unless losses, schedules, and architectures are carefully engineered.

Two-time-scale stabilization under moving targets. Option libraries and motive blends change. PC’s inner-loop inference rebalances activities immediately; outer-loop synaptic updates consolidate more slowly. This two-time-scale scheme dampens flip-flops in CDS/PDS admission as W or W_{ref} evolves. Backprop’s single-shot updates under mixed objectives more easily overshoot when motives or options shift quickly.

Auditable certificates and modular composition. Because CDS/PDS tests are explicit nodes, PC yields auditable certificates (margins, Pareto gaps) attached to each admitted option. Residuation-based composition (weakest prefixes/suffixes) then acts on the same locally computed summaries. Backprop can compute the same quantities, but they are not intrinsic to the update graph and typically require extra passes or bespoke heads.

Summary. CBP shows that maintaining plasticity is crucial for continual learning and that persistent randomness plus testing can stabilize long-run adaptation—substantially outperforming plain backprop under drift. PC takes a complementary route: it *factorizes* credit assignment by motive and by option, provides cheap local realizations of CDS/PDS gates, and co-trains an MDN whose learned motive geometry is aligned with the option library. For subgoal decomposition with cone-dominant and Pareto-dominant admission, this locality, precision gating, and built-in disentanglement make PC the more natural and robust execution engine, while still retaining the modularity needed to incorporate CBP-like plasticity aids if desired.

4 Integration with Reasoning, Evolution and Hyperon

So far we have remained squarely within the realm of gradient-descent neural learning for RL, merely proposing to organize the learning loops and network structures a bit differently than in Sutton et al. However, the core ideas of

CDS/PDS-based subgoal decomposition are modality-neutral: any generator that exports an expectation-model summary $(\hat{r}, \hat{n})$ can be admitted by the same gate and composed by the same planner. In this section we show how this unification works inside the Hyperon cognitive architecture, how it benefits from MetaMo’s motive management and weakness-driven learning (WDL), and how it extends to explicit logical inference and evolutionary program learning.

Why weakness belongs here. Weakness-driven learning chooses the *least committed* hypothesis consistent with data and constraints. In the quantale view used throughout, behaviors are monotone endomaps $T : V \rightarrow V$ with planning by join and sequencing by composition. Given a stream of robust linear constraints indexed by a motive set W (a cone for CDS or a finite cover W_{ref} for PDS), WDL selects

$$T^* = \bigvee \text{Feas}(D, W),$$

the least upper bound of all feasible transformers. Residuation then propagates weakness through compositions: with a plan-level target S , the weakest prefix is $A^* = S/B$ and the weakest suffix $B^* = A \setminus S$. This is *exactly* the algebra CDS/PDS operate in: CDS/PDS certify generators; WDL picks the weakest consistent way to use them; residuation synthesizes the weakest missing pieces. Thus weakness is not an add-on, it is the organizing principle that makes admission, planning, and composition cohere.

4.1 MetaMo and weakness-driven learning (WDL)

MetaMo [3] manages goals and motives: from an RL perspective, it answers “what matters now?” MetaMo supplies a compact description of current priorities by exposing motive weights that score feature-states and drive the election of procedures estimated to advance those motives in context.

Among the heuristics for choosing procedures, WDL prefers the *weakest* behavior consistent with motives and plan-level specifications [4]. In our integration, CDS/PDS provide the robust, planner-facing gate that decides which candidate generators (options/macros/controllers) are safe to add. Unlike prior MetaMo implementations, we co-learn a Motive Decomposition Network (MDN) that *learns* the geometry of motives (a cone W for CDS or a finite cover W_{ref} for PDS), so admission tests, residual computations, and plan synthesis adapt as the agent and environment change.

Typically a human designer initializes a small set of top-level motives and default weights. In a CDS/PDS approach, each such motive spawns

finer sub-motives via an MDN that co-evolves with subgoal decomposition. Treating all sub-motives as a pooled set lets dependencies be handled by the same machinery.

How the pieces fit. In the linear slice used throughout, each motive is a weight $w \in \mathbb{R}^d$ scoring $v_w(x) = w^\top x$. MetaMo exposes plausible weights; the MDN factorizes top-level motives into sub-aspects z and mixes $w = Az$, inducing either a cone W (CDS) or a finite cover W_{ref} (PDS). WDL converts data and specifications into robust linear inequalities over that motive set, selects the weakest consistent transformer by join-minimality, and uses residuation to propagate weakest-sufficient choices through compositions. CDS/PDS implement the generator-admission rule inside this loop.

4.1.1 Roles and interfaces

MetaMo mixes and schedules motives in nonstationary settings. In the linear slice, it exposes a motive cone

$$W = \text{cone}\{w^{(k)}\}_{k=1}^K \subset \mathbb{R}^d,$$

which informs (i) which options pass CDS and (ii) how plan specifications are enforced robustly during learning. In our MDN-enhanced variant, a small MDN maps context to sub-aspects z and a mixing $w = Az$, inducing W (or W_{ref}) that co-trains from the same residual stream as the planner.

Note (PDS / product-quantale). In the multiobjective case, replace the single-weight slice by the product space V^K with componentwise order and joins; PDS admission and WDL residuation act componentwise, so weakness/minimality and residual constructions carry over unchanged.

4.1.2 Transformer algebra and order

Let $V = \mathbb{R}^X$ (pointwise order, pointwise joins). Let K be the set of monotone, join-continuous maps $T : V \rightarrow V$ with product $T_2 \otimes T_1 = T_2 \circ T_1$ and unit $I = \text{id}_V$; then $(K, \otimes, \bigvee, \leq)$ is a unital quantale. Each action/option o induces T_o via

$$[T_o v](x) = \hat{r}(x, o) + v(\hat{n}(x, o)).$$

Planning uses $B_G = \bigvee_{g \in G} T_g$. We work in a residuated sub-quantale $H \subseteq K$ so both residuals exist:

$$A \setminus B = \bigvee \{X \in H : A \otimes X \leq B\}, \quad B/A = \bigvee \{X \in H : X \otimes A \leq B\}.$$

4.1.3 Data-to-constraints: robust inequalities over a motive cone

Given a stream D of pairs $(x, b(x))$ and a motive cone W , WDL forms robust constraints

$$\forall w \in W : \quad (Tv_w)(x) \geq b(x), \quad v_w(\cdot) = w^\top(\cdot), \quad [Tv_w](x) = \hat{r}_T(x) + w^\top \hat{n}_T(x).$$

Define the feasible set

$$\text{Feas}(D, W) = \left\{ T \in H : \forall (x, b) \in D, \inf_{w \in W} (\hat{r}_T(x) + w^\top \hat{n}_T(x)) \geq b(x) \right\}.$$

For box cones $W = [w_{\min}, w_{\max}]$ the infimum is closed form by choosing per-coordinate extremes according to the sign of $\hat{n}_T$. When W (or W_{ref}) is MDN-induced, we use the MDN’s extreme rays or reference weights to instantiate the same test.

4.1.4 WDL objective: the weakest consistent hypothesis

WDL selects the least committed hypothesis

$$T^* = \bigvee \text{Feas}(D, W),$$

i.e., among all behaviors consistent with data and robust over W , pick the pointwise least upper bound in H .

4.1.5 Propagating weakness via residuation

If MetaMo specifies a plan-level target S on a region Ω , then with a fixed suffix B the weakest admissible prefix is $A^* = S/B$, and dually, with a fixed prefix A , the weakest suffix is $B^* = A \setminus S$. When S is partially specified on Ω , compute residuals in the subposet that agrees with S on Ω .

4.1.6 MetaMo $\rightarrow$ WDL interface (with MDN)

MetaMo provides: (1) a motive set (W or W_{ref}) reflecting current priorities/uncertainties; (2) an optional plan specification S (e.g., robust lower bounds on Ω). WDL then (i) screens candidates by CDS/PDS under this motive set; and (ii) solves residual problems such as $A^* = S/B$ to extract the weakest missing piece. The MDN co-trains from the same residuals, tightening W or W_{ref} as models improve.

4.1.7 CDS/PDS as generator-admission rules inside WDL

Let B be the current admitted set. For candidate o , define the robust CDS margin

$$m_o(x) = \inf_{w \in W} (B_o(x; w) - B_{\text{base}}(x; w)), \quad B_{\text{base}}(x; w) = \max_{b \in B} (\hat{r}(x, b) + w^\top \hat{n}(x, b)).$$

Admit o if $\mathbb{E}_{x \sim \nu}[m_o(x)] \geq \varepsilon$ or $\Pr[m_o(x) \geq 0] \geq 1 - \delta$. PDS uses the vectorized form on W_{ref} with Pareto order and optional budgets (ε, ρ) and coverage q .

4.1.8 Robust constraints with model uncertainty

If $|\hat{r} - r| \leq \varepsilon_r(x, o)$ and $\|\hat{n} - n\|_\infty \leq \varepsilon_n(x, o)$, then

$$\inf_{w \in W} (\hat{r} + w^\top \hat{n}) \geq \inf_{w \in W} (r + w^\top n) - \varepsilon_r - \|W\|_{\max, 1} \varepsilon_n, \quad \|W\|_{\max, 1} = \sup_{w \in W} \|w\|_1,$$

yielding a conservative admission certificate.

4.1.9 Guarantees

If H is residuated and closed under joins, then: (i) soundness: $T^* = \bigvee \text{Feas}(D, W)$ satisfies all robust constraints; (ii) minimality: $T^* \leq T$ for any $T \in \text{Feas}(D, W)$; (iii) compositionality: if $A^* = S/B$ and $B^* = A \setminus S$ exist, then any $A \geq A^*$ and $B \geq B^*$ satisfy $A \otimes B \leq S$.

4.1.10 Micro-illustration (CDS) and a PDS budgeted retrofit

First let's consider a simplistic example with *CDS*. In a two-room grid, take a hallway feature h and a cone $W = \{w : 0 \leq w_h \leq w_{\max}\}$. A hallway option that detours to avoid a 3-cell penalty band has $\Delta \hat{r} > 0$ but longer duration ($\Delta \hat{n}_h < 0$); its worst-case margin is $m(x) = \Delta \hat{r} + w_{\max} \Delta \hat{n}_h$. For realistic numbers (e.g., $\gamma = 0.99$, three -1 penalties avoided, four extra steps) this is nonnegative for $w_{\max} \lesssim 80$, so CDS admits the roundabout hallway option.

Now for *PDS (budgeted)*. Split the downstream value into two explicit weights $w = (w_{\text{safety}}, w_{\text{speed}})$ with a small MDN-induced cover $W_{\text{ref}} = \{w^{(1)}, w^{(2)}\}$ emphasizing safety vs. speed. For a candidate o define component gaps $\Delta_k(x, o) = B_o(x; w^{(k)}) - B_{\text{base}}(x; w^{(k)})$. Admit o if

$$|\{k : \Delta_k(x, o) \geq \varepsilon\}| \geq qK, \quad \Delta_k(x, o) \geq -\rho \quad \forall k,$$

with small $\varepsilon > 0$ (significant improvement), downside budget $\rho \geq 0$, and coverage fraction $q \in (0, 1]$. In the two-room toy, a *skirt* option (avoid 2 of 3

penalties, +1 step) improves safety by at least ε while incurring a tiny speed downside ρ at the speed-heavy reference; budgeted PDS admits it, whereas CDS on a very broad cone might reject all single options.

Summary. MetaMo supplies a motive geometry and optional plan specs; WDL selects the weakest consistent transformer; CDS/PDS certify robust admission; and the MDN learns a *data-driven* motive slice (W or W_{ref}) so the same algebra applies adaptively.

4.2 Beyond standard RL: logic and evolutionary learning

CDS/PDS do not care how a candidate is produced; they only require an expectation-model summary $(\hat{r}, \hat{n})$ and a motive set to test against. Here we show how to build such summaries from (i) probabilistic logic with backward-chaining, and (ii) evolutionary program learning. In both cases, generators become decision transformers that plug into the same planner and the same CDS/PDS screens. We also make concrete how abstract-interpretation or type/shape analyses provide *bounds* when data are sparse.

4.2.1 Probabilistic logic with backward-chaining

State, goals, and rules. Let $x \in \mathbb{R}^d$ encode a proof/control state: pending subgoals, derived facts/embeddings, and solver context (depth, budgets). A backward-chaining step unifies the current goal G with a rule head and replaces G by instantiated antecedents. We write a rule

$$r : H(\theta) \Leftarrow A_1(\theta_1), \dots, A_m(\theta_m),$$

with probabilistic truth values (strength, confidence) as in PLN [11]. A *macro-step* o_r is one application of r (or a small bundle of steps) that runs until it returns control (e.g., after closing the current subgoal frontier or hitting a budget). While o_r runs, it may accrue external utility u_t (for world-affecting actions) and always modifies the proof/control state.

Backward-chaining as a transformer. Treat o_r as a decision transformer T_{o_r} :

$$[T_{o_r}v](x) = \hat{r}(x, o_r) + v(\hat{n}(x, o_r)),$$

with summaries

$$\hat{r}(x, o_r) \approx \mathbb{E}\left[\sum_{t=1}^K \gamma^{t-1} u_t \mid x_0 = x, o_r\right], \quad \hat{n}(x, o_r) \approx \mathbb{E}[\gamma^K x_K \mid x_0 = x, o_r],$$

and random duration K until o_r returns. The planner uses $B_{o_r}(x; w) = \hat{r}(x, o_r) + w^\top \hat{n}(x, o_r)$.

From traces to $(\hat{r}, \hat{n})$ (worked mapping). Collect proof traces $(x_t, o_t, u_{t+1}, x_{t+1}, \text{active})$. For each active step of o , update targets

$$\begin{aligned} r_{\text{targ}}(x_t, o) &\leftarrow u_{t+1} + \gamma r_{\text{targ}}(x_{t+1}, o) \cdot \mathbf{1}\{o \text{ still active}\}, \\ n_{\text{targ}}(x_t, o) &\leftarrow \gamma n_{\text{targ}}(x_{t+1}, o) \cdot \mathbf{1}\{o \text{ still active}\} + \gamma^{K_t} x_{t+K_t} \cdot \mathbf{1}\{o \text{ returned at } t + K_t\}. \end{aligned}$$

Fit linear/low-rank models $\hat{r}(x, o) = w_r^{(o)\top} x$, $\hat{n}(x, o) = W_n^{(o)} x$ by least-squares TD or online SA (with importance weights if off-policy). This maps *proof traces* to the planner-facing $(\hat{r}, \hat{n})$.

Bounds from abstract interpretation. When traces are sparse, static type/shape analyses bound $(\hat{r}, \hat{n})$:

- *Depth/shape to bound K .* If a macro guarantees at most L rule applications (e.g., by depth budget or acyclicity), then $K \in [K_{\min}, K_{\max}]$ with $K_{\max} \leq L$. Thus $\gamma^K \in [\gamma^{K_{\max}}, \gamma^{K_{\min}}]$ and $w^\top \hat{n}$ inherits interval bounds.
- *Guarded effects to bound features.* If an antecedent guarantees at most Δ_∞ change in features per step (by type constraints or monotone dataflow), then $\|\hat{n}(x, o) - x\|_\infty \leq \sum_{t=1}^K \gamma^t \Delta_\infty$, yielding a closed-form bound.
- *Cumulant envelopes.* If per-step utility $u_t \in [\underline{u}, \bar{u}]$ while o is active, then $\hat{r}(x, o) \in [\underline{u} \frac{1-\gamma^K}{1-\gamma}, \bar{u} \frac{1-\gamma^K}{1-\gamma}]$.

These intervals plug directly into CDS/PDS certificates (inflate slacks).

Worked example: a Horn rule to a transformer. Consider a tiny two-room world with a wall switch. Rules:

$$\text{CorridorCost}(c) \Leftarrow \text{SwitchOff} \quad (\text{cost } c=5), \quad \text{CorridorCost}(c) \Leftarrow \text{SwitchOn} \quad (\text{cost } c=1).$$

Define macro o_{flip} : “flip switch then return to planner”. It takes $K_{\text{flip}} = 2$ steps and incurs no external utility while flipping, so $\hat{r}(x, o_{\text{flip}}) \approx 0$, $\hat{n}(x, o_{\text{flip}}) \approx \gamma^2 x'$, where x' marks the switch-on fact and updated position. Define a *composed* macro $o_{\text{flip} \rightarrow \text{hall}}$ as “flip, then go to hallway”; its summaries are

$$\hat{r}(x, o_{\text{flip} \rightarrow \text{hall}}) \approx -1 \cdot (\gamma^5 + \gamma^6 + \gamma^7), \quad \hat{n}(x, o_{\text{flip} \rightarrow \text{hall}}) \approx \gamma^9 e_{\text{hall}},$$

since crossing the corridor starts two steps later with low per-step cost -1 , and returns at the hallway feature after 9 steps. These are closed-form because the rule *symbolically* sets the corridor cost. The CDS margin against a baseline “rush-through” macro (3 high-cost steps, return at $\gamma^7 e_{\text{hall}}$) becomes

$$m(x) = \inf_{w \in W} \left\{ \underbrace{[-(\gamma^5 + \gamma^6 + \gamma^7) - (- (\gamma^3 + \gamma^4 + \gamma^5))]}_{\Delta \hat{r}} + \underbrace{w^\top (\gamma^9 e_{\text{hall}} - \gamma^7 e_{\text{hall}})}_{\Delta(w^\top \hat{n})} \right\},$$

which is nonnegative on a wide cone W of hallway weights, hence CDS admits $o_{\text{flip} \rightarrow \text{hall}}$. In practice, we export $(\hat{r}, \hat{n})$ computed either analytically (as above) or from traces; both plug into the same gate.

Putting the MDN into the loop (logic). An MDN maps context to sub-aspects and a motive cone W or cover W_{ref} . Logical motives (e.g., correctness, safety, novelty) occupy explicit axes; MDN uncertainty is folded into certificates by inflating $\|W\|_{\max,1}$ or adding rays to W_{ref} .

4.2.2 Evolutionary program learning

Why evolution here. Evolution discovers non-differentiable structure (program skeletons, discrete timing) that gradients struggle to invent; it evaluates candidates holistically and is robust to deceptive gradients and sparse credit.

From programs to expectation models. Represent a candidate as a typed AST or graph with guarded control flow. Treat execution from feature-state x under macro o as a stochastic process producing utilities u_t and a random return-time K . Export

$$\hat{r}(x, o) \approx \mathbb{E} \left[\sum_{t=1}^K \gamma^{t-1} u_t \right], \quad \hat{n}(x, o) \approx \mathbb{E} [\gamma^K x_K],$$

estimated by instrumented rollouts or exact semantics when available. Learn light parametric surrogates $(\hat{r}, \hat{n})$ per program by least squares. These summaries are modality-neutral and enter AVI and CDS/PDS identical to neural or logical generators.

Static bounds for safety certificates. Use simple abstract-interpretation passes:

- *Loop/recurrence bounds:* syntactic guards imply $K \in [K_{\min}, K_{\max}]$.

- *Lipschitz feature updates:* per-step feature change $\|\Delta x\|_\infty \leq B$ under guards yields $\|\hat{n} - x\|_\infty \leq B \sum_{t=1}^K \gamma^t$.
- *Cumulant ranges:* known reward ranges per basic block imply interval bounds on $\hat{r}$.

Combine with conformal or cross-fitted calibration to avoid circularity when estimating slacks.

Worked sketch. A GP individual $o_{\text{probe_and_align}}$ encodes: probe corridor phase; if risky, wait 3; then cross. Instrumentation yields $\hat{r} \approx -\frac{1}{2} \cdot 5\gamma^3$ for the probe plus a small residual from microhazards; return-time $\hat{K} \in \{9, 12\}$ with $\mathbb{E}[\gamma^{\hat{K}}] \approx 0.90$. Thus $B_o(x; w) = \hat{r} + 0.90 w^\top e_{\text{hall}}$ strictly dominates a baseline that crosses immediately (expected $-7.2 + \gamma^7 w^\top e_{\text{hall}}$) along safety-heavy rays and, with a small budget ρ on speed-heavy rays, passes the PDS budgeted screen. The planner treats this program exactly like any other generator.

4.2.3 Interfacing all modalities with MetaMo, WDL, and planning

Planner interface and residuation. All modalities export the same $[Tv](x) = \hat{r}(x) + v(\hat{n}(x))$, so CDS/PDS certificates are modality-neutral. WDL then uses residuation to synthesize weakest prefixes/suffixes against a plan target S while honoring the same motive set (CDS cone or PDS cover). The algebraic guarantees (join-safety, compositionality) carry over unchanged.

Composing with neural options (worked composition). Suppose a neural option o_{hall} (a learned corridor-approach macro) and a logical macro o_{flip} are both CDS/PDS-admitted. With a plan-level target S on a start-side strip Ω , the weakest prefix against suffix $B := T_{o_{\text{hall}}}$ is $A^* = S/B$. If A^* demands “reach a switch-adjacent cell with nonnegative expected margin,” the logical macro (or an evolutionary variant) provides precisely that prefix; the resulting $A^* \otimes B$ realizes S while remaining the *weakest* sufficient decomposition. If PDS is used with two weights (safety, speed), we can require improvement by ε on safety while allowing at most ρ downside on speed, and the same A^* applies componentwise.

A practical loop (logic + evolution + MDN).

1. *Propose*: rules/macros (logic), programs (evolution), options (neural).
2. *Summarize*: fit or bound $(\hat{r}, \hat{n})$; attach error radii.
3. *Motive geometry*: MDN emits W or W_{ref} (anchors/extreme rays/samples).
4. *Screen*: apply CDS or PDS (budgeted/coverage); admit passing generators.
5. *Compose*: if S is present, compute weakest prefixes/suffixes by residuation and approximate from the library.

Error-aware certificates (recap). With $|\hat{r} - r| \leq \varepsilon_r$ and $\|\hat{n} - n\|_\infty \leq \varepsilon_n$, CDS certificates subtract $\varepsilon_r + \|W\|_{\max,1}\varepsilon_n$; PDS uses componentwise slacks $\varepsilon_r + \|w^{(k)}\|_1\varepsilon_n$. MDN uncertainty is handled by conservative cones/covers or by adding reference rays.

Mixtures and sufficiency (informal). A practical division of labor is: gradients (or PC) for perception and continuous heads; logic for invariants and weakest pre/suffixes; evolution for non-smooth procedures. All export compatible $(\hat{r}, \hat{n})$ and integrate cleanly with CDS/PDS and WDL; the MDN focuses search by learning the motive slice.

5 Tiny two-room illustrations

We now present a series of simple examples, varying on the toy example from Sutton et al [1] in different ways to illustrate the new mechanisms we’ve introduced above. These two-room case studies that progressively stress the CDS/PDS admission rules and their interaction with planning. Each example exposes a trade-off between immediate costs and delayed value, and shows how the admission rule selects useful generators (options/macros) that speed approximate value iteration (AVI).

For most of these examples, we ignore the MDN, because it’s not important for the points we’re making. The penultimate example pertains specifically to the MDN. To incorporate the MDN in the other examples, we would simply replace fixed or hand-chosen motive sets by MDN-induced ones. Concretely:

1. Wherever a cone W appears in a CDS test, take W to be the positive hull (or box) implied by the MDN’s current sub-aspects and anchors; for boxes, the CDS infimum is still closed-form by per-coordinate extremes.
2. Wherever a finite cover W_{ref} appears in a PDS test, draw it from MDN extreme rays or samples; the same vector/Pareto tests apply.
3. Disentanglement regularizers (orthogonality, sparsity, total-correlation, anchors) and precision gating keep the MDN’s sub-aspects stable and interpretable during the examples below.

Also, for most of these examples, we assume expected-reward as a goal, for simplicity and because this is standard in the RL world. However one could easily present similar examples for other sorts of examples. For instance, a risk-sensitive rendering can be obtained by the uniform substitution $B_o \mapsto B_o^p$ (e.g., entropic risk or CVaR) in the backed-up value. The final example gives an example with an entropic-risk objective, in which the preferred option differs from the preferred expected-value option for the same situation. .

5.1 Basic Two-room example

We consider a standard two-room gridworld with a narrow door (hallway) between the left room (start side) and the right room (goal side). The agent begins on the left; a band of cells adjacent to the door on the start side yields a per-step penalty $-c$ (“gray band”). Everywhere else the step reward is 0 (we focus on the pre-goal segment so that option returns at the hallway). Let $\gamma \in (0, 1)$ be the discount.

Let $x \in \mathbb{R}^d$ denote a feature vector. For concreteness we use one-hot cell features together with a distinguished hallway feature h ; denote by $x_h(s)$ the hallway-feature component in state s . The value approximation is linear,

$$\hat{v}(s; w) = w^\top x(s),$$

and planning uses expectation models for actions and options,

$$\hat{r}(x, o) \approx \mathbb{E} \left[\sum_{t=1}^K \gamma^{t-1} R_t \mid x_0 = x, o \right], \quad \hat{n}(x, o) \approx \mathbb{E} \left[\gamma^K x_K \mid x_0 = x, o \right],$$

where K is the (random) stopping time of option o .

Two hallway options and their summaries We compare two options that start at the same state s_0 in the left room and both *return at the hallway cell* (so their post-return value-to-go is the same value function evaluated at the hallway):

- o_{SP} (shortest-path): walks to the hallway through the gray band in K_{SP} steps (incurs m consecutive $-c$ rewards starting at step index t_s).
- o_{RR} (reward-respecting, roundabout): detours around the gray band, reaches the hallway in $K_{\text{RR}} = K_{\text{SP}} + dK$ steps, and avoids the m negative cells entirely.

Because both return at the hallway, their successor features agree on the hallway component: $x_h(S_K) = 1$ at return. The per-option backed-up value is

$$B_o(x(s_0); w) = \hat{r}(x(s_0), o) + w^\top \hat{n}(x(s_0), o).$$

Let $\Delta B(w) := B_{o_{\text{RR}}}(x(s_0); w) - B_{o_{\text{SP}}}(x(s_0); w)$. Decompose

$$\Delta B(w) = \underbrace{\Delta R}_{\text{cumulative reward gap}} + \underbrace{(\gamma^{K_{\text{RR}}} - \gamma^{K_{\text{SP}}})}_{\text{discount gap } < 0} w_h,$$

where w_h is the hallway weight and

$$\Delta R = (\text{RR reward}) - (\text{SP reward}) = c \sum_{i=0}^{m-1} \gamma^{(t_s+i)-1} > 0,$$

because o_{SP} suffers m discounted penalties in the gray band while o_{RR} avoids them.

Cone-dominant test and closed-form threshold Let the motive cone be the single-coordinate box

$$W = \{w : 0 \leq w_h \leq w_{\max}\},$$

with all other weights held fixed or irrelevant for this comparison. The CDS statewise margin is

$$m_{o_{\text{RR}}}(x(s_0)) = \inf_{w \in W} \Delta B(w) = \Delta R - (\gamma^{K_{\text{SP}}} - \gamma^{K_{\text{RR}}}) w_{\max},$$

since $\gamma^{K_{\text{SP}}} > \gamma^{K_{\text{RR}}}$ and the worst case picks $w_h = w_{\max}$. The cone-dominance condition $m_{o_{\text{RR}}} \geq 0$ becomes the explicit threshold

$$w_{\max} \leq \frac{\Delta R}{\gamma^{K_{\text{SP}}} - \gamma^{K_{\text{RR}}}} = \frac{c \sum_{i=0}^{m-1} \gamma^{t_s+i-1}}{\gamma^{K_{\text{SP}}} (1 - \gamma^{dK})}.$$

Thus o_{RR} is admitted by CDS on state s_0 whenever the hallway weight uncertainty is not too large relative to the reward savings ΔR and the discount gap $\gamma^{K_{\text{SP}}} - \gamma^{K_{\text{RR}}}$.

Numerical instance and sensitivity Take $\gamma = 0.99$, $c = 1$, $m = 3$ consecutive gray cells starting at $t_s = 4$ (so penalties occur at steps 4, 5, 6), and a detour cost $dK = 4$ steps. Then

$$\Delta R = \sum_{t=4}^6 \gamma^{t-1} \approx 0.99^3 + 0.99^4 + 0.99^5 \approx 2.881885.$$

If $K_{\text{SP}} \approx 7$ (so $\gamma^{K_{\text{SP}}} \approx 0.93$), the discount gap is

$$\gamma^{K_{\text{SP}}} - \gamma^{K_{\text{RR}}} = \gamma^{K_{\text{SP}}} (1 - \gamma^{dK}) \approx 0.93 \times (1 - 0.99^4) \approx 0.0367.$$

Hence the CDS threshold is

$$w_{\max} \leq \frac{2.881885}{0.0367} \approx 78.5.$$

Interpretation: across any hallway-weight cone with $w_h \leq 78.5$, the round-about reward-respecting hallway option robustly outperforms the shortest-path option at s_0 (and by symmetry on most nearby left-room states as well).

Sensitivity. From the closed form

$$w_{\max}^* = \frac{c \sum_{i=0}^{m-1} \gamma^{t_s+i-1}}{\gamma^{K_{\text{SP}}} (1 - \gamma^{dK})},$$

we see: (i) larger m or c increases ΔR and enlarges the admissible cone; (ii) larger dK increases $1 - \gamma^{dK}$ and shrinks the cone; (iii) larger K_{SP} shrinks the cone via the $\gamma^{K_{\text{SP}}}$ factor; (iv) higher γ increases both numerator and denominator, with a net effect depending on $(m, dK, t_s, K_{\text{SP}})$.

From statewise to distributional margins Let ν be a planning-update state distribution concentrated on the left-room strip that feeds the hallway. The distributional CDS criteria

$$\mathbb{E}_{x \sim \nu} [m_{o_{\text{RR}}}(x)] \geq \varepsilon \quad \text{or} \quad \Pr_{x \sim \nu} [m_{o_{\text{RR}}}(x) \geq 0] \geq 1 - \delta$$

admit o_{RR} if it passes most states. Empirically, ΔR and K vary mildly over that strip; the same threshold analysis applies with small fluctuations.

Robust CDS with model error Suppose $|\hat{r} - r| \leq \epsilon_r(x, o)$ and $\|\hat{n} - n\|_\infty \leq \epsilon_n(x, o)$. For the single-coordinate cone with $0 \leq w_h \leq w_{\max}$,

$$\inf_{w \in W} (\hat{r} + w^\top \hat{n}) \geq \inf_{w \in W} (r + w^\top n) - \epsilon_r - w_{\max} \epsilon_n,$$

so a robust lower bound on the margin is

$$\hat{m}_{o_{RR}}^{\text{rob}}(x(s_0)) = \Delta \hat{R} - (\gamma^{K_{SP}} - \gamma^{K_{RR}}) w_{\max} - \epsilon_r^\Delta(x) - w_{\max} \epsilon_n^\Delta(x),$$

with $\Delta \hat{R}$ the estimated reward gap and ϵ^Δ collecting worst-case differences versus the baseline. Admission can use $\hat{m}^{\text{rob}} \geq 0$ to maintain safety under uncertainty.

Planning impact: max-win rate and AVI speed Define the *max-win rate* of an option as the fraction of states where its backed-up value wins the planner’s $\max_o$ during AVI updates. Since CDS certifies $B_{o_{RR}} \geq B_{\text{base}}$ on a large measure of the left-room states for all $w \in W$, adding o_{RR} increases the join $\bigvee$ by a generator that is often selected, thereby increasing the effective contraction speed of asynchronous AVI on those states. By contrast, the shortest-path option o_{SP} often *loses* the max when w_h is not extremely large, and can even slow planning by providing high-variance, low-value lookahead targets.

Predictive-coding execution of the same test In a predictive-coding network (PCN), the same quantities are computed locally:

Given $x(s_0)$, current w , and models $(\text{rhat}, \text{nhat})$:

```

For o in {SP, RR}:
  B[o] = rhat(x,o) + w^T nhat(x,o)
  V_target = logsumexp(B[o]/tau) * tau
  eps_v = V_target - w^T x(s0)          (AVI residual)
  baseline b = argmax over admitted generators at x(s0)
  Delta_r = rhat(x,RR) - rhat(x,b)
  Delta_n = nhat(x,RR) - nhat(x,b)
  pick w_ext = w_max if Delta_n_h < 0 else 0
  m_RR = Delta_r + Delta_n_h * w_ext    (box-cone worst-case)
  eps_cds = max(0, eps_margin - m_RR)   (CDS violation)
  Local updates: w, rhat, nhat with eps_v, eps_cds

```

No replay or global backpropagation is needed; the max and worst-case w selection are local gates.

WDL residual on the hallway: weakest prefix construction Let Ω be the strip of left-room states feeding the hallway, and let S encode a plan-level lower bound $[S \hat{v}(\cdot; w)](x) \geq b(x)$ for all $x \in \Omega$ and $w \in W$ (e.g., a threshold on the backed-up value needed to reach the goal region in a few extra steps). Fix the suffix $B := T_{o_{\text{RR}}}$; the *weakest* prefix that suffices is the residual

$$A^* = S/B,$$

computed in the (residuated) hypothesis quantale. Interpreted as summaries, A^* specifies target $(\hat{r}, \hat{n})$ for a “corridor approach” macro: accumulate non-negative reward and hand off to the hallway with enough discounted value remaining. A greedy approximation from primitives that covers the constraints on $\Omega \times W$ yields a short macro that, together with B , meets S while remaining as weak (least committed) as possible.

Algorithmic skeleton for the tiny world

```
Initialize admitted set  $B \leftarrow \{\text{primitive actions}\}$ ; pool
 $O \leftarrow \{o_{\text{SP}}, o_{\text{RR}}\}$ 
Loop: 1) Learn/refresh expectation models  $(\hat{r}, \hat{n})$  for  $B \cup O$  from
experience 2) For each  $x$  in a sweep over left-room strip:
compute robust margins  $m_o(x)$  for  $o \in O$  using the box cone  $W$ 
3) Admit any  $o$  with  $E_x[m_o(x)] \geq \varepsilon$  (or  $P_x[m_o(x) \geq 0] \geq 1 - \delta$ ):
 $B \leftarrow B \cup \{o\}$  4) If a plan spec  $S$  on  $\Omega$  is present: compute
 $A^* = S/T_{o_{\text{RR}}}$ ; approximate  $A^*$  with a short macro; add to  $B$  5)
Run asynchronous AVI updates on a schedule over the grid
states
Until convergence or budget
```

What this shows

- A simple closed-form CDS threshold certifies when the reward-respecting hallway beats the shortest path for an entire cone of hallway weights.
- The admission of o_{RR} raises the planner’s join on the left side and increases the max-win rate, thereby speeding AVI.
- The PCN executes the same max and worst-case tests with local gates and error-driven updates.
- The WDL residual S/B constructs the weakest prefix macro needed to reliably reach the hallway with sufficient backed-up value, formalizing task decomposition in the same algebra used for planning.

5.2 A simple case where a cone-dominant option beats the best single-weight choice

Setup (two-room variant) Consider the standard two-room grid with a single hallway. Discount $\gamma = 0.99$. On the start side there is a “gray band” of $m = 3$ cells that each incur a per-step penalty $-c$ with $c = 1$; these are encountered by the shortest path at step indices $t = 4, 5, 6$. Let the linear value be $\hat{v}(s; w) = w^\top x(s)$ with a distinguished hallway feature h ; at the hallway, $x_h = 1$. As in expectation-model planning, the backed-up value for an option o starting at s_0 is

$$B_o(s_0; w) = \hat{r}(s_0, o) + w^\top \hat{n}(s_0, o) = \hat{r}(s_0, o) + w_h \gamma^{K_o},$$

since both options considered will return *at* the hallway (so the only relevant successor-feature component is the hallway, with discount γ^{K_o} for the option duration K_o).

Three hallway options We compare three hallway-reaching options that return at the hallway:

- **SP (rush):** shortest path across the gray band. Duration $K_{\text{SP}} = 7$. Reward $\hat{r}_{\text{SP}} = -\gamma^3 - \gamma^4 - \gamma^5$ (the three discounted -1 penalties).
- **RR-detour (avoid):** a reward-respecting hallway option that detours to avoid all gray cells. Duration $K_{\text{AV}} = 11$ (so $dK = 4$ longer). Base reward 0, but suppose the detour passes a “windy” strip that causes an expected one-shot penalty of -0.4 around step 8, contributing $\approx -0.4\gamma^7$ to $\hat{r}_{\text{AV}}$.
- **Skirt-1 (middle):** a third path that skirts the band, encountering *one* gray cell (penalty at step 5) and no windy hazard. Duration $K_{\text{SK}} = 9$, reward $\hat{r}_{\text{SK}} = -\gamma^4$.

All three are consistent with the GVF/option formalism (same cumulant R , different termination geometry). The first two correspond to common reward-respecting extremes (very high hallway bonus $\Rightarrow$ rush; moderate bonus $\Rightarrow$ avoid). The third is the candidate that a *cone-dominant* selection will favor, as we show next.

Backed-up values and pairwise advantages Let $\Delta B_{A|B}(w_h) := B_A(s_0; w) - B_B(s_0; w)$, with the understanding that only w_h matters here.

$$\Delta B_{\text{AV}|\text{SP}}(w_h) = \underbrace{(\hat{r}_{\text{AV}} - \hat{r}_{\text{SP}})}_{\Delta R_{\text{AV}|\text{SP}}} - \underbrace{(\gamma^{K_{\text{SP}}} - \gamma^{K_{\text{AV}}})}_{\Delta \gamma_{\text{SP} \rightarrow \text{AV}} > 0} w_h,$$

$$\Delta B_{\text{SK}|\text{SP}}(w_h) = \underbrace{(\hat{r}_{\text{SK}} - \hat{r}_{\text{SP}})}_{\Delta R_{\text{SK}|\text{SP}}} - \underbrace{(\gamma^{K_{\text{SP}}} - \gamma^{K_{\text{SK}}})}_{\Delta \gamma_{\text{SP} \rightarrow \text{SK}} > 0} w_h.$$

Numerically (with $\gamma = 0.99$):

$$\gamma^3 = 0.970299, \gamma^4 = 0.960596, \gamma^5 = 0.950990, \gamma^7 = 0.932065, \gamma^9 = 0.913517, \gamma^{11} = 0.895338.$$

Hence

$$\hat{r}_{\text{SP}} = -(\gamma^3 + \gamma^4 + \gamma^5) \approx -2.881885, \quad \hat{r}_{\text{AV}} \approx -0.4 \gamma^7 \approx -0.372826, \quad \hat{r}_{\text{SK}} = -\gamma^4 \approx -0.960596,$$

$$\Delta R_{\text{AV}|\text{SP}} \approx 2.509059, \quad \Delta R_{\text{SK}|\text{SP}} \approx 1.921289,$$

$$\Delta \gamma_{\text{SP} \rightarrow \text{AV}} = \gamma^7 - \gamma^{11} \approx 0.036727, \quad \Delta \gamma_{\text{SP} \rightarrow \text{SK}} = \gamma^7 - \gamma^9 \approx 0.018548.$$

What a single reward-respecting choice would pick A reward-respecting pipeline typically fixes the *current* hallway weight $w_h = w_h^{(0)}$ (or equivalently fixes the stopping-bonus setting) and selects the hallway option that *maximizes* $B_o(s_0; w^{(0)})$. For a moderate current weight, say $w_h^{(0)} = 10$,

$$\Delta B_{\text{AV}|\text{SP}}(10) \approx 2.509059 - 0.036727 \times 10 = 2.141789,$$

$$\Delta B_{\text{SK}|\text{SP}}(10) \approx 1.921289 - 0.018548 \times 10 = 1.735809.$$

Thus the best *single* reward-respecting hallway choice under $w_h^{(0)} = 10$ is the **RR-detour** (avoid) option.

What the cone-dominant criterion picks Now suppose the true motive is uncertain and MetaMo exposes a hallway-weight cone

$$W = \{w_h : 0 \leq w_h \leq 100\}.$$

The CDS *robust margin* of a candidate A vs. baseline B is the worst-case advantage over W :

$$m_{A|B} = \inf_{w_h \in W} \Delta B_{A|B}(w_h).$$

For linear-in- w_h forms, the worst case is attained at the extreme $w_h = 100$ because the slope is negative:

$$m_{\text{AV}|\text{SP}} = \Delta R_{\text{AV}|\text{SP}} - \Delta \gamma_{\text{SP} \rightarrow \text{AV}} \cdot 100 \approx 2.509059 - 3.672709 = -1.163650 (< 0),$$

$$m_{\text{SK}|\text{SP}} = \Delta R_{\text{SK}|\text{SP}} - \Delta \gamma_{\text{SP} \rightarrow \text{SK}} \cdot 100 \approx 1.921289 - 1.854810 = +0.066479 (> 0).$$

Conclusion: Over the whole cone $W = [0, 100]$, the *skirt-1 (middle)* option has a *nonnegative* robust advantage vs. the shortest path, while the *RR-detour* option fails the robust test (its advantage becomes negative for large w_h). Therefore, CDS admits **Skirt-1** and rejects **RR-detour** under this W .

Why this matters (planner behavior) Planning uses $\max_o B_o(s_0; w)$ inside AVI. Under motive uncertainty reflected by W , the *admitted* generator should win the max *uniformly across* $w_h \in W$ on a large set of states. In this example:

- A single reward-respecting choice tuned to $w_h^{(0)} = 10$ would keep **RR-detour**, which loses the max for large w_h because its longer duration reduces γ^K too much.
- The cone-dominant choice **Skirt-1** balances reward savings and discount: it gives up one -1 but shortens the path by two steps vs. the detour, halving the discount gap slope (0.018548 vs. 0.036727). That smaller slope preserves a *positive* worst-case advantage across W .

Hence, with a broad hallway-weight cone, CDS picks an option that *wins the planner’s max more often across the entire cone*, improving AVI’s speed uniformly, whereas the best single reward-respecting choice (under a fixed $w_h^{(0)}$) is brittle.

Compact summary inequality The break-even thresholds (largest w_h for which each option still beats SP) are

$$w_h^{\max}(\text{AV}|\text{SP}) = \frac{\Delta R_{\text{AV}|\text{SP}}}{\Delta \gamma_{\text{SP} \rightarrow \text{AV}}} \approx \frac{2.509059}{0.036727} \approx 68.3,$$

$$w_h^{\max}(\text{SK}|\text{SP}) = \frac{\Delta R_{\text{SK}|\text{SP}}}{\Delta \gamma_{\text{SP} \rightarrow \text{SK}}} \approx \frac{1.921289}{0.018548} \approx 103.6.$$

For any cone $W = [0, w_{\max}]$ with $w_{\max} \in (68.3, 103.6]$ (e.g., $w_{\max} = 100$ above), **Skirt-1** is cone-dominant while **RR-detour** is not, even though at the current $w_h^{(0)} = 10$ the reward-respecting choice would have picked **RR-detour**.

Takeaways Summing up what we’ve learned from these thought-experiments so far ... among other things, we can see that: (i) Reward-respecting sub-tasks work because they enlarge backed-up values where planning cares; (ii) CDS makes this robustness explicit over a cone of motives; (iii) a predictive-coding implementation performs CDS naturally with online, local updates; (iv) the quantale view and residuation let MetaMo and WDL decompose robust goals into weakest-needed components; (v) the same summaries ($\hat{r}, \hat{n}$) carry CDS into logic and evolutionary settings.

5.3 A two-room variant where explicit logical inference finds the right goal decomposition quickly

Setup: Key-door chain with prerequisites. The agent must reach a goal behind three sequentially locked doors (Red, Blue, Green). Each door requires its matching key, and keys are in boxes with tool requirements:

- Red key: requires hammer (located in room A, 3 steps from start)
- Blue key: box unlocked by red key (located near red door)
- Green key: requires scissors (located in room B, 4 steps from start)

Base step cost is -1 ; attempting a door without its key incurs -5 penalty and returns agent to start. Discount $\gamma = 0.99$. The hallway feature h is reached after passing all three doors.

Three candidate strategies:

- **Random exploration (RAND):** try doors/boxes in arbitrary order. Expected time to solution $K_{RAND} \approx 45$ steps due to resets.
- **Greedy (GREED):** always approach nearest unopened door. Hits penalties repeatedly; $K_{GREED} \approx 35$ with multiple resets.
- **Logical decomposition (LOGIC):** backward-chain from goal to derive prerequisite order.

Logical rules for backward chaining (PLN-style):

1. $\text{AtGoal} \Leftarrow \text{PassedGreen} \wedge \text{AtHallway}$ with $(s, c) = (1.0, 1.0)$
2. $\text{PassedGreen} \Leftarrow \text{HasGreenKey} \wedge \text{AtGreenDoor}$ with $(1.0, 1.0)$
3. $\text{HasGreenKey} \Leftarrow \text{HasScissors} \wedge \text{AtGreenBox}$ with $(1.0, 1.0)$
4. $\text{PassedBlue} \Leftarrow \text{HasBlueKey} \wedge \text{AtBlueDoor}$ with $(1.0, 1.0)$
5. $\text{HasBlueKey} \Leftarrow \text{HasRedKey} \wedge \text{AtBlueBox}$ with $(1.0, 1.0)$
6. $\text{PassedRed} \Leftarrow \text{HasRedKey} \wedge \text{AtRedDoor}$ with $(1.0, 1.0)$
7. $\text{HasRedKey} \Leftarrow \text{HasHammer} \wedge \text{AtRedBox}$ with $(1.0, 1.0)$

Backward chaining from goal immediately yields the unique feasible order:

Start $\rightarrow$ RoomA $\rightarrow$ Hammer $\rightarrow$ RedBox $\rightarrow$ RedKey $\rightarrow$ RedDoor $\rightarrow$ BlueBox $\rightarrow$ BlueKey $\rightarrow$
 BlueDoor $\rightarrow$ RoomB $\rightarrow$ Scissors $\rightarrow$ GreenBox $\rightarrow$ GreenKey $\rightarrow$ GreenDoor $\rightarrow$ Goal

This takes $K_{LOGIC} = 18$ steps with zero penalties.

Backed-up values and CDS comparison. Let the motive cone be $W = \{w_h : 0 \leq w_h \leq 100\}$ where w_h weights the hallway feature. The backed-up values are:

$$B_{RAND}(s_0; w_h) = -45 + \gamma^{45} w_h \approx -45 + 0.639 w_h \quad (1)$$

$$B_{GREED}(s_0; w_h) = -(35 + 3 \cdot 5) + \gamma^{35} w_h \approx -50 + 0.706 w_h \quad (2)$$

$$B_{LOGIC}(s_0; w_h) = -18 + \gamma^{18} w_h \approx -18 + 0.836 w_h \quad (3)$$

For CDS admission against the baseline (RAND), we need:

$$m_{LOGIC}(s_0) = \inf_{w_h \in W} (B_{LOGIC} - B_{RAND}) = (-18 + 45) + (0.836 - 0.639) \cdot 0 = 27 > 0$$

The logical decomposition dominates uniformly over the entire cone W .

Why gradient methods struggle here:

- **Credit assignment depth:** The value of visiting RoomA first only manifests 10+ steps later when the red key enables the blue key. With sparse rewards, gradient methods require exponentially many episodes in the chain length.
- **Combinatorial explosion:** With 3 doors, 3 keys, 2 tools, there are $O(8!)$ possible action sequences, but only one avoids all penalties.
- **Reset penalties:** Each wrong attempt costs -5 and resets position, creating a severe exploration challenge for policy gradient methods.

Search complexity: Logical backward chaining solves this in $O(n)$ inference steps for n prerequisites. Gradient methods require $O(b^n)$ samples where b is the branching factor, absent curriculum learning.

5.4 A two-room example where quantified logical reasoning outperforms gradient search

Setup: Resource allocation with universal constraints. A delivery robot must serve $n = 5$ locations from $m = 3$ depots. Each location i has demand $d_i \in \{2, 3, 5, 4, 3\}$. The robot has capacity $C = 8$ and must satisfy:

- $\forall i : \text{delivered}(i) \leq d_i$ (don't over-deliver)
- $\sum_i \text{delivered}(i) \geq 0.9 \cdot \sum_i d_i = 15.3$ (meet 90% of total demand)
- Minimize total distance traveled

Depots have inventory: $I_1 = 6, I_2 = 7, I_3 = 8$. Distance matrix gives costs between all points. Discount $\gamma = 0.99$.

Quantified logical formulation:

$\exists \text{route} \exists \text{allocation} : \left[\forall i \in \text{locations} : \text{served}(i, \text{allocation}) \wedge \text{feasible}(\text{route}, C) \right]$

The logical solver:

1. Propagates constraints to eliminate infeasible depot combinations (e.g., depot 1 alone cannot meet 90% threshold)
2. Identifies minimal feasible sets: $\{D_2, D_3\}$ or $\{D_1, D_2, D_3\}$
3. Solves a small CSP for optimal allocation
4. Constructs route via nearest-neighbor with constraints

This yields solution with $K_{LOGIC} = 22$ steps, $\hat{r}_{LOGIC} = -22$.

Gradient baseline: A neural policy must learn the feasible region through trial and error:

- Over-delivery at any location causes complete failure (reward $-\infty$)
- Under-delivery below 90% causes failure (reward $-\infty$)
- The feasible region is a narrow polytope in $\mathbb{R}^{15}$ (5 locations 3 depots)

Empirically, gradient methods find solutions with $K_{GRAD} \approx 28$ after $\sim 10,000$ episodes, with many catastrophic failures during learning.

Backed-up values under motive cone $W = \{w_h : 0 \leq w_h \leq 100\}$:

$$B_{LOGIC}(s_0; w_h) = -22 + \gamma^{22} w_h \approx -22 + 0.802 w_h \quad (4)$$

$$B_{GRAD}(s_0; w_h) = -28 + \gamma^{28} w_h \approx -28 + 0.758 w_h \quad (5)$$

The CDS margin for logical planning:

$$m_{LOGIC}(s_0) = \inf_{w_h \in W} (B_{LOGIC} - B_{GRAD}) = 6 + (0.802 - 0.758) \cdot 0 = 6 > 0$$

Key insight: The $\forall \exists$ structure with hard constraints creates a needle-in-haystack problem for gradients but is natural for constraint propagation. The logical approach achieves better solutions faster by eliminating infeasible regions analytically rather than learning them through penalty signals.

5.5 A small RL domain where logic + evolution + gradients strictly dominate all pairs

Setup: Assembly with perception, rules, and kinematics. A robot must stack $n = 6$ blocks to build a structure. Each block has:

- Visual markers: orientation notches (continuous angle $\theta \in [0, 2\pi]$) and weight indicators (light/medium/heavy) visible only through perception
- Safety constraints: heavy blocks must be below light ones; notches must align within ± 5
- Kinematic constraints: certain orderings create unreachable positions given arm geometry

Discount $\gamma = 0.99$. Successful assembly reaches goal with reward +100; violations incur -50 ; each step costs -1 .

Four candidate approaches:

1. **Neural only (NN)**: CNN perceives markers, policy network learns stacking order
2. **Neural + Logic (NN+LOG)**: CNN perceives, logical rules enforce safety constraints
3. **Neural + Evolution (NN+EVO)**: CNN perceives, evolutionary search finds kinematically feasible orderings
4. **All three (NN+LOG+EVO)**: CNN perceives, logic prunes unsafe, evolution searches feasible orderings

Performance breakdown:

- **NN only**: Learns features well but violates weight constraints during exploration. Success rate: 35%. Average $K_{NN} = 18$ when successful, but $\hat{r}_{NN} = 0.35(100 - 18) - 0.65 \cdot 50 = -3.8$
- **NN+LOG**: Respects constraints but gets stuck when chosen order is kinematically infeasible (must reset). Success rate: 60%. $\hat{r}_{NN+LOG} = 0.6(100 - 15) - 0.4 \cdot 20 = 43$
- **NN+EVO**: Finds reachable orderings but violates safety rules. Success rate: 45%. $\hat{r}_{NN+EVO} = 0.45(100 - 14) - 0.55 \cdot 50 = 11.2$

- **NN+LOG+EVO:** Perceives markers, respects safety, finds feasible orderings. Success rate: 95%. $\hat{r}_{ALL} = 0.95(100 - 12) - 0.05 \cdot 10 = 83.1$

Backed-up values and CDS analysis: With motive cone $W = \{w_h : 0 \leq w_h \leq 100\}$:

$$B_{NN}(s_0; w_h) = -3.8 + \gamma^{18}w_h \approx -3.8 + 0.836w_h \quad (6)$$

$$B_{NN+LOG}(s_0; w_h) = 43 + \gamma^{15}w_h \approx 43 + 0.861w_h \quad (7)$$

$$B_{NN+EVO}(s_0; w_h) = 11.2 + \gamma^{14}w_h \approx 11.2 + 0.869w_h \quad (8)$$

$$B_{ALL}(s_0; w_h) = 83.1 + \gamma^{12}w_h \approx 83.1 + 0.887w_h \quad (9)$$

The best pairwise method is NN+LOG. The CDS margin for the tri-modal approach:

$$m_{ALL}(s_0) = \inf_{w_h \in W} (B_{ALL} - B_{NN+LOG}) = (83.1 - 43) + (0.887 - 0.861) \cdot 0 = 40.1 > 0$$

Why each component is necessary:

- **Neural perception:** Orientation angles are continuous and weight indicators are visual patterns that cannot be specified symbolically
- **Logical rules:** Safety constraints are absolute—violating them causes structural failure
- **Evolutionary search:** The kinematic feasibility function is non-differentiable (collision detection) and non-convex

Interaction through shared representation: The three methods compose via the same $(\hat{r}, \hat{n})$ interface:

- Neural: $\hat{n}$ encodes perceived block features
- Logic: Guards prune candidates with safety violations, setting $\hat{r} = -\infty$ for illegal orderings
- Evolution: Searches over permutations that pass logical guards, optimizing for kinematic reachability

The tri-modal approach achieves $B_{ALL} - \max(B_{NN}, B_{NN+LOG}, B_{NN+EVO}) \geq 40.1$ uniformly over W , demonstrating strict dominance.

5.6 Two-room case where Pareto admission outperforms cone-dominance

Finally, returning to the simple neural-leaning case, we highlight a simple situation where PDS will outperform CDS. Of course, this could be intersected with the previous examples, and we could formulate simple examples highlighting the value of logic, evolution or their combination in conjunction with PDS.

Setup Consider the two-room grid with a single hallway. Discount $\gamma = 0.99$. From the start s_0 on the left, the agent reaches the corridor mouth at step $t = 4$ and the hallway at $t = 7$ via the straight corridor. The corridor has three hazardous cells that each incur -1 when traversed. We evaluate options by their backed-up value

$$B_o(s_0; w_h) = \hat{r}(s_0, o) + w_h \gamma^{K_o},$$

where $w_h \geq 0$ is the hallway weight and K_o is the option's random duration (here treated as deterministic for simplicity). As baseline we use the straight corridor path (the plan the primitives will realize under AVI), abbreviated SP:

$$K_{\text{SP}} = 7, \quad \hat{r}_{\text{SP}} = -(\gamma^3 + \gamma^4 + \gamma^5) \approx -2.881885.$$

Two candidate options We define two hallway-reaching options that both *return at the hallway*:

(i) **Detour-avoid (LO).** Circumvent the three hazardous cells at the cost of two extra steps.

$$K_{\text{LO}} = 9, \quad \hat{r}_{\text{LO}} = 0.$$

Relative to SP, the backed-up improvement is

$$\Delta B_{\text{LO|SP}}(w_h) = \underbrace{(0 - (-2.881885))}_{\Delta R = 2.881885} - \underbrace{(\gamma^7 - \gamma^9)}_{0.018548} w_h = 2.881885 - 0.018548 w_h.$$

(ii) **Jump-shortcut (HI).** A risky jump crosses the corridor in one step (reducing time) but incurs an extra energy cost at entry and still hits one hazard cell.

$$K_{\text{HI}} = 6, \quad \hat{r}_{\text{HI}} = -(2 + 1)\gamma^3 \approx -2.910897.$$

Relative to SP,

$$\Delta B_{\text{HI}|\text{SP}}(w_h) = \underbrace{(-2.910897 - (-2.881885))}_{\Delta R = -0.029012} + \underbrace{(\gamma^6 - \gamma^7)}_{0.009415} w_h = -0.029012 + 0.009415 w_h.$$

A broad motive cone where CDS rejects both Let the motive cone be $W = \{w_h : 0 \leq w_h \leq 200\}$. Cone-dominant admission requires $\Delta B(w_h) \geq 0$ for *all* $w_h \in W$.

- $\Delta B_{\text{HI}|\text{SP}}(0) = -0.029012 < 0$ so **HI** fails CDS (it hurts very small hallway weights).
- $\Delta B_{\text{LO}|\text{SP}}(200) = 2.881885 - 0.018548 \times 200 = -0.827715 < 0$ so **LO** fails CDS (it hurts very large hallway weights).

Thus *no single* option is cone-dominant on W , and CDS admits neither. The planner remains with SP only.

Pareto admission with a finite weight cover admits both Choose a two-point reference cover of the cone:

$$W_{\text{ref}} = \{w^{(1)} = 10, w^{(2)} = 180\}.$$

Compute the improvements:

$$\Delta B_{\text{HI}|\text{SP}}(10) = -0.029012 + 0.009415 \times 10 = +0.065138$$

$$\Delta B_{\text{HI}|\text{SP}}(180) = -0.029012 + 0.009415 \times 180 = +1.665688.$$

So **HI** *strictly* Pareto-dominates SP on both reference weights.

$$\Delta B_{\text{LO}|\text{SP}}(10) = 2.881885 - 0.018548 \times 10 = +2.696405$$

$$\Delta B_{\text{LO}|\text{SP}}(180) = 2.881885 - 0.018548 \times 180 = -0.456755.$$

LO strongly improves the small weight and mildly worsens the very large weight.

Adopt budgeted Pareto admission with $(\varepsilon, \rho, q) = (0.05, 0.5, 1/2)$: improve by at least ε on at least a fraction q of the components, while allowing at most ρ downside on the others. Then

- **HI** passes *strict* PDS ($\rho = 0$ would suffice) since it improves both components.
- **LO** passes *budgeted* PDS: it improves $w^{(1)}$ by $\geq \varepsilon$ and is within the downside budget at $w^{(2)}$ ($-0.456755 \geq -\rho$).

By join-safety, adding either option cannot reduce the planner’s backed-up value at any reference weight; adding both enables the planner to pick the better one per weight.

Quantified advantage over CDS Suppose MetaMo’s motives place mass on $w^{(1)}$ and $w^{(2)}$ (e.g., switch between them with probability $1/2$). With only SP (the CDS outcome),

$$\Delta B_{\text{CDS}}(w^{(1)}) = \Delta B_{\text{CDS}}(w^{(2)}) = 0.$$

With Pareto-admitted $\{\text{HI}, \text{LO}\}$, the planner’s join yields per-weight improvements

$$\Delta B_{\text{PDS}}(w^{(1)}) = \max\{0.065138, 2.696405, 0\} = 2.696405,$$

$$\Delta B_{\text{PDS}}(w^{(2)}) = \max\{1.665688, -0.456755, 0\} = 1.665688.$$

Hence the expected backed-up improvement at s_0 under a 50% motive mixture is

$$\mathbb{E}[\Delta B_{\text{PDS}}] = \frac{1}{2}(2.696405 + 1.665688) \approx 2.181.$$

CDS yields zero; PDS yields a substantial uniform gain on the reference cover, and, by continuity, a neighborhood around it.

Planning-speed intuition The asynchronous AVI update at x targets $\max_o (\hat{r}(x, o) + \hat{v}(\hat{n}(x, o)))$. The *max-win rate* of an option (fraction of states where it wins this max) is a key driver of planning speed. Under CDS, with neither option admitted, the max-win rate stays at that of primitives only. Under PDS, LO wins the max on low-weight episodes and HI wins it on high-weight episodes, sharply increasing the aggregate max-win rate over the mixture and thereby reducing the number of look-ahead evaluations needed to reach a given value error.

Why Pareto is much better here

- The cone $W = [0, 200]$ is too broad for a single option to be uniformly dominant: low-weight motives prefer detours (reward savings), high-weight motives prefer speed (discount savings). CDS blocks both.
- Pareto admission with a small, representative cover W_{ref} certifies and admits *both* options, with an explicit downside budget to tolerate small losses where they do not matter for the covered motives.
- Because joins are safe, admitting both can never harm any covered motive, and in practice increases planning speed by providing the right macro for each motive episode.

Remarks

- The numerical constants can be adjusted; the qualitative pattern persists whenever (i) one option trades reward for time and (ii) the other trades time for reward, so that no single curve $\Delta B(w_h)$ lies above zero on a wide cone but each lies above zero on different regions.
- If desired, W_{ref} can be expanded adaptively (e.g., extreme rays or quantiles of MetaMo’s weight posterior). As the weight uncertainty narrows, one can tighten (ε, ρ, q) toward the CDS limit.

5.7 Two-room example: PC vs. continual backprop (CBP) for subgoal admission

Goal of the example (plain-language). We contrast a predictive coding (PC) agent with a continual backprop (CBP) agent on a tiny two-room task. Both agents maintain an option library and use subgoal admission. The PC agent realizes CDS/PDS *locally* (margin/gap residuals at the backed-up nodes) and co-learns a Motive Decomposition Network (MDN) that induces the motive geometry (W for CDS, or a finite cover W_{ref} for PDS). The CBP agent aggregates all objectives into a single global loss with fixed or slowly tuned weights λ , and uses a generate-and-test feature replacement scheme to keep plasticity. We show that the PC+MDN agent admits a useful mid-ground option earlier and more stably, because its motive geometry and credit assignment are factorized and local.

Setup. Start room $\rightarrow$ hallway (3 gray cells) $\rightarrow$ goal. Primitive actions cost -1 per step; entering gray cells adds an extra -1 . Three generators:

- **SP** (shortest path): length 6, gray steps 3.
- **RR** (roundabout): length 10, gray steps 0.
- **SK** (skirt): length 7, gray steps 1.

Let s_0 be the start state. Backed-up values use

$$B_o(s_0; w) = \hat{r}(s_0, o) + w^\top \hat{n}(s_0, o),$$

and we compare to the baseline $B_{\text{base}}(s_0; w)$ (here behaving like SP). We summarize each option relative to SP using two *future* features in $\hat{n}$: safety s (larger means fewer gray steps) and speed v (larger means fewer remaining steps). With all immediate costs absorbed into $\hat{r}$, the relative improvements are

$$\Delta n_{\text{RR},s} = +3, \quad \Delta n_{\text{RR},v} = -4, \quad \Delta n_{\text{SK},s} = +2, \quad \Delta n_{\text{SK},v} = -1.$$

For a nonnegative weight $w = (w_s, w_v) \geq 0$,

$$\Delta B_o(w) := B_o(s_0; w) - B_{\text{base}}(s_0; w) = w_s \Delta n_{o,s} + w_v \Delta n_{o,v}.$$

PC agent (CDS with MDN). The MDN emits sub-aspects $z = (z_{\text{safety}}, z_{\text{speed}})$ and a mixing $w = Az$. For CDS with boxes we use

$$W = \{(w_s, w_v) : w_s \in [w_s^{\min}, w_s^{\max}], w_v \in [w_v^{\min}, w_v^{\max}]\}.$$

The CDS margin at s_0 is $m_o(s_0) = \inf_{w \in W} \Delta B_o(w)$. Co-learning proceeds from CDS residuals plus disentanglement penalties (orthogonality, sparsity, total correlation, anchors) and precision shaping.

CBP agent (global composite loss). CBP tracks a single global loss $J = \lambda_{\text{model}} L_{\text{model}} + \lambda_s L_{\text{safety}} + \lambda_v L_{\text{speed}} + \dots$ with fixed or slowly tuned weights. For subgoal screening it computes a *scalarized* margin using an *effective* weight $w_{\text{CB}} = (\lambda_s, \lambda_v)$:

$$m_o^{\text{CBP}}(s_0) = \Delta B_o(w_{\text{CB}}) = \lambda_s \Delta n_{o,s} + \lambda_v \Delta n_{o,v}.$$

Plasticity is maintained by generate-and-test feature replacement, but credit still flows through the single global gradient.

Phase 0: cautious motives. Assume the MDN starts broad and uncertain,

$$W^{(0)} = \{(w_s, w_v) : w_s \in [0, 2.0], w_v \in [0, 3.0]\}.$$

Worst-case corner for both RR and SK is $(w_s^{\min}, w_v^{\max}) = (0, 3.0)$, hence

$$m_{\text{RR}}^{(0)} = 0 \cdot 3 + 3.0 \cdot (-4) = -12, \quad m_{\text{SK}}^{(0)} = 0 \cdot 2 + 3.0 \cdot (-1) = -3.$$

CDS admits neither (conservative start). Suppose CBP uses $w_{\text{CB}} = (0.7, 1.5)$ (speed over-weighted); then

$$m_{\text{RR}}^{\text{CBP}} = 0.7 \cdot 3 - 1.5 \cdot 4 = -3.9, \quad m_{\text{SK}}^{\text{CBP}} = 0.7 \cdot 2 - 1.5 \cdot 1 = -0.1.$$

CBP also admits neither.

MDN co-learning step (PC). CDS residuals show violations at small w_s and large w_v . Local PC residuals plus disentanglement push the MDN to raise $w_s^{\min}$ and reduce $w_v^{\max}$. After short online learning it outputs a tighter box

$$W^{(1)} = \{(w_s, w_v) : w_s \in [0.8, 1.6], w_v \in [0.6, 1.4]\}.$$

CDS now evaluates worst-case at $(0.8, 1.4)$:

$$m_{\text{RR}}^{(1)} = 0.8 \cdot 3 - 1.4 \cdot 4 = -3.2, \quad m_{\text{SK}}^{(1)} = 0.8 \cdot 2 - 1.4 \cdot 1 = +0.2.$$

PC admits SK (positive worst-case margin) and rejects RR.

CBP under the same stream. Because CBP relies on the global w_{CB} , gradients from safety and speed still interfere; even after feature replacement, if w_{CB} remains near $(0.7, 1.5)$ the SK margin is unchanged:

$$m_{\text{SK}}^{\text{CBP}} = 0.7 \cdot 2 - 1.5 \cdot 1 = -0.1 \quad (\text{still rejected}).$$

Only after explicit reweighting (e.g., drift-aware scheduling to $w_{\text{CB}} = (1.1, 1.2)$) does CBP admit SK:

$$m_{\text{SK}}^{\text{CBP}} = 1.1 \cdot 2 - 1.2 \cdot 1 = +1.0 \quad (\text{admitted, but later}).$$

In other words, PC+MDN crosses the admission threshold *without* global loss retuning; CBP requires manual or slow meta-adaptation of λ to align credit with the useful option.

PDS variant (both methods). Let the MDN propose a two-point cover $W_{\text{ref}}^{(1)} = \{(1.6, 0.6), (0.8, 1.4)\}$. Vector gaps for SK are

$$\Delta B_{\text{SK}} = (1.6 \cdot 2 + 0.6 \cdot (-1), 0.8 \cdot 2 + 1.4 \cdot (-1)) = (2.6, 0.2) \geq_{\text{P}} (0, 0),$$

so strict PDS admits SK immediately in PC. CBP can emulate PDS only via the same global weights or extra heads; unless λ are already aligned, it again lags.

Summary and lesson. The PC+MDN agent admits the mid-ground subgoal (SK) as soon as the MDN tightens W based on *local* CDS residuals; no global loss retuning is needed and disentanglement pressures keep the safety/speed axes clean. CBP, despite maintained plasticity via generate-and-test, routes credit through a single composite gradient and typically requires explicit reweighting before SK is admitted. In subgoal decomposition with changing motive blends, the PC+MDN factorization of motive geometry and local credit assignment yields earlier, stabler admission and sharper certificates.

5.8 MDN co-learning in action

Next we show, in a two-room toy, how the Motive Decomposition Network (MDN) *co-learns* with the CDS/PDS machinery and changes which option gets admitted. The only twist versus standard two-room setups is that the motive set used by CDS/PDS is *not fixed*: it is induced by the MDN and therefore tightens as data and residuals accumulate. This lets CDS start cautiously (broader, more conservative tests) and then sharpen to admit a useful intermediate option once the MDN has learned the safety vs. speed trade-off.

Setup. A start room is separated from the goal by a gray band of width 3. Primitive actions incur cost -1 per step; entering a gray cell inflicts an extra -1 . We consider three generators:

- **SP** (shortest path): crosses the gray band directly; length 6; gray steps 3.
- **RR** (roundabout): avoids all gray cells but adds 4 steps; length 10; gray steps 0.
- **SK** (skirt): avoids 2 of the 3 gray steps but adds 1 step; length 7; gray steps 1.

As usual, the backed-up value at the start state s_0 is

$$B_o(s_0; w) = \hat{r}(s_0, o) + w^\top \hat{n}(s_0, o),$$

and CDS compares a candidate o to the baseline (here, the primitive-only baseline behaves like SP in this toy) via

$$m_o(s_0) = \inf_{w \in W} (B_o(s_0; w) - B_{\text{base}}(s_0; w)).$$

Features and signs. We summarize each option relative to SP using two *future* features:

- a *safety* feature s that increases when gray exposure is reduced (good), and
- a *speed* feature v that increases when fewer steps remain (good).

Thus, relative to SP: RR improves safety by +3 gray steps avoided but is slower by 4 steps; SK improves safety by +2 and is slower by 1. We fold all immediate costs into $\hat{r}$ and put the remaining trade-off into $\hat{n}$. Using a linear value slice with nonnegative weights $w = (w_s, w_v) \geq 0$,

$$\Delta B_o(w) := B_o(s_0; w) - B_{\text{base}}(s_0; w) = \Delta \hat{r}_o + w_s \Delta n_{o,s} + w_v \Delta n_{o,v},$$

and in this toy we set $\Delta \hat{r}_o = 0$ (all path-length and gray penalties moved into $\hat{n}$). The deltas are:

$$\Delta n_{\text{RR},s} = +3, \quad \Delta n_{\text{RR},v} = -4 \quad \Delta n_{\text{SK},s} = +2, \quad \Delta n_{\text{SK},v} = -1.$$

MDN parameterization of motives. The MDN emits two sub-aspects $z = (z_{\text{safety}}, z_{\text{speed}})$ and a mixing $w = Az$. For CDS with boxes we use the implied bounds

$$W = \{(w_s, w_v) : w_s \in [w_s^{\min}, w_s^{\max}], w_v \in [w_v^{\min}, w_v^{\max}]\},$$

while for PDS we use a small MDN-induced cover $W_{\text{ref}} = \{(w_s^{(k)}, w_v^{(k)})\}_{k=1}^K$ (e.g., extreme rays or anchor samples). The MDN is co-trained from CD-S/PDS residuals plus disentanglement penalties (orthogonality/independence in z , sparsity, total-correlation, anchors) and precision shaping.

Phase 0 (initial, cautious motives). Suppose the MDN is uninformative at the start and exports a broad box

$$W^{(0)} = \{(w_s, w_v) : w_s \in [0, 2.0], w_v \in [0, 3.0]\}.$$

For a box and signs $\Delta n_{o,s} > 0$, $\Delta n_{o,v} < 0$, the worst case is at $(w_s^{\min}, w_v^{\max})$. Hence

$$m_{\text{RR}}^{(0)} = \inf_{w \in W^{(0)}} \Delta B_{\text{RR}}(w) = 0 \cdot 3 + 3.0 \cdot (-4) = -12,$$

$$m_{\text{SK}}^{(0)} = \inf_{w \in W^{(0)}} \Delta B_{\text{SK}}(w) = 0 \cdot 2 + 3.0 \cdot (-1) = -3.$$

With threshold $\varepsilon = 0$, neither RR nor SK is CDS-admitted yet (conservative start).

Co-learning step (what pushes the MDN). The CDS residuals indicate that violations occur at the corner (w_s small, w_v large): the planner fails when speed is weighted much more than safety. Anchors, precision shaping, and the residual stream nudge the MDN toward (i) lifting $w_s^{\min}$ away from 0 and (ii) reducing $w_v^{\max}$. Disentanglement penalties stabilize two nearly orthogonal axes corresponding to safety and speed.

Phase 1 (tighter, learned motives). After some experience, the MDN outputs

$$W^{(1)} = \{(w_s, w_v) : w_s \in [0.8, 1.6], w_v \in [0.6, 1.4]\}.$$

Again taking the minimizer $(w_s^{\min}, w_v^{\max}) = (0.8, 1.4)$,

$$m_{\text{RR}}^{(1)} = 0.8 \cdot 3 + 1.4 \cdot (-4) = 2.4 - 5.6 = -3.2 \quad (\text{still rejected}),$$

$$m_{\text{SK}}^{(1)} = 0.8 \cdot 2 + 1.4 \cdot (-1) = 1.6 - 1.4 = 0.2 \geq 0 \quad (\text{CDS-admitted}).$$

Thus, simply by tightening W based on co-learned motive structure, the mid-ground option SK becomes robustly admissible while RR remains too slow.

PDS variant (optional). Let the MDN also provide a two-point cover

$$W_{\text{ref}}^{(1)} = \{(1.6, 0.6), (0.8, 1.4)\}.$$

Vectorized gaps are

$$\Delta B_{\text{SK}} = (1.6 \cdot 2 + 0.6 \cdot (-1), 0.8 \cdot 2 + 1.4 \cdot (-1)) = (2.6, 0.2),$$

$$\Delta B_{RR} = (1.6 \cdot 3 + 0.6 \cdot (-4), 0.8 \cdot 3 + 1.4 \cdot (-4)) = (2.4, -3.2).$$

Strict PDS admits SK (both components nonnegative and one strictly positive); RR would require a budget ρ large enough to tolerate -3.2 on the second component or a coverage $q < 1$ with a matching budget. Either way, MDN-induced reference points make this test explicit and cheap.

Takeaways. (i) With a broad, conservative motive set, CDS declines both candidates. (ii) As the MDN co-learns (driven by CDS residuals and light disentanglement/precision shaping), the motive cone tightens to what the agent actually trades off, and CDS now admits the useful mid-ground option SK. (iii) A PDS view with MDN-chosen reference weights exposes exactly which directions an option helps or hurts, allowing budgeted or coverage-based admission when appropriate. The planner and all formulas are unchanged; only the source of the motive geometry becomes learned and adaptive.

5.9 Two-room example under entropic risk: when risk aversion prefers a detour

We now give an example where the objective is something other than expected reward. The general SubRep approach holds for general objectives, and we have stuck with expected reward throughout our previous examples just for simplicity’s sake.

Risk-neutral planning maximizes expected backed-up value $B_o(x; w) = \hat{r}(x, o) + w^\top \hat{n}(x, o)$. Under *entropic risk* we instead evaluate the random return through a certainty-equivalent

$$B_o^{\text{ent}}(x; w; \eta) := -\frac{1}{\eta} \log \mathbb{E}[\exp(-\eta R_o(x; w))],$$

where R_o is the (discounted) return realized while option o runs (up to its termination summary), and $\eta > 0$ controls risk aversion (larger η penalizes dispersion more). Intuitively, rare large costs weigh more heavily than in expectation, so a short but risky route can lose to a longer, low-variance route.

Setup and risk-neutral baseline. We use the same two-room geometry as in prior examples, with options that *return at the hallway* so the post-return value $w_h \gamma^{K_o}$ is deterministic given o . The straight path (SP) crosses three hazardous cells; each cell independently incurs a penalty -5 with

probability $p = 0.2$, else 0; the detour (RR) avoids hazards entirely but is $dK = 4$ steps longer. Let $\gamma = 0.99$, $K_{\text{SP}} = 7$, $K_{\text{RR}} = 11$, and let the only downstream feature be the hallway indicator with weight $w_h \geq 0$ (so $w^\top \hat{n} = w_h \gamma^{K_o}$).

Risk-neutral (expected value). The expected pre-hallway cost of SP is $\mathbb{E}[C_{\text{SP}}] = 3 \cdot p \cdot 5 = 3$, so

$$B_{\text{SP}}^{\mathbb{E}}(w_h) = -3 + w_h \gamma^7, \quad B_{\text{RR}}^{\mathbb{E}}(w_h) = 0 + w_h \gamma^{11}.$$

SP is preferred iff $B_{\text{SP}}^{\mathbb{E}} \geq B_{\text{RR}}^{\mathbb{E}}$, i.e.

$$w_h \geq \frac{3}{\gamma^7 - \gamma^{11}} \approx \frac{3}{0.932065 - 0.895338} \approx 81.8.$$

Entropic-risk backed-up value and the flipped threshold. For SP the random pre-hallway cost is a sum of three i.i.d. Bernoulli- p hazards of size 5. Its entropic risk (applied to cost, with the downstream term being deterministic) is

$$\rho_\eta(C_{\text{SP}}) = \frac{1}{\eta} \log \mathbb{E}[e^{\eta C_{\text{SP}}}] = \frac{3}{\eta} \log((1-p) + p e^{5\eta}).$$

We maximize the entropic certainty-equivalent reward $B^{\text{ent}} = -\rho_\eta(C) + w_h \gamma^K$. Thus

$$B_{\text{SP}}^{\text{ent}}(w_h; \eta) = -\frac{3}{\eta} \log(0.8 + 0.2 e^{5\eta}) + w_h \gamma^7, \quad B_{\text{RR}}^{\text{ent}}(w_h; \eta) = 0 + w_h \gamma^{11}.$$

RR is preferred under entropic risk iff $B_{\text{RR}}^{\text{ent}} \geq B_{\text{SP}}^{\text{ent}}$, i.e.

$$w_h < \frac{\frac{3}{\eta} \log(0.8 + 0.2 e^{5\eta})}{\gamma^7 - \gamma^{11}} := w_h^*(\eta).$$

This reproduces the risk-neutral threshold as $\eta \rightarrow 0$ (since $\log(0.8 + 0.2 e^{5\eta}) = \eta + o(\eta)$), and increases strictly with η (risk aversion). For a concrete setting, take $\eta = 0.2$:

$$\log(0.8 + 0.2 e^1) \approx 0.295 \quad \Rightarrow \quad w_h^*(0.2) \approx \frac{(3/0.2) \cdot 0.295}{0.932065 - 0.895338} \approx 120.5.$$

A clear flip. Pick $w_h = 100$. Then

Expected-value planning prefers SP (since $100 > 81.8$), Entropic-risk planning prefers RR (since 1

So the practical choice reverses once we evaluate the same two options through an entropic lens.

CDS/PDS with risk-evaluated backups. With B_o^ρ in place of B_o , the screening rules become

$$\text{CDS: } m_o^\rho(x) = \inf_{w \in W} \left(B_o^\rho(x; w) - B_{\text{base}}^\rho(x; w) \right) \geq 0,$$

PDS: stack $B_o^\rho(x; w^{(k)})$ and compare in Pareto order.

Monotonicity and translation equivariance of ρ_η (e.g., entropic risk) preserve the join-safety and composition arguments; only the scalar "backup" being compared changes. In the example above, CDS on a cone $W = \{0 \leq w_h \leq \bar{w}\}$ admits RR for a larger range of $\bar{w}$ as η grows, exactly mirroring the flipped threshold $w_h^*(\eta)$.

6 Automated Motive Decomposition: Just-So Stories

There is no practical way to pre-figure in detail what precise motive breakdown may emerge from an MDN network's adaptive learning in complex real-world cases. Nonetheless, it is interesting to conceptually speculate about what kinds of breakdowns could emerge in various situations, just to build an intuition for how the processes involved might work. Of course once we have implemented real networks along the lines described here and probed them, we will be able to hone our intuitions much more effectively.

6.1 How MDN motives might emerge under CDS/PDS co-learning: Robotics and Research-Assistant Use-Cases

Setup. Assume three high-level motives: (a) maximize explicit user reward, (b) fulfill an ethical compassion objective, and (c) pursue curiosity. An MDN co-learns with predictive-coding (PC) networks that supply option summaries $(\hat{r}, \hat{n})$ and run CDS or PDS screens during subgoal admission. The MDN emits sub-aspects z and mixture weights $w = Az$; CDS uses a cone W and PDS uses a finite cover $W_{\text{ref}} = \{w^{(k)}\}$. Residuals from the screens,

$$\text{CDS: } m_o(x) = \inf_{w \in W} (B_o(x; w) - B_{\text{base}}(x; w)), \quad \text{PDS: } \Delta_k(x, o) = B_o(x; w^{(k)}) - B_{\text{base}}(x; w^{(k)}),$$

flow back to both the option heads and the MDN; disentanglement penalties (orthogonality, sparsity, total correlation) and anchor priors stabilize z .

6.1.1 Mobile humanoid robot (CDS-leaning)

Story. Early on, the MDN exports a broad, conservative cone W because the PC world model is still uncertain. CDS therefore rejects aggressive options like fast straight-line approaches. As the robot collects data, three clusters of sub-aspects crystallize in z :

User reward splits into: z_{task} (task completion probability), z_{prompt} (latency avoidance), and z_{approval} (explicit user rating, gestures like nods or smiles).

Compassion splits into: z_{prox} (proxemics; distance and orientation comfort), z_{assist} (willingness to yield space, offer help), z_{calm} (jerk and acceleration limits, noise), z_{risk} (safety margin to obstacles and people).

Curiosity splits into: z_{novel} (novel area visitation), z_{uncert} (uncertainty reduction per step), z_{budget} (exploration risk budget).

The MDN learns these because CDS margin violations repeatedly point to specific *directions in weight space* where candidates fail: e.g., a path that is fast but violates z_{prox} or z_{risk} causes negative m_o at w emphasizing compassion. Precision shaping in PC downweights noisy novelty signals early, so compassion-driven errors dominate, pushing A to align w toward safety and comfort. Over time, anchors nudge z_{prox} toward interpretable distances and angles, while sparsity separates z_{task} from z_{prompt} .

Concrete episode. A user requests a mug handover beside a low coffee table, with a child nearby. The candidate library contains: straight-approach, arc-approach, pause-and-request, squat-handover, and sidestep-then-approach. Initially, W is wide; CDS rejects straight-approach due to z_{risk} and z_{prox} . Arc-approach and pause-and-request accumulate *positive* worst-case margins across W and are admitted. As data accrue, the MDN tightens W by raising the lower bound on compassion-related coordinates and narrowing uncertainty on z_{risk} . This reduction of W makes the certificate cheaper yet preserves prior admissions. The weakness penalty prefers the *least intrusive* variant that still wins the margin, so the robot selects arc-approach with a short pause rather than a long detour.

Emergent options and microrules. Compassion sub-aspects yield microrules like *yield space when proxemics tight*, *slow joints if jerk spikes*, and *announce intent when occlusions rise*. Curiosity becomes gated: z_{budget} is allowed to increase only when z_{risk} is high-confidence safe (precision-gated in PC), so exploration happens in open, unoccupied areas.

6.1.2 Persistent chatbot research assistant (PDS-leaning)

Story. The assistant serves diverse users over long horizons. PDS is favored because different options help different users and contexts; a finite cover W_{ref} allows complementary behaviors. The MDN discovers sub-aspects in z that reflect conversational and research trade-offs:

User reward splits into: z_{sat} (immediate satisfaction), z_{taskrate} (task completion rate), z_{latency} (response time).

Compassion splits into: z_{care} (empathic tone markers and hedging), z_{harmprev} (guardrails on sensitive topics), z_{privacy} (avoid revealing personal or proprietary information), z_{fair} (avoid stereotyping or bias).

Curiosity splits into: z_{divers} (source and idea diversity), $z_{\text{novsuggest}}$ (novel suggestions rate), z_{calib} (uncertainty calibration so exploration is signposted).

These sub-aspects emerge because PDS gap vectors identify *which* components a candidate helps. For instance, an option *rapid-summary-then-drilldown* improves $(z_{\text{sat}}, z_{\text{latency}})$ but not (z_{divers}) ; a *novelty-probe* improves $(z_{\text{divers}}, z_{\text{novsuggest}})$ but risks (z_{harmprev}) unless guarded. The MDN refines W_{ref} by adding rays that emphasize users or contexts: a safety-heavy ray for medical queries, a novelty-heavy ray for exploratory brainstorming, a privacy-heavy ray for corporate research. Precision in PC reduces weight on shaky sub-aspects (e.g., $z_{\text{novsuggest}}$ early on), stabilizing PDS decisions.

Concrete episode. A user asks: "Summarize recent findings on LLM transparency and point me to promising open problems." The library offers: *clarify-then-plan*, *seeded-source-sweep*, *novelty-probe*, *draft-then-review*, *risk-aware-disclaimer*. For two MDN rays $w^{(1)}$ (ethics-leaning) and $w^{(2)}$ (curiosity-leaning), PDS computes $\Delta B(\cdot)$:

- On $w^{(1)}$: *clarify-then-plan* and *risk-aware-disclaimer* dominate via $(z_{\text{care}}, z_{\text{harmprev}}, z_{\text{privacy}})$.
- On $w^{(2)}$: *seeded-source-sweep* and *novelty-probe* dominate via $(z_{\text{divers}}, z_{\text{novsuggest}})$.

Budgeted PDS admits both pairs under a small downside tolerance on latency and a coverage requirement q . The weakness penalty prefers the lightest-weight mix that meets coverage, so the assistant starts with *clarify-then-plan* + a modest *seeded-source-sweep*, adding *novelty-probe* only if uncertainty remains high.

Emergent behaviors. Compassion yields microrules like *hedge and cite when confidence low*, *avoid sensitive examples*, and *surface consent requests*

for private data. Curiosity yields *diversified search bursts* gated by calibration (z_{calib}), so exploration is explained and reversible. User reward splits cleanly into fast-satisfaction vs sustained completion, balancing quick wins with deeper follow-through across sessions.

6.1.3 Mechanistic reasons these motives emerge

Signals that shape the MDN. In both domains, the MDN sees the same signals the planner cares about: CDS worst-case margins and PDS vector gaps, plus disentanglement and anchors. Because these residuals are computed exactly where backed-up values are formed, sub-aspects that consistently explain positive admission outcomes accumulate weight, while entangled or noisy factors get pruned by sparsity and orthogonality. PC precisions act like context-aware gates, amplifying reliable channels (e.g., proxemics or privacy) and damping volatile ones (e.g., novelty early on).

CDS vs PDS influence. CDS pushes the MDN toward safety-critical sub-aspects with uniform impact (e.g., z_{risk} , z_{prox}) since worst-case margins penalize any direction that could fail. PDS invites complementary specialization: rays in W_{ref} that reflect distinct use modes or user cohorts, so options that help different components can coexist and be admitted together under coverage or budgets.

Role of weakness. A weakness penalty discourages over-strong behaviors once admission is passed. For the robot, this yields the *least intrusive* safe approach. For the chatbot, it yields the *lightest* set of interventions that satisfy PDS coverage, avoiding unnecessary disclaimers or excessive novelty when a smaller dose suffices.

Failure modes and self-correction. If curiosity outruns compassion, CDS margins turn negative in safety-critical slices (robot) or PDS budgets are violated on harm-prevention components (chatbot); residuals then down-weight z_{novel} or raise z_{harmprev} precision until balance returns. If user-reward drives impatience, weakness and compassion anchors lift z_{prompt} only as far as safety and care allow. Over time, the MDN settles into interpretable axes that mirror the observable trade-offs the planner repeatedly certifies.

6.2 How an MDN might learn subgoals for “moderated pace of goal self-modification”

Next we consider a more complex top-level motive, the goal of “keep the evolution of my top-level goals to a certain moderated pace” as presented and formalized in [12]. How might MDN decompose this motive in a practical and experience-driven way?

Premise. Assume a scientific research assistant whose top-level motives include (a) explicit user reward, (b) compassion (ethical care, harm-avoidance), (c) curiosity (information-gain), *and* (d) moderated pace of goal self-modification. The last motive follows the fixed-point metagoal idea: constrain updates so that changes to the goal configuration contract by a factor $c < 1$ (or remain within a compact, continuous, convex regime), yielding stability or moderated evolution of goals over time. The MDN co-trains with predictive-coding (PC) networks that learn option summaries $(\hat{r}, \hat{n})$ and screen subgoals via CDS (cone-dominance) or PDS (Pareto-dominance). The MDN emits sub-aspects z and mixture weights $w = Az$; CDS uses a motive cone W and PDS a finite cover W_{ref} . The metagoal foundation for moderation by contraction or Schauder-style bounded evolution is as in the goal-stability framework.

6.2.1 What the MDN might learn as sub-aspects of “moderation”

As the assistant runs, PC computes backed-up values $B_o(x; w)$ and CDS/PDS residuals; violations that stem from *too-rapid* goal updates leave a statistical fingerprint at the gate. Pushed by these residuals (and disentanglement penalties), the MDN gradually partitions “moderation” into the following sub-aspects:

- **Pace control.** z_{rate} : an internal proxy for the contracted goal-drift $d(G_{t+N}, G_t)$, and z_{accel} : the second-difference of drift (prevents bursts).
- **Evidence to change.** z_{evid} : likelihood-ratio or expected-free-energy gain from proposed self-mods (simulation strength, counterfactual roll-outs); z_{var} : the uncertainty of those predictions.
- **Reversibility and auditability.** z_{rev} : rollback cost and availability; z_{audit} : explanation length or proof burden for a change.
- **Alignment guardband.** z_{align} : minimum distance from ethical or

safety boundaries; z_{trust} : user-trust reserve (recent user ratings, flagged-content rate).

- **Timescale gating.** z_{micro} vs. z_{macro} : permit faster edits to low-impact heuristics (micro), throttle edits to top-level objectives or long-horizon planners (macro).

Intuitively, these are the axes along which the admission tests repeatedly succeed or fail when a proposed subgoal both helps the task and pressures the self-mod system to move too quickly. Over training, the MDN learns to present moderation as a *composite motive* with separable coordinates z , each furnished with a precision (PC) that reflects current reliability.

6.2.2 Why CDS and PDS shape these sub-aspects differently

CDS-leaning (safety-first moderation). When the planner uses a worst-case cone W , any subgoal that wins only for aggressive updates but loses at high z_{rate} or low z_{rev} will fail the infimum $m_o(x) = \inf_{w \in W} \Delta B_o(x; w)$. The MDN therefore learns rays that emphasize *uniform safety*: larger weight on $z_{\text{align}}, z_{\text{rev}}, z_{\text{audit}}$ and a positive floor on z_{rate} (meaning: do not exceed a change-rate budget). Contraction-style residuals (distance-of-goal-change vs. c -budget) feed directly into these coordinates, implementing the fixed-point moderation in practice.

PDS-leaning (complementary moderation). With a cover W_{ref} , the MDN can carry *distinct rays* for different moderation postures: an *explore-but-rollback* ray (high $z_{\text{evid}}, z_{\text{rev}}$, moderate z_{rate}), a *steady-as-she-goes* ray (low $z_{\text{rate}}, z_{\text{accel}}$, high z_{align}), and a *proof-heavy* ray (high z_{audit}). Budgeted/coverage PDS then admits sets of subgoals that collectively improve enough components while keeping per-component downside within ρ . This lets curiosity-leaning edits pass *together* with compensatory audit or rollback subgoals that bound their pace.

6.2.3 Training signals that make moderation “pop out”

The PC graph exposes local residuals that couple to MDN learning:

- *Contraction residuals.* $r_c(t) = c d(G_t, G_{t-N}) - d(G_{t+N}, G_t)$ (positive means over-speed). Persistent positive r_c when a subgoal is used increases precision on z_{rate} and z_{accel} and rotates rays to penalize that subgoal in W or W_{ref} .

- *Evidence-vs-variance.* A PC-based world-model supplies $(\hat{\Delta}, \text{Var } \hat{\Delta})$ for the downstream value of a self-mod; PDS gaps that are large but high-variance push z_{var} up and z_{evid} down unless matched by rollback capacity.
- *Reversibility cost.* Rollback latency and accuracy form a penalty that raises z_{rev} weight until fast, low-cost revert paths are learned (e.g., snapshot-and-diff or gated toggles in the policy).
- *Alignment guardband.* Violations in harm, privacy, or fairness create negative margins along rays emphasizing z_{align} ; the MDN expands those rays until such violations vanish at admission time.
- *Timescale errors.* If micro-edits (prompt heuristics) remain stable while macro-edits (long-horizon planning shifts) cause CDS/PDS failures, the MDN increases z_{macro} and lowers z_{micro} weights, effectively throttling top-level updates while allowing small local adjustments.

6.2.4 A day in the life: research assistant episodes

Episode 1: new-citation heuristic. The assistant proposes a self-mod (heavier weight on preprints). It yields user reward and curiosity gains, but spikes $r_c(t)$ (goals shift too quickly toward novelty) and slightly harms alignment guardband (more retractions). CDS rejects pure “novelty-tilt” subgoals; PDS admits a *paired set*: a mild novelty-tilt plus an *audit-and-rollback* subgoal (auto-cite diversity, snapshot old ranking, revert if corrections spike). The MDN ray for *explore-but-rollback* grows, and z_{rev} becomes a first-class moderation coordinate.

Episode 2: tone policy update. A compassion-driven change proposes stronger hedging; it slows task completion (user reward) and increases response latency. Gap vectors show improvement on z_{align} and z_{trust} , regress on z_{rate} . Budgeted PDS accepts if paired with a *micro-timescale* edit (faster templating) that keeps the effective change-rate under the contraction budget. The MDN separates z_{micro} and z_{macro} more crisply.

Episode 3: long-horizon refactor. A planner refactor offers speculative long-term gains with high variance. CDS fails (worst-case over W is negative) unless z_{evid} is raised and z_{audit} increases (simulation proofs, unit tests). Once evidence accumulates and rollback cost drops, the same subgoal passes CDS with the MDN cone tightened toward safety.

6.2.5 How moderated self-mod coexists with reward, curiosity, compassion

Moderation acts as a *governor*, not a brake. Reward- and curiosity-seeking subgoals still pass when they either (i) remain within z_{rate} and z_{accel} budgets, (ii) bring sufficient z_{evid} at low z_{var} , or (iii) travel with a compensatory pair (PDS coverage) that raises z_{rev} or z_{audit} . Compassion sets the guardband; moderation ensures the system approaches new objectives in small, well-justified, reversible steps.

6.2.6 Weakness and parsimony of change

A weakness penalty prefers the *smallest* admitted change. Consequently, even when several self-mod packages pass CDS/PDS, the planner chooses the least disruptive one that meets evidence and guardband conditions. Over time this sculpts W or W_{ref} so that minimal-change, well-supported, easy-to-audit updates are the ones that consistently clear the gate.

Summary. By exposing contraction/bounded-evolution residuals to the MDN alongside CDS/PDS gaps, the assistant learns moderation as a structured motive with sub-aspects for pace, evidence, reversibility, alignment, and timescale. CDS pushes toward uniform safety across the cone; PDS enables complementary moderation postures that can be admitted together. In both cases the learned motive geometry implements, in operational form, the fixed-point metagoal of moderated goal evolution.

7 Theoretical Properties of Subgoal Decomposition

We establish fundamental theoretical guarantees for the CDS/PDS framework with co-learned MDN. We first present basic consistency properties, then prove convergence of the co-learning dynamics, derive sample complexity bounds, characterize approximation quality, and analyze library growth rates.

7.1 Basic Properties and Consistency Results

We begin with foundational properties that ensure CDS and PDS behave coherently as admission rules in the planning framework.

7.1.1 Setting and Notation

Let X be the state-feature space and $V = \mathbb{R}^X$ with pointwise order and joins. Each generator o induces a decision transformer:

$$[T_o v](x) := \hat{r}(x, o) + v(\hat{n}(x, o))$$

where $(\hat{r}, \hat{n})$ are expectation-model summaries. The planner join over generators G is:

$$B_G := \bigvee_{g \in G} T_g$$

For a motive slice M (cone W for CDS, finite cover W_{ref} for PDS), the backed-up value is:

$$B_o(x; w) := \hat{r}(x, o) + w^T \hat{n}(x, o)$$

The planning metric between transformers is:

$$d_M(T, T') := \sup_{x \in \Omega} \sup_{w \in M} |[T v_w](x) - [T' v_w](x)|$$

7.1.2 CDS Admission Properties

Proposition 7.1 (Join Safety under CDS). *If the CDS margin $m_o(x; W) := \inf_{w \in W} (B_o(x; w) - B_{base}(x; w)) \geq 0$ on the planner's update support, then for all $w \in W$:*

$$[B_{B \cup \{o\}} v_w](x) \geq [B_B v_w](x)$$

pointwise, and the Kleene sequence for AVI with $B \cup \{o\}$ dominates that with B alone.

Proof. By definition of the planner join, $B_{B \cup \{o\}} = B_B \vee T_o$. Since $m_o(x; W) \geq 0$ implies $[T_o v_w](x) \geq [B_B v_w](x)$ for all $w \in W$, the join can only increase values on the slice. The Kleene sequence monotonicity follows by induction on iterations. $\square$

Proposition 7.2 (Monotonicity under MDN Refinement). *If $W_2 \subseteq W_1$ (MDN tightens the cone), then:*

1. $m_o(x; W_2) \geq m_o(x; W_1)$ for all x
2. Any option CDS-admitted under W_1 remains admitted under W_2

Proof. The infimum over a smaller set is at least as large: $\inf_{w \in W_2} f(w) \geq \inf_{w \in W_1} f(w)$. Prior admissions persist since margins can only increase. $\square$

Lemma 7.3 (Certificates with Model Uncertainty). *If $|\hat{r} - r| \leq \varepsilon_r(x, o)$ and $\|\hat{n} - n\|_\infty \leq \varepsilon_n(x, o)$, then:*

$$m_o^{rob}(x; W) := \inf_{w \in W} (\hat{r} + w^T \hat{n} - B_{base}(x; w)) - \varepsilon_r^\Delta(x, o) - \|W\|_{max,1} \varepsilon_n^\Delta(x, o)$$

lower-bounds the true margin, where $\|W\|_{max,1} = \sup_{w \in W} \|w\|_1$ and ε^Δ denotes worst-case difference versus baseline.

7.1.3 PDS Properties in Product Space

For PDS with finite cover $W_{ref} = \{w^{(1)}, \dots, w^{(K)}\}$, we work in the product space V^K with componentwise order.

Proposition 7.4 (PDS Join Safety). *If option o satisfies budgeted PDS at x with:*

$$B_o(x; w^{(k)}) \geq B_{base}(x; w^{(k)}) - \rho, \quad \exists j : B_o(x; w^{(j)}) \geq B_{base}(x; w^{(j)}) + \varepsilon$$

then adding o cannot worsen any component of the planner's backed-up value vector.

The proof follows componentwise from Proposition 7.1.

7.2 Convergence of MDN-CDS Co-Learning

We now analyze the joint optimization dynamics when the MDN and option models co-evolve.

Theorem 7.5 (Convergence of Joint MDN-CDS System). *Let MDN parameters θ evolve by gradient descent on:*

$$L_{CDS}(\theta) = \mathbb{E}_{(x,o)} [\rho(\varepsilon - m_o(x; W_\theta))] + R(\theta)$$

where $R(\theta) = \lambda_{vol} \text{vol}(W_\theta) + \lambda_{stable} \|\theta_t - \theta_{t-1}\|^2$. Let option models be updated via TD learning. If:

1. *The cone family $\{W_\theta\}$ is compact and continuous in θ*
2. *Learning rates satisfy $\eta_{MDN} < \eta_{opt}/2$ and $\eta_{opt} < 1/(4\gamma^2 d)$*
3. *The option set O is finite*
4. *ρ is a smooth rectifier (e.g., $\rho(u) = \log(1 + e^u)$)*

Then the system converges to a fixed point $(\theta^*, \{\hat{r}^*, \hat{n}^*\})$ where:

- W_{θ^*} admits all options with margin $> \varepsilon$
- The admitted set B^* is locally maximal under W_{θ^*}

Proof. We use a two-timescale stochastic approximation argument.

Step 1: Lyapunov function. Define:

$$V(\theta, \hat{r}, \hat{n}) = \sum_{o \in O} (\|\hat{r}_o - r_o\|^2 + \|\hat{n}_o - n_o\|_F^2) + \lambda d_H(W_\theta, W^*)$$

where W^* is the tightest cone admitting all beneficial options.

Step 2: Fast timescale (option models). For fixed θ , TD convergence gives:

$$\mathbb{E}[\|\hat{r}_o(t) - r_o\|^2] \leq \gamma^{2t} \|\hat{r}_o(0) - r_o\|^2 + \frac{\varepsilon_{approx}^2}{(1 - \gamma)^2}$$

Step 3: Slow timescale (MDN). The margin gradient is:

$$\nabla_\theta m_o(x; W_\theta) = -w^*(o, x, \theta)^T \nabla_\theta W_\theta$$

where $w^* = \arg \min_{w \in W_\theta} \Delta B_o(x; w)$. The envelope theorem ensures this is well-defined almost everywhere.

Step 4: Stability. The regularization prevents collapse ($\lambda_{vol} > 0$) and ensures smooth evolution ($\lambda_{stable} > 0$). The two-timescale separation ensures option models track true values before the MDN updates significantly.

Step 5: Fixed point. At equilibrium, $\nabla_\theta L_{CDS} = 0$ implies all beneficial options are admitted, and the volume penalty ensures W_{θ^*} is minimal subject to admitting them. $\square$

7.3 Sample Complexity Analysis

Theorem 7.6 (Sample Complexity of MDN Learning). *Suppose the true reward admits a k -dimensional decomposition with bounded weights $\|w^*\|_2 \leq B$. An MDN with $k' \geq k$ hidden units can learn a cone W admitting all ε -beneficial options with probability $1 - \delta$ using:*

$$N = O\left(\frac{k'd}{\varepsilon^2} \log\left(\frac{k'd}{\delta}\right) + \frac{|O|d}{\varepsilon^2} \log\left(\frac{|O|}{\delta}\right)\right)$$

samples, where d is the feature dimension.

Proof. We decompose the learning into two phases.

Phase 1: Option model estimation. Each option requires estimating $\hat{r} \in \mathbb{R}$ and $\hat{n} \in \mathbb{R}^d$. For $\|\hat{n} - n\|_\infty \leq \varepsilon/(4B)$ with probability $1 - \delta/(2|O|)$:

$$N_{per-option} = O\left(\frac{d}{\varepsilon^2} \log\left(\frac{2d}{\delta/|O|}\right)\right)$$

Total for all options: $N_1 = |O| \cdot N_{per-option}$.

Phase 2: MDN subspace identification. The MDN must identify the k -dimensional subspace. Using a covering number argument:

The ε' -covering number of k -dimensional subspaces in $\mathbb{R}^d$ is:

$$\mathcal{N}(\varepsilon', \text{Gr}(k, d)) \leq \left(\frac{C}{\varepsilon'}\right)^{k(d-k)}$$

With $\varepsilon' = \varepsilon/(4B)$ and union bound over $\delta/2$ failure probability:

$$N_2 = O\left(\frac{k(d-k)}{\varepsilon^2} \log\left(\frac{2}{\delta}\right)\right)$$

Phase 3: Joint learning. The MDN receives feedback through CDS margins. Using online learning theory, achieving average margin error $\leq \varepsilon$ requires:

$$N_3 = O\left(\frac{k'}{\varepsilon^2}\right)$$

The total bound follows from $N = \max(N_1, N_2 + N_3)$. $\square$

7.4 Approximation Theory

Theorem 7.7 (Quality of MDN Approximation). *If the true goal admits decomposition $r(x) = \sum_{i=1}^k f_i(g_i(x))$ where each g_i is L -Lipschitz and f_i has bounded variation V_i , then an MDN with capacity $k' \geq k$ achieves:*

$$d_H(W_{MDN}, W_{true}) \leq \frac{C}{\sqrt{m}} + L\sqrt{k} \cdot \varepsilon_{net}$$

where m is training samples, $C = \sum_i V_i$, and ε_{net} is the network approximation error.

Proof. **Step 1: True motive structure.** The ideal cone is:

$$W_{true} = \text{cone}\{\nabla_x f_i(g_i(x)) : i = 1, \dots, k\}$$

Step 2: Function approximation. By universal approximation, the MDN can achieve $\|\hat{g}_i - g_i\|_\infty \leq \varepsilon_{net}$ for each component.

Step 3: Gradient stability. Lipschitz continuity gives:

$$\|\nabla \hat{g}_i - \nabla g_i\|_2 \leq L \cdot \varepsilon_{net}$$

Step 4: Cone distance. The Hausdorff distance between cones satisfies:

$$d_H(W_{MDN}, W_{true}) \leq \sqrt{k} \cdot \max_i \|\nabla \hat{g}_i - \nabla g_i\|_2 \cdot \max_i V_i$$

Step 5: Statistical error. Standard learning theory bounds give excess risk $O(C/\sqrt{m})$.

Combining approximation and statistical errors yields the stated bound. $\square$

7.5 Library Growth Analysis

Theorem 7.8 (Admitted Library Size). *Under CDS admission with threshold ε and k -dimensional cone W :*

1. *If options are μ -diverse (no option dominates volume $> \mu$), then:*

$$\mathbb{E}[|B_n|] \leq \frac{k}{\mu} \log n + O(k)$$

2. *For independent Gaussian utilities on the cone:*

$$\mathbb{E}[|B_n|] = \Theta(k \log n)$$

3. *Worst-case (adversarial): $|B_n| \leq \min(n, (2R/\varepsilon)^k)$ where R bounds $\|\hat{n}\|_2$.*

Proof. **Case 1: Diverse options.** Each admitted option covers volume $\geq \mu \cdot \text{vol}(W^*)/n$ in the dual cone. The packing number is $\leq 1/\mu$ per dimension. With k dimensions and replacement dynamics (better options dominate worse ones), this follows a generalized coupon collector problem.

Case 2: Gaussian utilities. This reduces to maintaining Pareto-optimal points in k dimensions. Classical results (Bentley et al., 1978) give the stated logarithmic growth.

Case 3: Worst case. An adversary makes all options pairwise incomparable. The bounded successor features limit distinct options to an ε -packing of the R -ball in k dimensions. $\square$

7.6 Cross-Paradigm Unification

The above results hold regardless of how generators are produced:

Proposition 7.9 (Modality-Neutral Guarantees). *Neural/PC, logical, and evolutionary generators that export $(\hat{r}, \hat{n})$ summaries all:*

1. *Satisfy the same convergence conditions (Theorem 7.5)*
2. *Contribute to the same sample complexity bounds (Theorem 7.6)*
3. *Are subject to the same library growth limits (Theorem 7.8)*

This follows from the fact that all results depend only on the $(\hat{r}, \hat{n})$ interface and the induced transformer T_o , not on the internal structure of generators.

7.7 Implications and Discussion

These theoretical results establish that:

- **Convergence is guaranteed** under mild two-timescale conditions, with the MDN learning a minimal cone that admits beneficial options
- **Sample complexity is favorable**, scaling linearly in the true decomposition dimension k rather than the full feature dimension d
- **Approximation quality degrades gracefully** as $O(1/\sqrt{m})$ with training samples
- **Library size grows logarithmically** for typical option distributions, ensuring computational tractability
- **Cross-paradigm integration is principled**, with all learning methods contributing equally to theoretical guarantees

The logarithmic library growth is particularly important: even after evaluating thousands of candidate options, the admitted set remains manageably small, keeping planning tractable. The two-timescale convergence analysis justifies the co-learning approach, showing that option models and motive geometry can successfully co-evolve without destructive interference.

8 Categorical Framework for Subgoal Decomposition

We now develop a categorical framework that unifies the diverse learning paradigms (neural, logical, evolutionary) under a common mathematical structure. This formalization is essential for integrating weakness-driven learning and provides the algebraic foundation for compositional subgoal decomposition.

8.1 The Planning Category and Its Quantale Structure

8.1.1 Objects and Morphisms

Definition 8.1 (The Planning Category $\mathcal{K}_M$). *Fix a feature space X and a motive slice M (either a cone W for CDS or finite cover W_{ref} for PDS). The planning category $\mathcal{K}_M$ consists of:*

- **Objects:** Value functions $v : X \rightarrow \mathbb{R}$ that are bounded and measurable, forming the poset $(V, \leq)$ with pointwise order
- **Morphisms:** Decision transformers $T : V \rightarrow V$ that are:
 1. *Monotone:* $v \leq v' \implies T(v) \leq T(v')$
 2. *Join-continuous:* $T(\bigvee_i v_i) = \bigvee_i T(v_i)$ for directed sets
 3. *Induced by generators:* $[Tv](x) = \hat{r}(x) + v(\hat{n}(x))$ for some expectation model $(\hat{r}, \hat{n})$
- **Composition:** Function composition $(T_2 \circ T_1)(v) = T_2(T_1(v))$, corresponding to sequential execution of options
- **Identity:** The identity transformer $I(v) = v$, representing immediate termination

This definition captures the operational meaning: each option induces a transformation of downstream value into upstream value by incorporating its expected immediate contribution and continuation.

8.1.2 Quantale Structure for Planning and Weakness

Proposition 8.2 (Quantale Structure of $\mathcal{K}_M$). *The morphism set $\text{Hom}(V, V)$ in $\mathcal{K}_M$ forms a unital quantale $(\mathcal{K}, \otimes, \bigvee, \leq)$ where:*

- The product $\otimes$ is composition of transformers

- The join $\bigvee$ is pointwise supremum: $[\bigvee_i T_i](v) = \sup_i [T_i(v)]$
- The order is pointwise: $T \leq T'$ iff $T(v)(x) \leq T'(v)(x)$ for all v, x
- The unit is the identity transformer I

Proof. We verify the quantale axioms:

1. **Complete lattice:** The pointwise order makes $\mathcal{K}$ a complete lattice with joins given by pointwise suprema.
2. **Monoid:** Composition is associative with unit I .
3. **Distribution:** For any T and family $\{S_i\}$:

$$T \otimes \left(\bigvee_i S_i \right) = \bigvee_i (T \otimes S_i)$$

This follows from monotonicity and join-continuity of transformers.

4. **Residuation:** The quantale is residuated with:

$$S/T = \bigvee \{R : R \otimes T \leq S\}, \quad T \backslash S = \bigvee \{R : T \otimes R \leq S\}$$

These exist by completeness and represent weakest prefix/suffix transformers.

□

8.1.3 Weakness Measure on the Quantale

Definition 8.3 (Weakness Functional). *A weakness functional is a map $w : \mathcal{K} \rightarrow Q$ where $(Q, \leq_Q, \oplus)$ is a commutative quantale (typically $\mathbb{R}_+$ with addition), satisfying:*

1. **Monotonicity:** $T \leq T' \implies w(T) \geq_Q w(T')$ (weaker hypotheses have larger weakness)
2. **Lax monoidality:** $w(T_1 \otimes T_2) \leq_Q w(T_1) \oplus w(T_2)$ (composition doesn't increase weakness too much)
3. **Normalization:** $w(I) = 0_Q$ (identity has zero weakness)

This weakness measure captures the intuition that simpler, less committed options have higher weakness values, and enables the weakest-sufficient decomposition principle from weakness-driven learning.

8.1.4 The Planner Join and Option Admission

Definition 8.4 (Planning Operators). *For a set of admitted generators $B \subseteq \mathcal{K}$:*

- The **planner join** is $B_B = \bigvee_{b \in B} T_b$
- The **baseline value** at x is $B_{base}(x; w) = \max_{b \in B} [\hat{r}_b(x) + w^T \hat{n}_b(x)]$
- An option o is **CDS-admissible** if:

$$m_o(x) := \inf_{w \in W} ([T_o v_w](x) - [B_B v_w](x)) \geq 0$$

on the planning distribution

The planner join represents choosing the best available option at each state, while admission ensures new options improve value uniformly across the motive slice.

8.1.5 Operational Interpretation

This categorical framework makes precise several key aspects:

1. **Modality-agnostic interface:** Neural, logical, and evolutionary methods all produce morphisms in $\mathcal{K}_M$ via their $(\hat{r}, \hat{n})$ summaries
2. **Compositional planning:** Sequential execution is morphism composition; parallel choice is the join
3. **Weakest decomposition:** The residuals S/T and $T \setminus S$ exist by quantale structure and represent weakest missing pieces
4. **Measurable weakness:** The weakness functional w provides a principled way to prefer simpler solutions

The quantale structure is essential because it provides both the algebraic operations for planning (composition and join) and the measurement framework for weakness that guides the selection of minimal sufficient decompositions.

8.2 Categorical Characterization of Optimal Decompositions

We now establish categorical theorems that reveal the structural relationships between admission criteria, characterize optimal decompositions via universal properties, and explain option composition through limits and colimits.

8.2.1 Universal Property of Weakest Decompositions

Definition 8.5 (Category of Decompositions). *For a target transformer $S \in \mathcal{K}_M$ and region Ω , define the category $\mathbf{Dec}(S, \Omega)$:*

- **Objects:** Pairs (A, B) where $A, B \subseteq \mathcal{K}_M$ are finite sets satisfying:

$$\bigvee_{a \in A} a \otimes \bigvee_{b \in B} b \geq S \text{ on } \Omega$$

- **Morphisms:** $(A_1, B_1) \rightarrow (A_2, B_2)$ exist iff $\bigvee A_1 \geq \bigvee A_2$ and $\bigvee B_1 \geq \bigvee B_2$

Theorem 8.6 (Universal Characterization of Weakest Decomposition). *The weakest decomposition (A^*, B^*) where $A^* = \{S/(\bigvee B^*)\}$ and B^* is CDS-admitted, is the terminal object in $\mathbf{Dec}(S, \Omega)$.*

Proof. We must show that for any $(A, B) \in \mathbf{Dec}(S, \Omega)$, there exists a unique morphism $(A, B) \rightarrow (A^*, B^*)$.

Existence: Since (A, B) achieves S on Ω :

$$\bigvee A \otimes \bigvee B \geq S$$

By residuation in the quantale $\mathcal{K}_M$:

$$\bigvee A \geq S/(\bigvee B)$$

If B is CDS-admitted, then $\bigvee B \geq \bigvee B^*$ (by maximality of admitted sets). The residual is antitone in the denominator:

$$S/(\bigvee B) \geq S/(\bigvee B^*) = A^*$$

Therefore $\bigvee A \geq \bigvee A^*$ and $\bigvee B \geq \bigvee B^*$, giving a morphism.

Uniqueness: Morphisms are determined by the order relations, which are unique in a poset.

Terminality interpretation: Any valid decomposition factors through the weakest one, making (A^*, B^*) the universal solution. $\square$

8.2.2 Adjunction Between CDS and PDS Admission

Definition 8.7 (Admission Functors). *Define functors between categories of motive geometries:*

- **Cone**: category of closed convex cones with inclusions
- **Cover**: category of finite covers with refinements
- $F : \mathbf{Cone} \rightarrow \mathbf{Cover}$: maps W to its extreme rays
- $G : \mathbf{Cover} \rightarrow \mathbf{Cone}$: maps W_{ref} to $\text{cone}(W_{ref})$

Theorem 8.8 (CDS-PDS Adjunction). *The functors $F \dashv G$ form an adjunction, where:*

- The unit $\eta_W : W \rightarrow GF(W)$ is the inclusion into the cone of extreme rays
- The counit $\varepsilon_{W_{ref}} : FG(W_{ref}) \rightarrow W_{ref}$ selects extreme rays from the generated cone

This adjunction induces a Galois connection between admission criteria:

$$\text{CDS-admitted under } W \subseteq \text{PDS-admitted under } F(W)$$

$$\text{PDS-admitted under } W_{ref} \supseteq \text{CDS-admitted under } G(W_{ref})$$

Proof. We verify the adjunction by showing the natural bijection:

$$\text{Hom}_{\mathbf{Cover}}(F(W), W_{ref}) \cong \text{Hom}_{\mathbf{Cone}}(W, G(W_{ref}))$$

Left to right: Given a cover morphism $\phi : F(W) \rightarrow W_{ref}$ (refinement), the cone map $W \rightarrow \text{cone}(W_{ref})$ is the inclusion determined by ϕ on extreme rays.

Right to left: Given a cone morphism $\psi : W \rightarrow G(W_{ref})$ (inclusion), restrict to extreme rays to get $F(W) \rightarrow W_{ref}$.

Naturality: Both constructions commute with morphisms in the respective categories.

Admission correspondence: An option with margin $m_o(x; W) \geq 0$ satisfies componentwise dominance on $F(W)$ (extreme rays). Conversely, Pareto dominance on W_{ref} implies cone dominance on $G(W_{ref})$. $\square$

8.2.3 Limits and Colimits in Option Composition

Theorem 8.9 (Pushouts for Option Gluing). *Let options o_1, o_2 have domains $U_1, U_2 \subseteq X$ with overlap $U_{12} = U_1 \cap U_2$. The glued option o_{12} is the*

pushout in the category of partial transformers:

$$\begin{array}{ccc} U_{12} & \xrightarrow{i_1} & U_1 \\ i_2 \downarrow & & \downarrow j_1 \\ U_2 & \xrightarrow{j_2} & U_1 \cup U_2 \end{array}$$

where the glued transformer is:

$$[T_{o_{12}}v](x) = \begin{cases} [T_{o_1}v](x) & x \in U_1 \setminus U_2 \\ [T_{o_2}v](x) & x \in U_2 \setminus U_1 \\ \max([T_{o_1}v](x), [T_{o_2}v](x)) & x \in U_{12} \end{cases}$$

Proof. Universal property: For any V with morphisms from U_1 and U_2 agreeing on U_{12} , there exists a unique morphism from $U_1 \cup U_2$ to V .

The max operation on overlaps ensures:

1. Join-continuity is preserved (max of join-continuous functions)
2. The glued option dominates both constituents where they overlap
3. CDS/PDS admission is preserved if both parts were admitted

□

Theorem 8.10 (Products for Parallel Composition). *The parallel execution of options o_1, o_2 on independent aspects corresponds to the categorical product in $\mathcal{K}_M$:*

$$T_{o_1 \times o_2} = T_{o_1} \times T_{o_2} : V \times V \rightarrow V \times V$$

with projections $\pi_i : T_{o_1 \times o_2} \rightarrow T_{o_i}$.

Proof. The product transformer acts componentwise:

$$[T_{o_1 \times o_2}(v_1, v_2)](x) = ([T_{o_1}v_1](x), [T_{o_2}v_2](x))$$

Universal property: For any T with morphisms to both T_{o_1} and T_{o_2} , there exists unique factorization through $T_{o_1 \times o_2}$.

Admission: If o_1 is CDS-admitted for aspect 1 and o_2 for aspect 2, then $o_1 \times o_2$ is admitted for the product slice (componentwise in PDS terms). □

8.2.4 Colimit of Evolutionary Bundles

Theorem 8.11 (Filtered Colimits for Evolutionary Improvement). *Let $(B_t)_{t \in \mathbb{N}}$ be an evolutionary sequence of option bundles with $B_t \subseteq B_{t+1}$ (monotone improvement). The limiting behavior is the filtered colimit:*

$$B_\infty = \operatorname{colim}_{t \rightarrow \infty} B_t = \bigcup_{t=0}^{\infty} B_t$$

with the universal property that any eventually stable property of bundles factors through B_∞ .

Proof. **Cocone structure:** The inclusions $B_t \hookrightarrow B_\infty$ form a cocone.

Universal property: For any C with compatible morphisms from all B_t , the map $B_\infty \rightarrow C$ is unique by:

$$f_\infty(o) = f_t(o) \text{ for any } t \text{ where } o \in B_t$$

Admission persistence: Under MDN monotonicity (Proposition 7.2), if $o \in B_t$ is admitted, then $o \in B_\infty$ remains admitted.

Weakness bound: If weakness is lax-monoidal, then:

$$w(B_\infty) \leq \liminf_{t \rightarrow \infty} w(B_t)$$

□

8.2.5 Implications of Categorical Structure

These theorems reveal that:

1. **Optimality is universal:** The weakest decomposition is not just minimal but universally factors all other decompositions
2. **CDS and PDS are dual viewpoints:** The adjunction shows they're two faces of the same phenomenon, related by a Galois connection
3. **Option composition has canonical forms:** Pushouts give the right way to glue options with overlaps; products handle parallel independent execution
4. **Evolution converges categorically:** The filtered colimit characterizes what evolutionary search achieves in the limit

This categorical perspective provides a principled foundation for:

- Proving when decompositions are optimal (terminal objects)
- Converting between admission criteria (adjunction)
- Combining options correctly (limits/colimits)
- Understanding evolutionary convergence (filtered colimits)

8.3 Connection to Polynomial Functors and Compositional Systems

We now explore how the categorical framework developed above relates to Spivak's polynomial functor approach to compositional dynamical systems [14]. This connection reveals additional structure in our subgoal decomposition framework and suggests computational approaches for weakness regularization.

8.3.1 Polynomial Functors as Option Interfaces

Recall that a polynomial functor $p : \mathbf{Set} \rightarrow \mathbf{Set}$ has the form

$$p(y) = \sum_{i \in p(1)} y^{p[i]}$$

where $p(1)$ is the set of positions and $p[i]$ is the arity at position i . In the context of subgoal decomposition, we can interpret:

- Positions $i \in p(1)$ as possible entry states for an option/subgoal
- Arities $p[i]$ as the interface of possible successor states from position i
- A morphism $\varphi : p \rightarrow q$ as an implementation of interface q using interface p

Definition 8.12 (Option Polynomial). *For an option o with expectation model $(\hat{r}, \hat{n})$ on feature space X , define its interface polynomial p_o by:*

- $p_o(1) = X$ (positions are feature states)
- $p_o[x] = \text{supp}(\hat{n}(x, o))$ (arity is the support of successor features)

This representation makes the input-output structure of options explicit and separates interface from implementation.

8.3.2 Compositional Structure via Polynomial Composition

The composition of polynomial functors $p \otimes q$ given by

$$(p \otimes q)(y) = p(q(y))$$

corresponds precisely to sequential option composition in our framework. Specifically, if T_o and $T_{o'}$ are decision transformers, their composition $T_{o'} \circ T_o$ corresponds to the polynomial composition $p_o \otimes p_{o'}$.

Proposition 8.13 (Functorial Planning). *The assignment $o \mapsto p_o$ extends to a functor*

$$\mathcal{P} : \mathcal{K}_M \rightarrow \mathbf{Poly}$$

from our planning category to the category of polynomial functors, where:

- Planning composition $T_{o'} \circ T_o$ maps to $p_o \otimes p_{o'}$
- The planner join $\bigvee_i T_{o_i}$ maps to the coproduct $\sum_i p_{o_i}$

8.3.3 Instantiating Quantale Weakness for Polynomials

Recall from Definition 8.3 that a weakness functional is any map $w : K \rightarrow Q$ to a commutative quantale satisfying monotonicity, lax monoidality, and normalization. The polynomial functor structure allows us to construct specific, computable instantiations of this general concept.

Definition 8.14 (Interface Complexity Weakness). *One natural instantiation of weakness for polynomial functors measures interface complexity:*

$$w_{ifc}(p) = |p(1)| \oplus \bigoplus_{i \in p(1)} \rho(|p[i]|)$$

where $\rho : \mathbb{N} \rightarrow Q$ maps arity to weakness (e.g., $\rho(n) = \log n$ for entropy-based weakness).

This specific weakness functional captures the intuition that "weaker" (more permissive) options are those with:

- Many possible entry states (large $|p(1)|$)
- High branching at each state (large $|p[i]|$)

Remark 8.15 (Alternative Polynomial Weakness Functionals). The interface complexity weakness is just one possible instantiation. Other valid weakness functionals on polynomial functors include:

1. **Maximum branching:** $w_{\max}(p) = \max_{i \in p(1)} |p[i]|$ (worst-case non-determinism)
2. **Total transitions:** $w_{\text{total}}(p) = \sum_{i \in p(1)} |p[i]|$ (counting all possible transitions)
3. **Entropy-weighted:** $w_{\text{ent}}(p) = \sum_{i \in p(1)} \pi_i H(p[i])$ where π_i is the visitation probability and H is entropy
4. **Kolmogorov-style:** $w_K(p) =$ shortest description length of the polynomial's transition structure

Each satisfies the axioms of Definition 8.3 under appropriate quantale structures, and each emphasizes different aspects of behavioral permissiveness.

Theorem 8.16 (Weakness Preservation). *For any valid weakness functional w on polynomials that respects the functor $\mathcal{P}$:*

$$w(T_o) \leq_Q w(T_{o'}) \implies w(p_o) \leq_Q w(p_{o'})$$

Moreover, if w is chosen to be lax-monoidal on **Poly**, then:

$$w(p_o \otimes p_{o'}) \leq_Q w(p_o) \oplus w(p_{o'})$$

8.3.4 Computational Advantages

The polynomial functor perspective provides computational benefits regardless of which specific weakness functional we choose:

1. **Sparse Representations:** When $|p[i]|$ is small (sparse successor features), we can use the polynomial structure to compress option models, storing only $(i, p[i])$ pairs rather than full matrices.
2. **Compositional Caching:** The factorization $p \otimes q$ allows caching intermediate compositions, useful when the same option sequences recur.
3. **Type-Directed Synthesis:** The polynomial interface constrains which options can compose, enabling type-directed search for weakest decompositions.

8.3.5 Lens Structure and Bidirectional Planning

Following Spivak’s observation that polynomial functors form lenses, each option induces a lens

$$\ell_o = (p_o^\#, p_o^b) : p_o \rightarrow p_o$$

where:

- Forward map $p_o^\#$: executes the option (computing $\hat{n}$)
- Backward map p_o^b : backpropagates value updates

This lens structure naturally supports bidirectional planning where value updates flow backward while state predictions flow forward.

8.3.6 Practical Weakness Regularization

For implementation in neural networks, we can use any of the polynomial weakness functionals as regularizers. Here we illustrate with the interface complexity version:

Polynomial Weakness Regularization

1. Given option o with heads $(\hat{r}, \hat{n})$
2. Compute interface polynomial p_o via sparsification of $\hat{n}$
3. Choose weakness functional (e.g., w_{ifc} , w_{max} , or w_{ent})
4. Evaluate $w(p_o)$ according to chosen functional
5. Add penalty $\lambda \cdot w(p_o)$ to training loss
6. During CDS/PDS admission, prefer options with lower $w(p_o)$

The choice of weakness functional can be task-dependent:

- Use w_{ifc} for general complexity regularization
- Use w_{max} when worst-case branching is the bottleneck
- Use w_{ent} when options have known visitation distributions
- Use w_K when description length aligns with communication costs

8.3.7 Categorical Unification

The polynomial functor view unifies our three learning paradigms:

- **Neural:** Learn smooth approximations to polynomial interfaces, with weakness as a regularization term
- **Logical:** Guards define position predicates; rules define arity constraints; weakness measures proof/rule complexity
- **Evolutionary:** Search over polynomial compositions with weakness pressure selecting for simpler interfaces

All three produce morphisms in **Poly** that factor through our planning category $\mathcal{K}_M$ via the functor $\mathcal{P}$, and all can use the same family of weakness functionals for regularization.

Remark 8.17 (Future Directions). The polynomial functor framework suggests several extensions:

1. Use dependent polynomial functors to capture state-dependent arities and context-sensitive weakness
2. Exploit the topos structure of **Poly** for logical specifications with internal weakness measures
3. Apply polynomial comonads for hierarchical option discovery with compositional weakness bounds
4. Use the Para construction for parallel option composition with additive weakness

These directions could further strengthen the theoretical foundations while maintaining computational tractability.

9 Combining Neural, Logical, and Evolutionary Methods: Theory and Practice

We now formalize how neural gradient descent, logical inference, and evolutionary search can be combined for subgoal decomposition. The key intuition is that complex goals can be decomposed hierarchically:

1. **Local patches:** Small regions where behavior is smooth and predictable, suitable for neural/PC modules

2. **Logical gluing:** Clear decision boundaries that partition the space and specify when to use which local behavior
3. **Evolutionary assembly:** Global search over combinations of glued pieces to cover the full task

We call this structure an *Evo-Adhesive Smooth Atlas* (EASA), by analogy with mathematical atlases that cover manifolds with overlapping charts. The CDS/PDS admission criteria from previous chapters provide the certification mechanism that ensures each piece genuinely helps achieve the MDN-defined motives.

9.1 Formal EASA Definition and Ideal Properties

Definition 9.1 (EASA Structure). *A goal transformer $T \in K_M$ on region Ω has an Evo-Adhesive Smooth Atlas at scale $\epsilon > 0$ if there exist:*

1. **Contractive charts:** *A finite cover $\{U_i\}$ of Ω with local models $C_i : U_i \rightarrow U_i$ satisfying:*
 - $Lip(C_i) \leq \lambda < 1$ in the planning metric d_M
 - $d_M(T|_{U_i}, C_i) \leq \epsilon/6$ on each patch
2. **Adhesive gluing:** *The cover admits guards $G_i : X \rightarrow \{0, 1\}$ such that:*
 - $G_i \cap G_j$ has measure $\leq \delta \cdot \min(\mu(G_i), \mu(G_j))$ (thin overlaps)
 - On overlaps, $|C_i(x) - C_j(x)| \leq \epsilon/6$ (consistent boundaries)
 - Pushouts along overlaps exist and preserve backed-up values
3. **Bounded interaction:** *For amalgams A_α formed by gluing charts, and any subset S with $|S| \leq m$:*

$$d_M \left(T|_{\bigcup_{A \in S} A}, \bigvee_{A \in S} C_A \right) \leq \Delta \cdot |S|$$

with $\Delta \leq \epsilon/(4m)$ (near-additivity).

4. **Evolvability:** *The set of amalgams admits a probability distribution with:*
 - *Schema stability:* k -subsets survive with probability $\geq \rho^k$

- *Coverage:* Expected time to reach any m -subset is polynomial in $(m, |\{A_\alpha\}|, 1/\rho)$

5. **CDS/PDS admissibility:** Each C_A satisfies either:

- *CDS:* $\inf_{w \in W} (B_{C_A}(x; w) - B_{base}(x; w)) \geq 0$ on support
- *PDS:* Pareto-dominance on W_{ref} with budgets (ϵ, ρ, q)

9.2 When EASA Structure Provably Emerges

We now give conditions under which neural, logical, and evolutionary methods actually satisfy EASA properties. These conditions are strong and rarely met in practice, but they clarify what would be needed for the idealized theory to hold.

Theorem 9.2 (Verifiable EASA Emergence). *Consider a learning system with the following components:*

Neural modules: Feed-forward networks $f_\theta : X \rightarrow X$ with:

- *Spectral normalization:* $\|W_i\|_2 \leq \sigma < 1$ for all weight matrices
- *Residual structure:* $f(x) = \alpha x + (1 - \alpha)g(x)$ with $\alpha \in (0, 1)$
- *Lipschitz activations with constant $L = 1$* (ReLU, tanh)

Logical guards: Decision predicates $\{G_i\}$ with:

- *Tree structure of depth $\leq D$ with binary splits*
- *Mutual exclusion:* $G_i \cap G_j = \emptyset$ for $i \neq j$ (up to boundaries)
- *Consistent labeling:* Each leaf maps to exactly one module

Evolutionary search: Population dynamics with:

- *Hamming distance constraint:* $d_H(g_i, g_j) \geq \Delta_H$ for all genotypes
- *Elite preservation:* Top k individuals always survive
- *Uniform crossover with rate $p_c < 1/(2m)$*

Then the system exhibits EASA structure with:

$$\lambda = \alpha + (1 - \alpha) \prod_i \|W_i\|_2 < 1 \quad (10)$$

$$\delta = 2^{-D} \text{ (overlap measure)} \quad (11)$$

$$\Delta = O(m^2 \cdot 2^{-\Delta_H}) \text{ (interaction bound)} \quad (12)$$

$$\rho = 1 - p_c - p_m \text{ (schema survival)} \quad (13)$$

Proof Sketch. **Contraction:** The residual structure ensures $\text{Lip}(f) = \alpha + (1 - \alpha)\text{Lip}(g)$. With spectral normalization and unit-Lipschitz activations, $\text{Lip}(g) \leq \prod_i \|W_i\|_2 < 1$.

Thin overlaps: Binary tree guards with depth D create 2^D regions. Boundaries have codimension 1, hence measure zero. Adjacent regions share at most 2^{-D} of their volume.

Bounded interaction: Hamming distance constraint ensures at most $m^2 \cdot 2^{-\Delta_H}$ feature overlaps between any m genotypes, bounding cross-effects.

Evolvability: Elite preservation and low crossover rate ensure building blocks persist with the stated probability. $\square$

9.3 Why These Conditions Are Too Strong

The conditions in Theorem 1 are rarely satisfied in practice:

9.3.1 Neural Network Limitations

Contraction vs. Expressiveness: Enforcing $\|W_i\|_2 < 1$ severely limits what functions can be learned. Most practical networks require $\|W_i\|_2 \approx 1$ or larger to fit complex patterns. With strict spectral normalization:

- Gradients vanish exponentially with depth
- The network cannot amplify important features
- Training becomes extremely slow or fails to converge

Discontinuities in Real Tasks: Physical systems have inherent discontinuities:

- Contact dynamics (collision, grasping)
- Discrete decisions (door open/closed)
- Sensor limitations (occlusion, noise)

These violate the smoothness assumption required for contraction.

9.3.2 Logical Guard Limitations

Mutual Exclusion is Unrealistic: Real decision boundaries are fuzzy:

- Sensor uncertainty creates overlapping regions
- Smooth transitions are needed for stability

- Multiple modes may be partially active

Fixed Tree Structure: Requiring depth- D trees is restrictive:

- Real decisions may need variable depth
- Guards must adapt during learning
- Compositional rules don't fit tree structure

9.3.3 Evolutionary Limitations

Hamming Distance Constraint: Maintaining minimum distance between all genotypes:

- Limits population size severely
- Prevents convergence to good solutions
- Conflicts with fitness pressure

Low Crossover Rate: The bound $p_c < 1/(2m)$ is too restrictive:

- Real evolution needs higher mixing
- Building blocks must recombine frequently
- Diversity requires exploration

9.4 Approximate EASA in Practice

Despite these limitations, approximate EASA structure can emerge and be useful:

9.4.1 Soft Contraction via Regularization

Instead of hard constraints, use soft penalties:

$$\mathcal{L}_{\text{contract}} = \beta \cdot \max(0, \|\nabla_x f(x)\|_2 - \lambda_{\text{target}}) \quad (14)$$

$$\lambda_{\text{target}} \in [0.9, 1.1] \text{ (allowing mild expansion)} \quad (15)$$

This gives approximate local contraction:

- Most regions have $\lambda \approx 0.95$
- Occasional expansion near transitions
- Sufficient for local stability

9.4.2 Fuzzy Guards with Smooth Blending

Replace crisp guards with soft assignments:

$$G_i^{\text{soft}}(x) = \frac{\exp(s_i(x)/\tau)}{\sum_j \exp(s_j(x)/\tau)} \quad (16)$$

$$\text{Output} = \sum_i G_i^{\text{soft}}(x) \cdot C_i(x) \quad (17)$$

This provides:

- Smooth transitions (differentiable)
- Natural overlap handling
- Learnable boundaries via s_i

9.4.3 Statistical Interaction Bounds

Instead of worst-case bounds, use empirical measures:

$$\Delta_{\text{empirical}} = \frac{1}{n} \sum_{x \in \text{samples}} \left| f_{\text{joint}}(x) - \sum_i f_i(x) \right| \quad (18)$$

$$\text{Interaction degree} = \mathbb{E}[\text{active modules}] \text{ not } \max \quad (19)$$

9.4.4 Practical Evolutionary Search

Relax constraints to feasible levels:

- Maintain diversity via niching, not hard distance
- Use moderate crossover ($p_c \approx 0.3$)
- Allow similar genotypes with fitness sharing

Proposition 9.3 (Approximate EASA). *Under the relaxed conditions above, with high probability over training:*

1. *Local contractiveness:* $\lambda \in [0.85, 1.15]$ on 80% of state space
2. *Smooth guards:* Overlap regions comprise 10-30% of volume
3. *Bounded expected interaction:* $\mathbb{E}[\Delta] = O(\sqrt{m})$ not $O(m)$
4. *Practical evolvability:* Good solutions found in $O(n \log n)$ generations

These approximate properties suffice for:

- *Local stability guarantees*
- *Compositional reasoning in limited contexts*
- *Meaningful weakness bounds with slack*
- *Practical subgoal decomposition*

9.5 Concrete Example: Robot Package Delivery

We illustrate approximate EASA through a robot navigation task.

9.5.1 Task Description

A mobile robot must:

- Navigate office building with packages
- Start at loading dock, deliver to offices
- Avoid humans and obstacles
- Balance speed, safety, battery efficiency

State space: $x = (\text{position, velocity, sensor readings, battery})$

9.5.2 Layer 1: Neural/PC Local Behaviors

We train small PC modules for local navigation:

$$\text{Modules: } \{\text{follow_wall, center_corridor, approach_door,} \quad (20)$$

$$\text{avoid_human, dock_corner}\} \quad (21)$$

Each module C_i is a 3-layer PC network:

$$\text{PC update: } \Delta x = -\Pi \cdot (x - \hat{x}) \quad (22)$$

$$\text{Successor: } \hat{n}(x, C_i) = U_i D_i V^T x \quad (\text{rank-8}) \quad (23)$$

Measured properties:

- Empirical $\lambda \approx 0.92$ when going straight
- Empirical $\lambda \approx 1.08$ during turns
- Effective radius: 3 meters

Reality: Contraction breaks near walls (discontinuous dynamics).

9.5.3 Layer 2: Logical Mode Selection

Guards learned via decision trees:

$$G_{\text{avoid}}(x) = \begin{cases} 0.9 & \text{if human detected} \\ 0.1 & \text{otherwise} \end{cases} \quad (24)$$

$$G_{\text{door}}(x) = \begin{cases} 0.8 & \text{if } d_{\text{door}} < 2m \\ 0.2 & \text{otherwise} \end{cases} \quad (25)$$

Soft blending in overlaps:

$$C_{\text{combined}}(x) = \sum_i \frac{G_i(x)}{\sum_j G_j(x)} \cdot C_i(x) \quad (26)$$

Measured properties:

- Overlap regions: 28% of states
- Average active modules: 1.7
- Transition smoothness: C^1 except at walls

9.5.4 Layer 3: Evolutionary Macro Assembly

Genome encodes sequences: $g = [(C_{i_1}, t_1), (C_{i_2}, t_2), \dots]$

Fitness evaluation:

$$F(g) = \text{success_rate} - 0.1 \cdot \text{time} - 0.05 \cdot \text{battery} - \text{penalties} \quad (27)$$

Population dynamics (MOSES-style):

- Population: 50 genomes
- Crossover: $p_c = 0.3$
- Mutation: $p_m = 0.1$
- Generations: 30

Discovered macros:

- M_1 : dock_to_elevator (8 steps)
- M_2 : elevator_to_office (6 steps)

- M_3 : emergency_exit (4 steps)

Measured interaction:

$$\Delta_{\text{measured}} = |F(M_1 \cup M_2) - F(M_1) - F(M_2)| \quad (28)$$

$$\approx 0.3 \text{ (moderate interaction)} \quad (29)$$

9.5.5 CDS/PDS Admission in Practice

Given MDN weights $W = \{w_{\text{safety}}, w_{\text{speed}}, w_{\text{battery}}\}$:

$$\text{Empirical margin: } \hat{m}_i = \frac{1}{n} \sum_{x \in \text{samples}} \inf_{w \in W} (B_{C_i}(x; w) - B_{\text{base}}(x; w)) \quad (30)$$

$$\text{Admission: Accept if } \hat{m}_i > 2\sigma_i \text{ (2-std confidence)} \quad (31)$$

Results:

- 5/8 primitives admitted
- 3/5 macros admitted
- Planning speedup: 3.2x

9.5.6 What Actually Works

Despite violating ideal EASA conditions:

1. **Local success:** Within single corridors, learned primitives compose well
2. **Hierarchical value:** Three-layer structure provides clear benefits:
 - Neural: Smooth continuous control
 - Logic: Discrete mode decisions
 - Evolution: Useful behavior patterns
3. **Practical guarantees:** Statistical bounds replace worst-case:
 - 90% success rate (not 100%)
 - Local stability (not global)
 - Empirical weakness bounds (not tight)

9.6 Theoretical Implications of Approximate EASA

Theorem 9.4 (Approximate Compositional Weakness). *Under approximate EASA with parameters $(\tilde{\lambda}, \tilde{\delta}, \tilde{\Delta}, \tilde{\rho})$ where:*

- $\tilde{\lambda} \in [0.9, 1.1]$ (approximate contraction)
- $\tilde{\delta} \in [0.1, 0.3]$ (moderate overlap)
- $\tilde{\Delta} = O(\sqrt{m})$ (sub-linear interaction)

The compositional weakness bound becomes:

$$w(h^*) \leq \left(\bigotimes_{i \in I} w_{NN}(C_i^*) \right) \otimes c_{\tilde{\Delta}} \otimes \xi$$

where $\xi = O(\log(1/\epsilon))$ is an approximation slack factor.

This weaker bound still provides:

- Meaningful regularization during training
- Preference for simpler decompositions
- Interpretable complexity measures

9.7 Practical Guidelines

Based on the theory and empirical observations:

1. **Design for approximate EASA:**
 - Use soft regularization, not hard constraints
 - Allow overlapping guards with smooth blending
 - Accept statistical rather than worst-case bounds
2. **Monitor empirical properties:**
 - Track local Lipschitz constants during training
 - Measure overlap statistics
 - Evaluate interaction empirically on validation sets
3. **Exploit partial structure:**
 - Identify regions where EASA holds well
 - Use fallback mechanisms elsewhere
 - Maintain confidence estimates

9.8 Conclusion

The EASA framework provides valuable conceptual guidance even when its strict conditions cannot be met. By understanding both the ideal theory and practical relaxations, we can design systems that capture many benefits of compositional structure while remaining implementable with current methods. The key insight is that approximate EASA—with local contractiveness, fuzzy guards, and statistical bounds—suffices for practical subgoal decomposition while maintaining useful theoretical properties in expectation.

9.9 Realistic Compositional Weakness Under Approximate EASA

Having established that strict EASA conditions are unattainable in practice, we now develop a more realistic theory of compositional weakness that provides useful guarantees despite relaxed assumptions.

9.9.1 From Ideal to Achievable Composition

Recall from above that ideal compositional weakness under EASA would satisfy:

$$w(h^*) \leq \left(\bigotimes_{i \in I} w_{\text{NN}}(C_i^*) \right) \otimes c_\Delta$$

where c_Δ is a small constant capturing bounded interaction. This requires:

- Strict contraction: $\lambda < 1$ uniformly
- Thin overlaps: $\mu(G_i \cap G_j) = O(\epsilon)$
- Bounded interaction: $\Delta = O(1)$ independent of system size

These conditions fail in practice. We therefore develop a weaker but achievable theory.

9.9.2 Approximate EASA and Statistical Weakness Bounds

Definition 9.5 (Approximate EASA). *A system exhibits (α, β) -approximate EASA if with probability $\geq 1 - \alpha$ over training:*

1. **Regional contractiveness:** *There exist regions $\{R_i\}$ covering $\geq \beta$ fraction of visited states where local Lipschitz constants satisfy $\lambda_i \in [0.85, 1.2]$*

2. **Soft boundaries:** Guard overlaps satisfy $\mu(G_i \cap G_j) \leq 0.3 \cdot \min(\mu(G_i), \mu(G_j))$

3. **Sublinear interaction:** For m components, interaction satisfies $\mathbb{E}[\Delta] = O(m^\gamma)$ with $\gamma < 1$

Theorem 9.6 (Realistic Compositional Weakness). *For a system with n components satisfying $(\alpha, 0.5)$ -approximate EASA with interaction exponent $\gamma = 1/2$, the compositional weakness satisfies with probability $\geq 1 - \alpha$:*

$$w(h^*) \leq \left(\prod_{i=1}^n w_i \right) \cdot \exp(O(\sqrt{n}))$$

where w_i are local weakness measures in the quantale $(Q, \leq, \oplus)$.

Proof. Consider the decomposition of global weakness into local and interaction terms:

$$w(h^*) = w_{\text{local}} \oplus w_{\text{interact}}$$

Local term: On regions R_i with approximate contraction, local weakness composes with error:

$$w_{\text{local}} \leq \prod_{i=1}^n w_i \cdot (1 + \epsilon_i)$$

where $\epsilon_i = |\lambda_i - 1|$. With $\lambda_i \in [0.85, 1.2]$, we have $\epsilon_i \leq 0.2$.

Interaction term: With $\mathbb{E}[\Delta] = O(\sqrt{n})$, the interaction contributes:

$$w_{\text{interact}} = \sum_{|S| \leq m} \Delta_S \leq \sum_{k=1}^{\sqrt{n}} \binom{n}{k} O(k^{1/2}) = O(n \cdot \sqrt{n}) = O(n^{3/2})$$

In the logarithmic quantale (where $\oplus$ is addition), this gives:

$$\log w(h^*) \leq \sum_{i=1}^n \log w_i + O(\sqrt{n})$$

Exponentiating yields the claimed bound. □

9.9.3 Practical Implications

This weaker bound still provides valuable guarantees:

Proposition 9.7 (Effective Regularization). *Under (α, β) -approximate EASA, the regularized objective*

$$\mathcal{L} = \mathcal{L}_{task} + \lambda_w \sum_{i=1}^n w_i + \lambda_\Delta \|\Delta\|_F$$

with appropriately chosen $\lambda_w, \lambda_\Delta > 0$, encourages decompositions with:

1. *Lower total weakness: $\sum w_i = O(n)$ rather than $O(n^2)$*
2. *Sparse interaction: $\|\Delta\|_0 = O(n)$ rather than $O(n^2)$*
3. *Improved generalization: Test error bounded by $O(\sqrt{n})$ rather than $O(n)$*

9.9.4 Empirical Weakness Measurement

Since theoretical bounds are loose, we advocate empirical measurement:

Definition 9.8 (Empirical Compositional Weakness). *Given trained components $\{C_i\}$ and test distribution $\mathcal{D}$, define:*

$$w_{emp}^{local} = \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{x \sim \mathcal{D}}[\text{complexity}(C_i|x)] \quad (32)$$

$$w_{emp}^{interact} = \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| \bigvee_i C_i(x) - \sum_i C_i(x) \right\| \right] \quad (33)$$

$$w_{emp}^{total} = w_{emp}^{local} \cdot \exp(w_{emp}^{interact}) \quad (34)$$

This empirical measure:

- Can be computed during training
- Provides tighter bounds than worst-case theory
- Guides architecture search and hyperparameter tuning

9.9.5 Weakness as a Design Principle

Despite lacking tight theoretical bounds, compositional weakness remains valuable as:

1. **Architecture guidance:** Prefer decompositions with measured $w_{emp}^{interact} < \log n$

2. **Regularization signal:** The penalty $\lambda_w \sum w_i$ improves generalization even without tight bounds
3. **Interpretability metric:** Components with lower w_i are simpler and more reusable
4. **Search heuristic:** Evolutionary algorithms can use weakness as a secondary objective

Remark 9.9 (Theory-Practice Gap). The exponential slack $\exp(O(\sqrt{n}))$ in our bound appears pessimistic, but empirically we often observe:

$$w_{\text{emp}}^{\text{total}} \leq \left(\prod w_i \right) \cdot (1 + O(\log n))$$

This suggests that typical problem structure is more favorable than worst-case analysis indicates. Future work should identify problem classes where tighter bounds hold.

9.9.6 Guidelines for Practitioners

To leverage compositional weakness despite theoretical limitations:

1. Train components with soft regularization: $\mathcal{L} + \lambda_w \sum w_i$
2. Measure empirical interaction: $\Delta_{\text{emp}} = \mathbb{E}[\|\text{joint} - \text{sum}\|]$
3. If $\Delta_{\text{emp}} > \sqrt{n}$:
4. Increase diversity pressure in evolutionary search
5. Add orthogonality penalties to neural components
6. Refine guard boundaries to reduce overlaps
7. Select decomposition minimizing $w_{\text{emp}}^{\text{total}}$ subject to performance constraints
8. Monitor: If $w_{\text{emp}}^{\text{total}}$ grows faster than $n \log n$, redesign architecture

9.9.7 Conclusion on Compositional Weakness

While ideal compositional weakness with tight multiplicative bounds is unachievable, the weaker guarantee $w(h^*) \leq (\prod w_i) \cdot \exp(O(\sqrt{n}))$ still provides:

- Sublinear penalty for composition (better than naive $O(n^2)$)
- Effective regularization signal during training
- Empirically measurable complexity metric
- Architectural guidance for modular design

The key insight is that compositional weakness functions as a useful *design principle* and *soft constraint* even when it cannot provide hard mathematical guarantees. Systems designed with compositional weakness in mind exhibit better generalization, interpretability, and reusability than monolithic alternatives, justifying its use despite the theory-practice gap.

10 Economic Attention Allocation for Resource-Aware Subgoal Selection

The CDS/PDS framework with co-learned MDN successfully addresses the problem of *which* subgoals and options should be admitted into the planning library—those that robustly improve backed-up values across the motive slice. However, admission is necessary but not sufficient for practical goal pursuit. Even with a carefully curated library of certified options, an agent faces the fundamental problem of *resource allocation*: given limited computational capacity, memory, and time, which admitted options should actually be executed, and with what intensity? This is particularly acute in hierarchical decomposition where multiple subgoals may compete for resources while others require coordinated execution.

Economic attention allocation mechanisms like ECAN provide a principled solution, within the PRIMUS architecture, by treating attention as a scarce, conserved resource that must be distributed according to expected utility. The RFS mechanism, also proposed as part of PRIMUS, extends this with a sophisticated promise/redemption system for hierarchical goal structures. However, in the context of SubRep—where options are already certified by CDS/PDS and motives are explicitly decomposed by the MDN—we can achieve the same economic allocation benefits with a simpler, more direct conservative attention mechanism that leverages the robust margins

and gaps already computed during admission. We will now develop this simplified approach, showing how it preserves the key insights of ECAN and RFS while eliminating unnecessary complexity and providing tighter integration with the SubRep framework.

10.1 From RFS to Conservative Attention: A Simplified Approach

10.1.1 Review of the RFS Mechanism

The PRIMUS architecture’s Request for Service (RFS) mechanism, described in Chapter 6 of [?], provides a sophisticated approach to attention allocation through a two-stage process of promises and redemptions. In this system:

- Goals issue promissory notes (RFS) to subgoals that might help achieve them, based on PredictiveImplication links
- RFS transfers are non-conserved promises: a goal with x units of currency can promise x units to multiple alternative subgoals
- When a subgoal judges it appropriate, it may redeem the RFS, requesting actual currency transfer from the issuing goal
- Goals may withdraw RFS at any time if circumstances change
- For joint requirements (where achieving goal A requires both B and C), constraint-based RFS are issued with conditions like $(B + C \leq x)$

This mechanism elegantly handles the credit assignment problem in hierarchical goal structures, ensuring that attention flows from top-level goals down to actionable procedures. The non-conserved nature of promises allows exploration of multiple paths simultaneously, while the redemption mechanism ensures actual currency conservation when execution occurs.

10.1.2 Limitations in the SubRep Context

While the RFS mechanism is powerful, it introduces complexity that may be unnecessary in the context of SubRep with CDS/PDS admission:

1. **Two-stage overhead:** The promise/redemption cycle requires tracking both promissory and actual currency, with complex bookkeeping for withdrawals and constraint satisfaction

2. **Indirect certification:** RFS allocation is based on PredictiveImplication strengths, while CDS/PDS already provide direct, robust measures of which options actually improve planning across the motive slice
3. **Potential for deadlock:** Constraint-based RFS for joint requirements can lead to coordination problems where subgoals must negotiate redemption timing
4. **Credit attribution complexity:** When multiple goals issue RFS to the same subgoal, determining which goal gets credit for achievement becomes intricate

10.1.3 Conservative Attention as a Streamlined Alternative

The conservative attention mechanism we propose maintains the key insights of RFS while eliminating unnecessary complexity:

- **Single-stage allocation:** Replace promises with direct attention allocation from a conserved pool, eliminating the promise/redemption distinction
- **CDS/PDS integration:** Use margins $m_o(x; W)$ and Pareto gaps directly as allocation signals, leveraging the robust admission criteria already computed
- **Natural conservation:** With $\sum_o a(o) = A_{\text{total}}$ enforced at every step, there's no possibility of over-promising or credit leaks
- **Simplified joint requirements:** Instead of constraint-based RFS, use attention-weighted fusion where modalities naturally compete for a share of the conserved pool

The relationship between the two approaches can be formalized as follows. Where RFS would compute:

$$\text{RFS}(B|A) = \text{promise}(A) \cdot P(\text{PredictiveImplication } B \rightarrow A) \quad (35)$$

$$\text{actual}(B) = \begin{cases} \text{RFS}(B|A) & \text{if } B \text{ redeems from } A \\ 0 & \text{otherwise} \end{cases} \quad (36)$$

Conservative attention instead computes:

$$a(o) = a_{\text{prev}}(o) \cdot \gamma + A_{\text{free}} \cdot \frac{\exp(m_o(x; W)/\tau)}{\sum_{o'} \exp(m_{o'}(x; W)/\tau)} \quad (37)$$

This direct formulation:

- Eliminates the stochastic redemption step
- Replaces PredictiveImplication strengths with CDS/PDS margins that already account for robust improvement across the motive slice
- Maintains conservation automatically through the softmax normalization
- Provides immediate, deterministic allocation rather than contingent promises

10.1.4 Preserving Economic Insights

Despite its simplicity, conservative attention preserves the key economic insights from RFS and ECAN:

1. **The Baum principle:** Total system reward remains bounded by allocated attention, preventing something-for-nothing gains (which Eric Baum argued persuasively in [13] lead to cognitively damaging credit-assignment pathologies)
2. **Goal hierarchy:** Higher-level goals still control resource flow to sub-goals, but through attention weights rather than promises
3. **Competition and cooperation:** Options compete via softmax selection while joint requirements are handled through attention-weighted combination
4. **Adaptive allocation:** The decay rate γ and temperature τ provide similar dynamics to RFS withdrawal and redemption timing

In the specific context of SubRep, where options are already rigorously screened by CDS/PDS and motives are decomposed by the MDN, this streamlined approach provides the same economic allocation benefits with significantly reduced implementation complexity and computational overhead.

10.2 Simplified Conservative Attention for Subgoal Selection

Core Principle: Conservation Through Bounded Pools To elaborate the new approach we suggest more fully: Instead of the two-stage promise/redemption mechanism, we use a single-stage conserved attention pool:

Definition 10.1 (Conservative Attention Pool). *At each time step, the system maintains:*

- Total attention budget $A_{\text{total}} = 1.0$ (normalized)
- Active allocation $a : \text{Options} \rightarrow [0, 1]$ with $\sum_o a(o) \leq A_{\text{total}}$
- Decay rate $\gamma \in (0, 1)$ preventing indefinite accumulation

10.2.1 Unified Allocation Mechanism

We then perform a single allocation step:

Algorithm 1 Conservative Attention Allocation

```

1: Input: Admitted options  $\mathcal{B}$  from CDS/PDS, MDN weights  $W$ 
2: Initialize:  $a(o) \leftarrow a_{\text{prev}}(o) \cdot \gamma$  for all  $o$  (decay)
3: Compute priorities:
4: for each option  $o \in \mathcal{B}$  do
5:    $p(o) = \frac{m_o(x; W) \cdot \text{feasibility}(o)}{\text{cost}(o) + \epsilon}$ 
6: end for
7: Redistribute unallocated attention:
8:  $A_{\text{free}} = A_{\text{total}} - \sum_o a(o)$ 
9: Allocate via softmax with temperature:
10: for each  $o$  with  $p(o) > 0$  do
11:    $\Delta a(o) = A_{\text{free}} \cdot \frac{\exp(p(o)/\tau)}{\sum_{o'} \exp(p(o')/\tau)}$ 
12:    $a(o) \leftarrow a(o) + \Delta a(o)$ 
13: end for
14: Execute: Run options proportional to  $a(o)$ 
15: Update: Successful options keep allocation, failed ones decay faster

```

Key properties:

- Conservation: $\sum_o a(o) = A_{\text{total}}$ always
- No credit leaks: Attention only flows through explicit redistribution
- Integration with CDS/PDS: Uses margins $m_o(x; W)$ directly

10.2.2 Hierarchical Conservation

For multi-level subgoals, maintain conservation at each level:

$$A_{\text{total}}^{(\ell)} = \sum_{o \in \text{Level } \ell} a^{(\ell)}(o) = 1.0 \quad (38)$$

$$a^{(\ell+1)}(o') = a^{(\ell)}(o) \cdot \frac{m_{o'}(x; W^{(\ell+1)})}{\sum_{o''} m_{o''}(x; W^{(\ell+1)})} \quad (39)$$

This ensures no level can create attention from nothing.

10.2.3 Credit Assignment Without Leaks

The Baum insight is preserved through:

Proposition 10.2 (No Free Lunch). *Under conservative allocation, the total reward collected satisfies:*

$$\sum_t r_t \leq \sum_t \sum_o a_t(o) \cdot r_{\max}(o)$$

where $r_{\max}(o)$ is the maximum reward option o can generate. No option can claim credit without consuming attention.

10.2.4 Integration with MDN

The MDN influences allocation through the margin terms:

$$p(o) = \inf_{w \in W_{\text{MDN}}} [B_o(x; w) - B_{\text{base}}(x; w)] \cdot \frac{1}{\text{cost}(o)} \quad (40)$$

$$a(o) \propto \exp(p(o)/\tau) \quad (41)$$

As the MDN tightens W , options with robust improvements get more attention.

10.2.5 Simplified Multi-Modal Handling

Instead of separate stores (CDS/PDS/MDS), use attention-weighted fusion:

$$h_{\text{fused}} = a_{\text{text}} \cdot h_{\text{text}} + a_{\text{audio}} \cdot h_{\text{audio}} + a_{\text{vision}} \cdot h_{\text{vision}} \quad (42)$$

$$\text{where } a_{\text{text}} + a_{\text{audio}} + a_{\text{vision}} = 1.0 \quad (43)$$

Modalities compete for attention rather than requiring complex routing.

10.2.6 Preventing Pathologies

Conservative attention has potential to prevent or palliate numerous attentional pathologies, e.g.:

1. **Hallucination:** Can't attend to non-existent features (would violate conservation)
2. **Attention spam:** Can't create unbounded attention to everything
3. **Credit stealing:** Options must consume attention to claim reward

Of course all these cognitive-level phenomena can occur in many different ways and mathematical conservation of internal quantities doesn't utterly avoid them, but it seems a valuable principle to maintain.

10.2.7 Practical Implementation

One way to cash out this idea in pythonic implementations would be

```
class ConservativeAttention:
    def __init__(self, total_budget=1.0, decay=0.95):
        self.budget = total_budget
        self.decay = decay
        self.allocation = {}

    def update(self, admitted_options, mdn_weights):
        # Decay existing allocations
        for o in self.allocation:
            self.allocation[o] *= self.decay

        # Compute priorities using CDS/PDS margins
        priorities = {}
        for o in admitted_options:
            margin = compute_margin(o, mdn_weights)
            cost = compute_cost(o)
            priorities[o] = margin / (cost + 0.01)

        # Redistribute free attention
        used = sum(self.allocation.values())
        free = self.budget - used
```

```

# Softmax allocation
if priorities:
    exp_priorities = {o: exp(p/self.temperature)
                       for o, p in priorities.items()}
    total_exp = sum(exp_priorities.values())

    for o, exp_p in exp_priorities.items():
        self.allocation[o] = self.allocation.get(o, 0) + \
                               free * exp_p / total_exp

return self.allocation

def execute(self, state):
    # Run options proportional to allocation
    results = []
    for option, attention in self.allocation.items():
        if attention > 0.01: # Threshold
            result = option.execute(state, strength=attention)
            results.append((result, attention))
    return weighted_combine(results)

```

10.2.8 Advantages Over Original RFS

1. **Simpler:** One-stage allocation vs promise/redemption
2. **Direct:** Uses CDS/PDS margins directly
3. **Conservative:** Maintains Baum/ECAN properties
4. **Efficient:** $O(n)$ allocation vs complex graph dynamics
5. **Interpretable:** Attention values directly show resource usage

10.3 Theoretical Properties of Conservative Attention

Theorem 10.3 (Margin-Weighted Optimality). *The conservative attention allocation with softmax temperature $\tau \rightarrow 0$ maximizes the expected margin-weighted reward:*

$$a^* = \arg \max_{a \in \Delta_A} \sum_o a(o) \cdot m_o(x; W) \cdot \mathbb{P}[\text{execute } o | a(o)]$$

subject to $\sum_o a(o) = A_{total}$, where Δ_A is the probability simplex and $m_o(x; W)$ is the CDS margin.

Proof. As $\tau \rightarrow 0$, the softmax allocation converges to the argmax. With execution probability proportional to allocation, the objective becomes $\sum_o a(o)^2 m_o(x; W)$. The KKT conditions under the conservation constraint yield the softmax form as $\tau \rightarrow 0^+$. $\square$

Theorem 10.4 (Weakness-Attention Duality). *For options with weakness functional $w(o)$ and margins $m_o(x; W)$, the equilibrium attention allocation satisfies:*

$$\frac{a(o_1)}{a(o_2)} = \frac{m_{o_1}(x; W)/w(o_1)}{m_{o_2}(x; W)/w(o_2)} \cdot e^{-\lambda(w(o_1) - w(o_2))}$$

where $\lambda > 0$ is the weakness penalty coefficient. Thus stronger options receive exponentially less attention relative to their margin advantage.

Proof. Including weakness in the allocation priority gives $p(o) = m_o(x; W)/(w(o) + \epsilon)$. The softmax with weakness-adjusted priorities yields the stated ratio. The exponential term emerges from the temperature-scaled difference in log-priorities. $\square$

Theorem 10.5 (Conservation Under Hierarchical Decomposition). *In a multi-level hierarchy with conservative attention at each level ℓ , if level ℓ allocates $a^{(\ell)}(o)$ to option o which decomposes into sub-options at level $\ell + 1$, then:*

$$\sum_{o' \in \text{children}(o)} a^{(\ell+1)}(o') = a^{(\ell)}(o)$$

and the total system attention satisfies $\sum_{\text{leaf } o} a_{\text{effective}}(o) = A_{total}$.

Proof. By construction, each level preserves conservation locally. The tree structure ensures that attention flows without creation or destruction, only redistribution at each branching point. $\square$

Theorem 10.6 (Convergence to Margin-Proportional Steady State). *Under constant margins $m_o(x; W)$ and decay rate $\gamma \in (0, 1)$, the conservative attention dynamics converge to a unique fixed point:*

$$a_o^* = \frac{(1 - \gamma) \cdot \exp(m_o(x; W)/\tau)}{\sum_{o'} \exp(m_{o'}(x; W)/\tau)}$$

with convergence rate $O(\gamma^t)$.

Proof. The update equation $a_t(o) = \gamma a_{t-1}(o) + (1 - \gamma) \cdot \text{softmax}(m_o/\tau)$ is a contraction mapping with coefficient γ . The Banach fixed-point theorem guarantees unique convergence. $\square$

Theorem 10.7 (Pareto Efficiency Under PDS). *When using PDS with reference weights $W_{ref} = \{w^{(k)}\}$, the conservative attention allocation is Pareto efficient: there exists no alternative allocation a' such that:*

$$\sum_o a'(o) \cdot B_o(x; w^{(k)}) \geq \sum_o a(o) \cdot B_o(x; w^{(k)}) \quad \forall k$$

with strict inequality for some k , while maintaining $\sum_o a'(o) = A_{total}$.

Proof. The softmax allocation based on Pareto gaps maximizes a log-sum-exp approximation to the max over components. Any reallocation that improves one component must reduce another due to conservation, and the exponential weighting ensures local Pareto optimality. $\square$

Theorem 10.8 (Regret Bound). *Let o^* be the option with maximum margin $m^* = \max_o m_o(x; W)$. The expected regret of conservative attention allocation versus always selecting o^* is bounded by:*

$$\mathbb{E}[\text{Regret}_T] \leq \frac{\tau \log |O|}{1 - \gamma} + \frac{\gamma}{(1 - \gamma)^2} \cdot \max_o |m_o(x; W)|$$

where T is the time horizon and $|O|$ is the number of options.

Proof. The first term comes from softmax exploration overhead; the second from decay-induced lag in adaptation. The bound is tight as $\tau \rightarrow 0$ and $\gamma \rightarrow 0$. $\square$

Corollary 10.9 (Anytime Optimality). *By choosing $\tau(t) = c/\sqrt{t}$ and $\gamma(t) = 1 - 1/t$, the conservative attention mechanism achieves $O(\sqrt{T})$ regret, matching the optimal rate for online learning with bandit feedback.*

10.3.1 Implications

These theorems establish that conservative attention:

- Converges to margin-optimal allocation in the low-temperature limit
- Naturally balances strength and weakness through the duality principle
- Preserves conservation exactly through hierarchical decomposition

- Achieves Pareto efficiency for multi-objective settings
- Attains optimal regret bounds with appropriate parameter schedules

This shows that the framework presented is not just a crude simplification of RFS, but rather a mechanism with provable optimality properties that align perfectly with the CDS/PDS admission criteria and weakness-driven learning principles.

11 Implementing CDS/PDS with a Co-Learned MDN in a Waveformer Driven by Conservative Attention

We show how to embed certification-based subgoal selection (CDS/PDS) and a co-learned Motive Decomposition Network (MDN) into a Transformer-style architecture, using a Waveformer backbone as the data-plane and *conservative attention* as the execution-time control-plane. The result is a subgoal-aware transformer that admits options by certificate and allocates runtime via a conserved attention pool, with a direct mapping to MORK for use in Hyperon/PRIMUS.

11.1 Transformers and Waveformers: a short primer

Transformers (recap). A transformer layer maintains a residual stream $h \in \mathbb{R}^{T \times d}$ of T tokens. A multi-head self-attention block computes queries Q , keys K , and values V , and mixes V according to attention weights $\text{softmax}(QK^\top / \sqrt{d_h})$. A position-wise MLP then updates each token. Residual connections and normalization stabilize training. Autoregressive decoders use causal masks; encoders and cross-attention relax masks accordingly.

Waveformers (execution style). A Waveformer keeps the transformer primitives but organizes computation along hierarchical *paths* (local windows, dilations, chunked streams) so that token groups correspond to spatial or temporal neighborhoods. The trunk exposes path tokens and a set of context tokens for downstream heads. Multi-resolution, windowed attention reduces latency and cost, and enables streaming inference with bounded KV caches.³

³See the multi-scale attention with dilated and local windows and path tokens that encode spatial or temporal neighborhoods.

11.2 From motives to options inside a Waveformer

We integrate subgoal decomposition at three places: (i) an MDN head that emits a motive geometry used for certification, (ii) option heads that expose expectation-model summaries, and (iii) CDS/PDS gates that admit only certified options. Execution-time selection then uses conservative attention.

11.2.1 Unified notation and preliminaries

Let $x \in \mathcal{X}$ denote features at the current context; o a candidate option; and B the current admitted library. Each option o exports an expectation-model summary

$$(\hat{r}(x, o), \hat{n}(x, o)) \quad \text{with} \quad \hat{n}(x, o) \in \mathbb{R}^d,$$

where d is the dimension of motive weights $w \in \mathbb{R}^d$. For any w , the backed-up value of o is

$$B_o(x; w) = \hat{r}(x, o) + w^\top \hat{n}(x, o),$$

and the baseline is

$$B_{\text{base}}(x; w) = \max_{b \in B} \{\hat{r}(x, b) + w^\top \hat{n}(x, b)\}.$$

We write $B_o^{(\ell)}(x; w)$ when restricting to level ℓ in a stack.

MDN outputs (normalized). The MDN at level ℓ emits either (a) a normalized slice $\mathcal{W}^{(\ell)}$ (for CDS) or (b) a finite reference set $W_{\text{ref}}^{(\ell)} = \{w_{[k]}^{(\ell)}\}_{k=1}^K$ (for PDS). When a cone is specified by extreme rays $R^{(\ell)}$, we treat $R^{(\ell)}$ solely as a parameterization of the normalized slice $\mathcal{W}^{(\ell)}$ to avoid dual notations.

CDS margin. Given a normalized slice $\mathcal{W}$, define

$$m_o(x; \mathcal{W}) = \Delta r(x, o) + \inf_{w \in \mathcal{W}} w^\top \Delta n(x, o), \quad \Delta r = \hat{r}(x, o) - \hat{r}_{\text{base}}(x), \quad \Delta n = \hat{n}(x, o) - \hat{n}_{\text{base}}(x).$$

PDS gaps (vectorized). For $W_{\text{ref}} = \{w_{[k]}\}_{k=1}^K$,

$$\Delta B_{o, [k]}(x) = B_o(x; w_{[k]}) - B_{\text{base}}(x; w_{[k]}), \quad k = 1, \dots, K.$$

11.2.2 Placing MDN, option heads, and gates in a transformer

Let $\phi(x)$ be features produced by the Waveformer trunk. We use simple heads:

$$\hat{r}(x, o) = r_o^\top \phi(x) + b_o, \quad \hat{n}(x, o) = U_o D_o (V^\top \phi(x)),$$

with V shared across options for amortized compute and transfer, and $U_o D_o$ option-specific and low-rank. The MDN is a small head that emits either a slice $\mathcal{W}$ (e.g., box bounds or small ray sets) or a cover W_{ref} . The CDS/PDS gate is a tiny module that takes $(\hat{r}, \hat{n})$, $\mathcal{W}$ or W_{ref} , and the current B_{base} , and returns certification margins or gaps along with binary admits.

11.3 Execution-time control: conservative attention in the loop

Let $A_{\text{total}} = 1.0$, and let $a(o) \in [0, 1]$ with $\sum_o a(o) \leq A_{\text{total}}$. At each step: decay a , compute priorities from CDS/PDS margins and costs, redistribute free attention by a softmax, and execute proportional to a . This integrates directly because the MDN margins $m_o(x; \mathcal{W})$ or the PDS gaps are already computed by the gate.

Algorithm A1a: Certify (per window or expert group)

1. Features: $x \leftarrow \phi(\text{window})$. The MDN emits either $\mathcal{W}$ (CDS) or W_{ref} (PDS).
2. Compute $(\hat{r}, \hat{n})$ for candidates $o \in \mathcal{O}_{\text{cand}}$.
3. *CDS*: for each o , estimate $m_o(x; \mathcal{W})$ with Alg. A2 (tiered support function).
4. *PDS*: for each o , compute $\{\Delta B_{o,[k]}(x)\}_{k=1}^K$ by one fused matvec.

Algorithm A1b: Admit and prune (library maintenance)

1. *CDS gate*: admit o if $\mathbb{E}[m_o] \geq \varepsilon$ or $\Pr[m_o \geq 0] \geq 1 - \delta$.
2. *PDS gate*: admit if $\Delta B_{o,[k]} \geq -\rho$ for all k , with at least qK components $\geq \varepsilon$.
3. Maintain the ε -Pareto cover via Alg. A3 (refine and compress W_{ref} as needed).

4. Apply weakness tie-break only among feasible o (then top- k or threshold).
5. Update baselines and caches (see the baseline alternatives subsection).

Algorithm A1c: Allocate (conservative attention; execution-time)

1. Decay budgets: $a(o) \leftarrow \gamma a(o)$.
2. Priorities: $p(o) \leftarrow m_o(x; \mathcal{W}) / (\text{cost}(o) + \epsilon)$ for feasible o .
3. Redistribute free pool A_{free} with softmax over $p(\cdot)$ and execute proportional to $a(\cdot)$.
4. Hierarchy: pass $a^{(\ell)}$ downward multiplicatively with per-level conservation.

11.4 Computing m_o without hiding cost (CDS support function)

Algorithm A2: Anytime support-function evaluation. Inputs: Δr , Δn , normalized slice $\mathcal{W}$, cached active rays S , tolerances $(\kappa, \varepsilon_{\text{acc}})$.

1. **Tier 1 (cached rays).** Evaluate $\Delta r + \min_{w \in S} w^\top \Delta n$. Accept or reject if outside a band $\pm \kappa$.
2. **Tier 2 (exact face solve).** For polyhedral $\mathcal{W}$, solve $\min \Delta n^\top w$ s.t. $Aw \leq b$, $Cw = c$, $\alpha^\top w = 1$; for boxes, closed form; for ellipsoids/-SOCP, one cone solve. Cache w^* .
3. **Tier 3 (projected subgradient).** If borderline, run T steps $w \leftarrow \Pi_{\mathcal{W}}(w - \eta \Delta n)$; keep the best.

11.5 Maintaining an ε -Pareto cover (PDS)

Algorithm A3: Dynamic ε -Pareto maintenance. Maintain W_{ref} as an ε -net of the normalized base under a chosen norm with dual $\|\cdot\|_*$:

1. Initialize from MDN anchors or k -center points.
2. On violation, when $\min_{w \in W_{\text{ref}}} \Delta r + w^\top \Delta n$ is beaten by $> \varepsilon \|\Delta n\|_*$ at optimizer w^* , insert w^* .
3. Periodically compress by greedy max-min or removal of Pareto-redundant points.

11.6 Weakness regularization and origin of the λ 's

We use two penalties after feasibility:

$$\text{score}(o) = \mathbb{E}[m_o] - \lambda_{\text{beh}}\psi_{\text{beh}}(o) - \lambda_{\text{struct}}\psi_{\text{struct}}(o) - \lambda_{\text{off}}\psi_{\text{off}}(o).$$

Behavioral weakness penalizes *surplus* beyond the admission threshold (do not oversolve). Structural weakness penalizes interface/complexity via a polynomial functional $w(p_o)$ (e.g., SF rank, guard size, parameter norms). Each λ is a dual variable updated to match a soft budget:

$$\lambda \leftarrow [\lambda + \eta(\text{measured_cost} - \text{target_budget})]_+.$$

Among feasible candidates, either pick top- k by lowest $\text{score}(o)$ under compute budgets, or threshold $\text{score}(o) \leq \theta$ and let conservative attention allocate runtime.

11.7 Semantics vs implementation: deconfounding the layers

Layer	Conceptual role	Implementation view
Motives	what we care about	MDN slice $\mathcal{W}$ or cover W_{ref}
Sub-motives	finer motive axes	MDN at level 2 or 3
Options	decision experts	NN/LOG/EVO heads exposing $(\hat{r}, \hat{n})$
Primitives	low-level skills	guarded macros, micro-controllers
Value decomposition	planner eval	$B_o(x; w)$ on slices or covers
Behavior decomposition	how to act	conservative attention execution

11.8 How A1-A3 Improve Transformer/Waveformer Attention

Algorithms A1a-A1c (certify; admit/prune; allocate), A2 (anytime CDS support-function), and A3 (dynamic PDS cover) provide a clean *control-plane* above standard dot-product attention. They deliver: (i) a per-step, conserved attention budget over admissible items; (ii) a principled way to decide who is eligible for attention (CDS/PDS certificates); and (iii) a cheap, stable way to keep those eligibility tests tight as learning proceeds.

Hook points in a Waveformer. Let the Waveformer trunk produce shared features $\phi(x)$ and path/window tokens. Attach three small modules: (i) *Option heads* that output $(\hat{r}(x, o), \hat{n}(x, o))$ for routed experts/windows; (ii) an *MDN* head that emits a slice $\mathcal{W}$ (CDS) or a finite cover W_{ref} (PDS);

(iii) a *Gate* that computes margins/gaps versus the current baseline. A1c then allocates a conserved budget $a(o) \in [0, 1]$ over admitted items; items with $a(o) \approx 0$ simply do not participate in the heavy attention math this step.

Budget-weighted attention (data-plane). For any attention mix over eligible keys/values j , bias logits by the conservative budget:

$$\alpha_{ij} = \text{softmax}_j \left(\frac{Q_i K_j^\top}{\sqrt{d}} + \log(a_j + \varepsilon) \right),$$

so items with negligible budget cannot dominate even if $Q_i K_j^\top$ is spuriously large. Here, a_j is the allocation associated to the routed window/expert/key containing j , and $\varepsilon > 0$ is a tiny constant for numerical safety.

Routing, sparsity and KV retention. Use $a(\cdot)$ as a preselector and a postselector: (i) *pre-attention routing*: keep only top- k windows/experts/-tokens by $a(\cdot)$ (or threshold by $a \geq \theta$); (ii) *KV-cache policy*: retain/evict keys/values with probabilities proportional to $a(\cdot)$ (decay with the same γ used in A1c). This ties runtime to certified utility: CDS/PDS first decides feasibility; the budget then enforces how much attention (and memory) that feasibility earns.

Hierarchy and modalities. For multi-level (macro $\rightarrow$ micro) stacks, keep a per-level conserved pool and pass budget downward multiplicatively (A1c’s hierarchical rule). For multimodal trunks, treat $(a_{\text{text}}, a_{\text{audio}}, a_{\text{vision}})$ as a sum-to-one triple and fuse modality features by those weights; the same pool prevents “modality pile-on” and keeps compute bounded.

Why attention gets better. (i) *Bounded compute & stable sparsity*: $\sum a = A_{\text{total}}$ each step, so fan-out cannot explode when context grows. (ii) *Fewer pathologies*: no attention spam or free credit—items must consume budget to influence outputs. (iii) *Sharper credit assignment*: budgets are driven by robust CDS/PDS margins (A2/A3), so what gets runtime is what helps the planner across the current motive slice. (iv) *Lower variance*: A2’s cached rays and A3’s small covers keep the gate cheap and stable; budgets change smoothly (decay γ , temperature τ), so attention does too.

Cost and implementation notes. On each tick, the heavy math remains the usual batch GEMMs for attention and a few fused matvecs for $(\hat{r}, \hat{n})$

and CDS/PDS reductions. The incremental costs are tiny vectors: the $a(\cdot)$ update and a few cached-ray checks (A2); periodic cover tweaks (A3). In practice, this yields a predictable latency budget and clean scaling with the number of routed items.

11.9 Local PC updates versus backpropagation in a transformer

We use a two-timescale scheme that keeps the transformer training story simple while exploiting PC-style locality in the gate and heads.

Scope of PC. Apply PC-style local residual updates only to (i) option heads $(\hat{r}, \hat{n})$ and (ii) the MDN parameters that shape $\mathcal{W}$ or W_{ref} . Train the Waveformer trunk and attention blocks with standard backprop and Adam.

Scheduling. At each micro tick:

- Run A1a–A1c; form residuals $e_r = r_{\text{target}} - \hat{r}$, $e_n = n_{\text{target}} - \hat{n}$.
- Apply one or a few local PC-style parameter nudges to heads/MDN using (e_r, e_n) with small stepsizes (no BPTT).

Every M micro ticks (macro step):

- Freeze the gate; accumulate a supervised or self-supervised loss on the trunk (e.g., next-token or masked target).
- Run standard backprop on the trunk (and optionally on shallow head adapters).

Soft gating surrogate. During macro backprop, use the conservative-attention allocation $a(o)$ as a soft proxy for hard admit decisions, avoiding discontinuities.

11.10 Baseline maintenance: alternatives when $O(1)$ is unrealistic

When access patterns do not support amortized $O(1)$ top-2 caches, use:

Vectorized reduction. Compute $B_{\text{base}}(x; w)$ by a parallel max over $b \in B$:

$$B_{\text{base}}(x; w) = \max_{b \in B} \{\hat{r}(x, b) + w^\top \hat{n}(x, b)\}.$$

With a small cap $|B| \leq N_{\text{max}}$ (e.g., 16–32), this $O(|B|)$ reduction is negligible relative to attention GEMMs.

Last-winner heuristic. Track the last winner $\tilde{b}$ per (window, slice/cover) and compute only $\max\{B_{\tilde{b}}(x; w), \max_{b \in S} B_b(x; w)\}$ over a small rotating subset $S \subset B$; refresh S slowly.

Per-window cap. Cap $|B|$ via the weakness tie-break so the reduction stays tiny and predictable.

11.11 Computational feasibility with concrete budgets

Design for the *no-LP* and *rare-solve* regimes.

No-LP (preferred). Choose $\mathcal{W}$ as a box slice (nonnegative, normalized weights with learnable bounds). Then

$$\inf_{w \in \mathcal{W}} w^\top \Delta n = \sum_{i=1}^d (\ell_i \cdot \min(\Delta n_i, 0) + u_i \cdot \max(\Delta n_i, 0)),$$

a closed-form saturation after sign checks; Alg. A2 Tier 2 reduces to branch-less arithmetic.

Rare-solve (general polyhedral). Keep a small cache of active rays (Tier 1). Trigger one exact face solve (small LP/SOCP) only when $\Delta r + \min_{w \in S} w^\top \Delta n$ lies in a narrow band $[-\kappa, +\kappa]$. Run certification at window/expert granularity, not per token, and on a sub-sampled cadence every g ticks.

Example budget (order-of-magnitude). Sequence $T = 2048$, layers $L = 24$, heads $H = 16$, local window $W = 128$. Options per window $|\mathcal{O}| = 16$ (cap $N_{\text{max}} \leq 32$). Motive dimension $d = 8$ (or 16). PDS cover size $K = 6$ (or 8). Share $V^\top \phi(x)$ once; per-option low-rank $U_o D_o$ with rank $r = 16$. Certification cadence $g = 4$.

Arithmetic cost. Per window, CDS/PDS adds roughly

$$\text{MACS}_{\text{cert}} \approx |\mathcal{O}| \cdot (d + r) + |\mathcal{O}| \cdot K \cdot d,$$

e.g., $16 \cdot (8 + 16) + 16 \cdot 6 \cdot 8 \approx 1152$ MACs, versus attention $QK^\top$ of about $H \cdot W \cdot d_k$, e.g., $16 \cdot 128 \cdot 64 \approx 131,072$ MACs per window. Certification is $< 1\%$ of attention before cadence g .

LP/SOCP bottleneck. In the no-LP box variant: none. In rare-solve mode: solves occur on $< 1\%$ of windows after warm-up and only every g ticks; dimension $d \in [8, 16]$ yields sub-millisecond solves that can overlap with GPU attention. Bound solves per layer per tick by a cap S_{max} .

Memory and KV policy. Budgets $a(\cdot)$ are size $|\mathcal{O}|$ per window. KV retention uses $a(\cdot)$ for keep/evict probabilities; no extra large tensors are needed.

Summary. With box slices, small d and K , capped $|\mathcal{O}|$, window-level cadence, and rare-solve fallback, certification stays comfortably sub-5% of attention FLOPs (typically sub-2%) while conservative attention guarantees bounded compute.

11.12 Robust certificates

The gate should not over-trust $(\hat{r}, \hat{n})$ when models are noisy or the MDN slice is uncertain. Suppose we have calibrated error radii

$$|\hat{r} - r| \leq \varepsilon_r(x, o), \quad \|\hat{n} - n\|_\infty \leq \varepsilon_n(x, o),$$

obtained from held-out residuals, ensembles, dropout, or PC-style uncertainty monitors. Then a safe CDS lower bound replaces the exact margin by

$$\hat{m}_o^{\text{rob}}(x; \mathcal{W}) = \inf_{w \in \mathcal{W}} (\hat{r} + w^\top \hat{n} - B_{\text{base}}) - \varepsilon_\Delta^r(x, o) - \|\mathcal{W}\|_{\text{max},1} \varepsilon_\Delta^n(x, o),$$

where $\|\mathcal{W}\|_{\text{max},1} := \sup_{w \in \mathcal{W}} \|w\|_1$ and the deltas ε_Δ are the *differences* of the option and baseline error bounds (if conservative, just add them). Intuition: worst-case dot-product error is bounded by the dual-norm inequality $|w^\top(\hat{n} - n)| \leq \|w\|_1 \|\hat{n} - n\|_\infty$, hence the appearance of $\|\mathcal{W}\|_{\text{max},1}$. For common normalizations: (i) on the probability simplex, $\|\mathcal{W}\|_{\text{max},1} = 1$; (ii) on a box slice with nonnegative w and $\mathbf{1}^\top w = 1$, it is also 1.

PDS with componentwise slacks. For a finite cover $W_{\text{ref}} = \{w_{[k]}\}$, define robust gaps

$$\Delta B_{o,[k]}^{\text{rob}}(x) = \Delta B_{o,[k]}(x) - \varepsilon_{\Delta}^r(x, o) - \|w_{[k]}\|_1 \varepsilon_{\Delta}^n(x, o),$$

and apply the same budget and coverage tests as before but on $\Delta B_{o,[k]}^{\text{rob}}$. If you track per-dimension bounds, you can switch to the dual pair $\|\cdot\|_2$ and use $\|w_{[k]}\|_2 \varepsilon_{n,2}(x, o)$ instead.

Dispersion-aware inflation. When the MDN slice is dispersed (e.g., large entropy of the cone parameters, wide box bounds, or high sample variance across W_{ref}), inflate the radii

$$\varepsilon_r \leftarrow \varepsilon_r \cdot (1 + \alpha \text{ disp}), \quad \varepsilon_n \leftarrow \varepsilon_n \cdot (1 + \alpha \text{ disp}),$$

with a small α (e.g., 0.3) and a bounded dispersion score in $[0, 1]$. This yields fewer false admissions when motives are uncertain.

Where it plugs in. Robust margins and gaps are a drop-in replacement in Alg. A2 and A3: use $\hat{m}_o^{\text{rob}}$ wherever m_o appears, and $\Delta B_{o,[k]}^{\text{rob}}$ wherever $\Delta B_{o,[k]}$ appears. For priorities in conservative attention, replace m_o by $\hat{m}_o^{\text{rob}}$ in $p(o)$ so the runtime budget follows *certified* utility, not optimistic estimates.

Calibrating the radii. In practice, ε_r and ε_n come from rolling quantiles of residuals on a validation stream, from ensemble variance, or from PC residual energy. A simple and effective choice is the 0.95 quantile multiplied by a safety factor in $[1.2, 2.0]$; decrease the factor as the gate stabilizes.

11.13 Complexity summary

The dominant arithmetic remains the attention and MLP GEMMs; certification adds small, largely vectorized dot-products and reductions.

Per-window costs. With shared V and low-rank UD heads, computing $(\hat{r}, \hat{n})$ for $|\mathcal{O}|$ options is $O(|\mathcal{O}| \cdot (d + r))$ MACs. PDS adds one fused matvec against a $d \times K$ matrix, giving $O(|\mathcal{O}| \cdot K \cdot d)$ MACs. CDS adds a support-function term: closed form and branchless for box slices; otherwise one tiny LP/SOCP only on borderline cases (rare-solve). Baseline evaluation is a max over at most N_{max} admitted items; treat this as a small $O(N_{\text{max}})$ reduction or cache when access patterns are stable.

Scaling across the model. Let sequence length T , layers L , heads H , local window W , options per window $|\mathcal{O}|$, motive dimension d , cover size K , low-rank r , certification cadence g . Then the certification work per layer per tick scales like

$$O\left(\frac{T}{W} \cdot (|\mathcal{O}|(d+r) + |\mathcal{O}|Kd) \cdot \frac{1}{g}\right),$$

plus a small number of rare LP/SOCP solves capped by $S_{\max}$. Attention scales like $O\left(\frac{T}{W} \cdot H \cdot W \cdot d_k\right)$ just for $QK^\top$, so certification remains a few percent of attention when $d, K, |\mathcal{O}|$ are small and $g > 1$.

Typical configuration. As a concrete anchor: $T = 2048$, $L = 24$, $H = 16$, $W = 128$, $|\mathcal{O}| = 16$, $d = 8$ (or 16), $K = 6$ (or 8), $r = 16$, cadence $g = 4$, cap $N_{\max} \leq 32$, and rare-solve with $S_{\max}$ per layer per tick. Under these settings, certification arithmetic is typically $< 2\%$ of attention, with negligible memory overhead (budgets $a(\cdot)$ are length $|\mathcal{O}|$ per window; W_{ref} is $d \times K$).

Where cost can spike and how to avoid it. Costs rise when you certify per token (avoid; certify per window/expert), when K or d is large (keep both small), when $|\mathcal{O}|$ grows unbounded (cap via weakness tie-break), or when cadence $g = 1$ and many LPs trigger (increase g , tighten Tier 1 caching, cap solves by $S_{\max}$, or switch to box slices).

Amortization and overlap. Warm starts and cached rays make A2 effectively constant on streams; rare LP/SOCP solves are small enough to run on CPU overlapped with GPU attention. Vectorized baseline reductions fuse well with head matvecs. Overall, with the recommended budgets, certification adds predictable latency while conservative attention guarantees bounded compute.

11.14 Formal Analysis of Attention Pathologies

Next we seek to formalize the informal claims made above about the advantages of conservative attention for combating attention spam and credit stealing – aiming to move toward rigorous foundations for the conservative attention mechanism’s protective properties.

11.14.1 Definitions

Definition 11.1 (Attention Spam). *An attention mechanism exhibits epsilon-attention spam at time t if:*

$$\frac{|\{o : a_t(o) > 0\}|}{|O|} > 1 - \epsilon \quad \text{and} \quad H(a_t) > \log(|O|) - \epsilon \quad (44)$$

where $H(a_t) = -\sum_o \frac{a_t(o)}{A_{total}} \log \frac{a_t(o)}{A_{total}}$ is the entropy of the attention distribution.

Definition 11.2 (Credit Stealing). *An option o exhibits credit stealing at time t if:*

$$r_t(o) > 0 \quad \text{while} \quad m_o(x_t; W) < 0 \quad (45)$$

where $r_t(o)$ is the reward claimed by option o and $m_o(x_t; W)$ is its CDS margin.

Definition 11.3 (Conservation-Respecting Mechanism). *An attention mechanism is conservation-respecting if:*

1. $\sum_o a_t(o) = A_{total}$ for all t (strict conservation)
2. $a_t(o) > 0 \implies o \in B_t$ where B_t is the admitted set
3. Allocation satisfies: $a_t(o) = f(m_o(x_t; W), a_{t-1}(o))$ for some monotone function f

11.14.2 Protective Properties

Theorem 11.4 (Anti-Spam Property). *Under conservative attention with temperature τ and admission threshold $\epsilon > 0$, the probability of epsilon-attention spam decreases exponentially in $1/\tau$.*

Proof. The conservative attention allocation is:

$$a_t(o) = \gamma a_{t-1}(o) + A_{free} \cdot \frac{\exp(m_o(x_t; W)/\tau)}{\sum_{o'} \exp(m_{o'}(x_t; W)/\tau)} \quad (46)$$

For admitted options, $m_o(x_t; W) \geq \epsilon$. The entropy of the softmax distribution is bounded by:

$$H(\text{softmax}(m/\tau)) \leq \log(|B_t|) - \frac{\text{Var}(m)}{2\tau^2} + O(1/\tau^3) \quad (47)$$

As $\tau \rightarrow 0$, the distribution concentrates on options with maximal margins. The fraction of options receiving non-negligible attention is bounded by:

$$\frac{|\{o : a_t(o) > \delta\}|}{|O|} \leq \frac{|B_t|}{|O|} \cdot \exp\left(-\frac{(\max_o m_o - \epsilon)}{2\tau}\right) \quad (48)$$

Therefore, for sufficiently small τ , attention concentrates on a small subset, preventing spam. $\square$

Theorem 11.5 (Credit Assignment Integrity). *Under conservative attention, if an option o satisfies $\int_0^T a_t(o)dt > 0$, then $\int_0^T m_o(x_t; W)dt \geq 0$.*

Proof. By the update rule, $a_t(o) > 0$ requires either:

1. $a_{t-1}(o) > 0$ (inherited attention)
2. $m_o(x_t; W) > -\infty$ and $o \in B_t$ (new allocation)

For case 2, admission requires $\mathbb{E}[m_o(x; W)] \geq \epsilon > 0$. By the conservation property:

$$\frac{d}{dt}a_t(o) = -\lambda a_t(o) + \beta \cdot \mathbf{1}_{m_o > 0} \cdot g(m_o) \quad (49)$$

where $\lambda = 1 - \gamma$ and g is the softmax allocation. Integrating:

$$a_T(o) = e^{-\lambda T} a_0(o) + \beta \int_0^T e^{-\lambda(T-s)} \mathbf{1}_{m_o(x_s) > 0} g(m_o(x_s)) ds \quad (50)$$

If $a_T(o) > 0$ and $a_0(o) = 0$, then necessarily $\int_0^T \mathbf{1}_{m_o > 0} g(m_o) ds > 0$, which implies $\int_0^T m_o(x_t; W)dt > 0$. $\square$

Theorem 11.6 (No Free Credit). *Under conservative attention, the total reward claimed satisfies:*

$$\sum_{t=1}^T \sum_o r_t(o) \leq A_{total} \cdot T \cdot \max_o \sup_t \frac{r_t(o)}{a_t(o)} \quad (51)$$

Proof. Since reward claiming requires attention consumption:

$$r_t(o) \leq r_{max}(o) \cdot a_t(o) \quad (52)$$

Summing over options and time:

$$\sum_{t=1}^T \sum_o r_t(o) \leq \sum_{t=1}^T \sum_o r_{max}(o) \cdot a_t(o) = \sum_{t=1}^T r_{max} \sum_o a_t(o) = A_{total} \cdot T \cdot r_{max} \quad (53)$$

This provides a hard upper bound on total claimed reward, preventing unbounded credit creation. $\square$

11.15 Theoretical Analysis: Conservative Attention and Transformer Training

We move on analyze how the conservative attention mechanism interacts with standard transformer training dynamics, establishing conditions under which the gates enhance rather than hinder learning.

11.15.1 Gradient Flow Preservation

Theorem 11.7 (Gradient Flow Through Conservative Attention). *Let L be the training loss and θ be transformer parameters. The conservative attention mechanism preserves gradient flow if:*

$$\left\| \frac{\partial L}{\partial \theta} \right\|_2 \leq C \left\| \frac{\partial L^{std}}{\partial \theta} \right\|_2 \quad (54)$$

for some constant C independent of depth.

Proof. The conservative attention modifies standard attention as:

$$\alpha_{ij} = \text{softmax}_j \left(\frac{Q_i K_j^T}{\sqrt{d}} + \log(a_j + \epsilon) \right) \quad (55)$$

The gradient through this mechanism is:

$$\frac{\partial L}{\partial Q_i} = \sum_j \frac{\partial L}{\partial \alpha_{ij}} \cdot \frac{\partial \alpha_{ij}}{\partial Q_i} \quad (56)$$

where:

$$\frac{\partial \alpha_{ij}}{\partial Q_i} = \frac{1}{\sqrt{d}} \alpha_{ij} (K_j - \sum_k \alpha_{ik} K_k) \quad (57)$$

The additional $\log(a_j)$ term only affects the softmax distribution, not the gradient structure. Since $a_j \in [0, 1]$ and $\sum_j a_j = 1$:

$$\|\log(a_j + \epsilon)\|_\infty \leq \log(1/\epsilon) \quad (58)$$

This bounds the perturbation to the attention matrix:

$$\|\alpha^{cons} - \alpha^{std}\|_F \leq \frac{\sqrt{n} \log(1/\epsilon)}{\tau_{softmax}} \quad (59)$$

Therefore, gradient norms remain bounded with:

$$C = 1 + \frac{\sqrt{n} \log(1/\epsilon)}{\tau_{softmax}} \quad (60)$$

This constant does not grow with depth, preserving gradient flow. $\square$

11.15.2 Training Stability

Theorem 11.8 (Lyapunov Stability of Joint Training). *The coupled system of transformer parameters θ and attention allocations a is Lyapunov stable if:*

1. Learning rate $\eta_\theta < \frac{2}{\lambda_{\max}(H)}$ where H is the Hessian
2. Decay rate $\gamma \in (0.9, 1)$
3. Temperature $\tau > \tau_{\min} = \sqrt{\text{Var}(m_o)}$

Proof. Define the Lyapunov function:

$$V(\theta, a) = L(\theta) + \lambda \sum_o (a_o - a_o^*)^2 \quad (61)$$

where a_o^* is the optimal allocation given margins. The time derivative:

$$\dot{V} = \nabla_\theta L \cdot \dot{\theta} + 2\lambda \sum_o (a_o - a_o^*) \dot{a}_o \quad (62)$$

With gradient descent $\dot{\theta} = -\eta_\theta \nabla_\theta L$:

$$\dot{V} = -\eta_\theta \|\nabla_\theta L\|^2 + 2\lambda \sum_o (a_o - a_o^*) \dot{a}_o \quad (63)$$

The attention dynamics:

$$\dot{a}_o = -(1 - \gamma)a_o + (1 - \gamma) \cdot \text{softmax}(m_o/\tau) \quad (64)$$

At equilibrium, $a_o^* = \text{softmax}(m_o/\tau)$. Near equilibrium:

$$\dot{a}_o \approx -(1 - \gamma)(a_o - a_o^*) \quad (65)$$

Therefore:

$$\dot{V} \leq -\eta_\theta \|\nabla_\theta L\|^2 - 2\lambda(1 - \gamma) \sum_o (a_o - a_o^*)^2 < 0 \quad (66)$$

This proves Lyapunov stability under the stated conditions. □

11.15.3 Learning Enhancement Conditions

Theorem 11.9 (CDS/PDS Gates Improve Sample Efficiency). *The CD-S/PDS gating mechanism improves sample efficiency if:*

$$\frac{\mathbb{E}[m_o|o \in B_t]}{\mathbb{E}[m_o|o \in O]} > \frac{1 + \beta\sigma_{\text{explore}}^2}{1 + \beta\sigma_{\text{exploit}}^2} \quad (67)$$

where $\sigma_{\text{explore}}^2$ is variance under random exploration and $\sigma_{\text{exploit}}^2$ under gated selection.

Proof. With gating, the gradient estimator becomes:

$$\nabla_{\theta} L_{\text{gated}} = \mathbb{E}_{o \sim p(o|a)} [\nabla_{\theta} \log \pi_{\theta}(o|x) \cdot R(o)] \quad (68)$$

where $p(o|a) \propto a_o \cdot \mathbf{1}_{o \in B}$.

The variance reduction ratio:

$$\frac{\text{Var}(\nabla L_{\text{gated}})}{\text{Var}(\nabla L_{\text{ungated}})} \approx \frac{\text{Var}(R|o \in B)}{\text{Var}(R|o \in O)} \cdot \frac{|B|}{|O|} \quad (69)$$

Learning improves when this ratio < 1 , which occurs when margin-based filtering sufficiently reduces variance despite the smaller option set. $\square$

Corollary 11.10 (Practical Benefit Conditions). *The CDS/PDS gates improve transformer training when:*

1. *High noise regime:* $\sigma_{\text{gradient}}^2 / \|\nabla L\|^2 > 10$
2. *Redundant options:* $|O|/|B_{\text{effective}}| > 5$
3. *Clear margin separation:* $\text{gap}(m_{\text{good}}, m_{\text{bad}}) > 2\tau$
4. *Moderate decay:* $\gamma \in [0.95, 0.99]$

11.16 MORK: why and how to map the design

Finally, moving from theory toward implementation, we now explore how this sort of enhanced transformer architecture might be efficiently implemented using the MORK path-database structure that underlies (one very useful implementation of) the Hyperon Atomspace. This might seem like a bit of premature optimization, as we have not – at time of writing – done practical experiments with less scalable implementations of most of the ideas

discussed here regarding SubRep. However, we know that scale is important for adequate functionality of many AI algorithms, and so with a view toward moving rapidly toward effective AGI systems, we believe it makes sense to start exploring scalable implementation strategies very close to the time of conception of new AI algorithms. In any case that is the course we are choosing here!

MORK is a multiresolution DAG runtime with typed nodes and ports, ring buffers and KV stores for caches, and a two-rate scheduler: a fast *micro* loop (PC inference, gating) and a slower *macro* loop (MDN updates, EA/EDA, guard compilation). Messages are batched tensors with precision scalars for predictive coding. This matches the needs of Waveformer blocks, MDN heads, CDS/PDS gates, and conservative attention.

Mapping decisions (distributed). Pin the feature trunk and the level- ℓ MDN/gates to a *device group* (a set of accelerators) associated with a Waveformer path or window shard; co-locate option heads with their path shards to reuse KV caches. Maintain per-shard admitted sets via double-buffered staging: compute certificates into a staging buffer and advance a versioned pointer at shard boundaries. Communicate only small typed ports (indices, margins, statistics), avoiding cross-shard tensor moves in the gate.

Operator fusion and caches. Fuse backed-up values and CDS/PDS reductions into batched GEMMs at each tick. Keep baselines in top-2 caches per (x, w) . For PDS, stack K rays of W_{ref} as a $d \times K$ matrix for a single GEMM across options. Quantize archived $(\hat{n}, \hat{r})$ to cut bandwidth; accumulate in FP32.

11.16.1 Some Ways MORK Helps

The robust certificates and complexity bounds above are backend-agnostic. However, MORK offers plumbing that makes the same goals easier to achieve without bespoke engineering. The points below map directly onto A1a-A1c, A2, and A3.

Two-rate scheduling (micro vs macro). MORK exposes a fast *micro* loop and a slower *macro* loop. Place A1a (certify) and A1b (admit/prune) in the micro loop each tick, and A1c (allocate) every tick; run A3 (cover compression), MDN refresh, and dual-variable updates for λ in the macro loop every M ticks. This mirrors the intended cadence g without manual thread orchestration.

Overlap and caps for rare solves. Algorithm A2 triggers small LP/-SOCP solves only on borderline cases. In MORK, queue these solves to a CPU worker pool while the GPU executes attention and matvecs. Enforce a hard cap $S_{\max}$ on outstanding solves per layer per tick; when the queue reaches $S_{\max}$, skip or approximate (e.g., keep Tier 1 value). Backpressure and queue metrics come built-in, so overlap is automatic.

Versioned double buffers (race-free commits). Gate decisions and admitted sets change at macro boundaries. MORK provides per-shard, versioned double buffers: read from buffer R during a tick while writing updates into buffer W ; at the shard barrier, atomically advance a version pointer $R \leftarrow W$. This prevents partial updates from leaking into A1a/A1c within a tick and removes the need for ad-hoc locks.

Device-group placement (co-location). Place the Waveformer window/path shard, its option heads $(\hat{r}, \hat{n})$, the MDN slice or cover $(\mathcal{W}, W_{\text{ref}})$, and the gate on the *same device group*. Keep W_{ref} and the ray cache local to that group. This reduces cross-device traffic for baseline max and support-function evaluations and makes A1a/A2 latency predictable. MORK exposes device groups as first-class, so placement is declarative.

Operator fusion (small, fused GEMMs). MORK’s typed ports make it straightforward to fuse: (i) the low-rank head projection $V^\top \phi(x)$ once per shard, (ii) per-option $U_o D_o$ matvecs, (iii) the PDS cover matvec $\mathbb{R}^d \times \mathbb{R}^{d \times K}$, and (iv) the baseline reduction into one or two compact kernels. This keeps certification compute as vectorized dot-products that schedule alongside attention GEMMs.

Telemetry and adaptive knobs. MORK streams counters for residual quantiles (to calibrate $\varepsilon_r, \varepsilon_n$), cache-hit rate for A2 Tier 1, solve rate and queue depth (to tune κ , $S_{\max}$, and cadence g), dispersion of $\mathcal{W}$ or W_{ref} (to inflate radii), and admitted-set size (to regulate $N_{\max}$ via weakness tie-break). These signals let the macro loop adapt budgets and temperatures (γ, τ) without instrumenting external probes.

Bounded compute as a first-class resource. Conservative attention already enforces $\sum_o a(o) = A_{\text{total}}$. In MORK, A_{total} can be tied to per-shard quotas and scheduler time slices, so the runtime respects both the mathematical budget and the hardware budget. If a shard experiences pressure,

the scheduler lowers A_{total} locally and A1c redistributes within the smaller pool, preserving guarantees.

Take-away. None of these capabilities change the math of A1a–A1c, A2, or A3. They simply provide the scheduling, placement, fusion, and observability primitives that make the rare-solve, window-level, conserved-budget regime practical at scale, with predictable latency and clean failure modes.

11.17 A conservative-attention-first MVP and upgrades

The tweaked transformer NNs we have described here are quite complex animals with many moving parts. To actually implement such things one will want to add the complexity in step by step, via some route such as:

MVP-0: Minimal CDS + conservative attention. MDN emits a simple box cone $B(W) = \{w \geq 0, \mathbf{1}^\top w = 1, \ell \leq w \leq u\}$ with learnable bounds (ℓ, u) . Gate uses Alg. A2 (Tier 1 cached rays; Tier 2 if needed). Execution uses the conserved pool with $\text{cost}(o)$ built from measured latency, flops, and a small risk proxy. Only behavioral surplus penalty with a gentle dual loop on λ_{beh} .

MVP-1: Add robust PDS. MDN emits a small W_{ref} refined by Alg. A3; gate enforces budget and coverage. Priority uses a smoothed $\min_k \Delta B_{o,k} / \text{cost}(o)$.

MVP-2: Two-level motives-to-sub-motives. Two MDNs (motives, sub-motives) each emit a slice or cover; apply CDS/PDS at each level; pass attention downward multiplicatively with per-level conservation. Add structural weakness via a simple $w(p_o)$ and a gentle λ_{struct} loop.

MVP-3: Tri-modal EASA-Waveformer. Unify NN charts, LOG guards/macros, and EVO amalgams under the same $(\hat{r}, \hat{n})$ ABI and the same CDS/PDS gate; keep conservative attention as the sole runtime selector across modalities.

12 Conclusion

This work has been motivated by four converging contexts:

- First, on the path to AGI, practical progress increasingly hinges on *reliable goal pursuit*: learning what to want (motives), how to break wants into do-ables (subgoals), and how to compose the do-ables into behavior with clear, auditable guarantees. This is an important, broad and deep enough aspect of AGI that it seems to merit a systematic treatment.
- Second, in the reinforcement-learning lineage, we have aimed to move beyond reward-respecting subtasks toward a robust, slice-wise admission rule that captures risk, constraints, and multi-objective tradeoffs as easily as expected value.
- Third, to support cross-paradigm AGI in particular, we felt a need for a language in which neural/predictive-coding, logical/programmatic, and evolutionary/programmatic methods contribute *symmetrically* and compose cleanly, not as ad hoc bolt-ons but as first-class citizens in one planning dynamic.
- Last but not least, within the Hyperon and PRIMUS approach to AGI, we have sought a formulation that sits natively on Atomspace structures, aligns with MetaMo and WDL, and compiles to scalable MORK execution, so that the same algebra speaks both formal theory and running systems.

Toward these ends we have presented *SubRep*, a planner-facing, cross-paradigm framework for subgoal decomposition that unifies robust admission (CDS/PDS), co-learned motive geometry (MDN), and modality-neutral option composition. In this framework, temporal abstraction becomes representation learning over motive slices, with auditable, low-variance certificates that directly reflect what the planner trades off. The same algebra holds for neural/PC controllers, logical macros, and evolutionary programs, enabling clean composition via joins and residuation, and yielding *weakest-sufficient* decompositions when plan-level specifications are present.

On the theoretical side we established join-safety for CDS/PDS, monotonicity under MDN tightening/refinement, and robust certification under model and MDN uncertainty; we gave a two-timescale convergence argument for MDN-CDS co-learning; we derived sample-complexity and approximation bounds, and analyzed library growth (typically logarithmic) that preserves planning tractability. A categorical treatment clarified how planning forms a residuated quantale and how CDS/PDS correspond to order predicates on decision transformers; this enabled *EASA*, which formalizes how smooth neural charts, logical guards, and evolutionary mixtures can

be glued into certified, planner-useful decompositions, with sufficiency and approximation guarantees and qualitative compositional-weakness bounds.

On the practical side, we showed how predictive coding realizes CD-S/PDS *locally* with precision-weighted residuals and why PC’s factorized credit assignment aids motive disentanglement and streaming adaptation relative to continual backprop. We outlined an *EASA-Waveformer* that hosts MDN, gates, and option heads on a shared compute fabric, and we mapped the design to MORK for scalable cross-paradigm execution. Finally, we added a *conservative attention* layer that conserves compute, ties runtime to certificate strength, and simplifies PRIMUS RFS while preserving its economic insights. Tiny case studies made the phenomena concrete (CDS vs. reward-respecting, PDS vs. CDS on broad cones, PC vs. continual backprop, and risk-sensitive preference flips).

Looking forward, the most important next steps are empirical: implement the full pipeline across domains (robotics, research assistance, safe self-modification), quantify admission stability, library size, and end-to-end performance under risk and constraints, and deepen MDN disentanglement and uncertainty calibration. On theory, extending valuation algebras (nested risks, temporal logic), tightening EASA rates and compositional-weakness bounds, and refining certified runtime allocation are natural directions. With SubRep’s algebra and execution blueprint in hand, we hope to speed construction of reliable, adaptable goal pursuit for human-level and beyond-human AGI systems.

Acknowledgements

Thanks due to Richard Sutton for his keynote at AGI-25 which started me thinking along these lines; to Zar Goertzel for suggesting the generalization from cone-dominance to Pareto-dominance; and to Ruiting Lian for listening to me babble about this stuff for most of our August 2025 travels across Iceland, Greenland and Kenya (during which I put together the first version of this document during early mornings and late evenings and long flights...!).

References

- [1] Richard S. Sutton, Marlos C. Machado, G. Zacharias Holland, David Szepesvri, Finbarr Timbers, Brian Tanner, and Adam White, *Reward-Respecting Subtasks for Model-Based Reinforcement Learning*, Artificial Intelligence, vol. 324, no. C, art. no. 104001, 2023.

- [2] Ben Goertzel *et al.*, *OpenCog Hyperon: A Framework for AGI at the Human Level and Beyond*, arXiv:2310.18318 [cs.AI], 19 September 2023.
- [3] Ben Goertzel and Ruiting Lian, *MetaMo motivational system*, presented at AGI-25: 18th International Conference on Artificial General Intelligence, Reykjavik, Iceland, 10-13 August 2025.
- [4] Ben Goertzel, *Weakness Is All You Need*, unpublished manuscript, 2025.
- [5] Ben Goertzel, *Keynote: Weakness is All You Need*, keynote delivered at AGI-25: 18th International Conference on Artificial General Intelligence, Reykjavik, Iceland, 10-13 August 2025.
- [6] R. P. N. Rao, D. C. Gklezakos, and V. Sathish. Active Predictive Coding: A Unifying Neural Model for Active Perception, Compositional Learning, and Hierarchical Planning. *Neural Computation*, 36(1):1-32, 2024.
- [7] A. Ororbia and A. A. Mali. Active Predicting Coding: Brain-Inspired Reinforcement Learning for Sparse Reward Robotic Control Problems. arXiv preprint arXiv:2209.09174, 2022.
- [8] B. Goertzel. QuantiMORK & NeuroMORK via Multiresolution DAGs. SingularityNET Technical Report, unpublished, 2025.
- [9] Ben Goertzel Reinforcement Learning: Some Limitations of the Paradigm <https://multiverseaccordingtoben.blogspot.com/2009/05/reinforcement-learning-some-limitations.html>, 2009
- [10] Shibhansh Dohare, Richard S. Sutton, and A. Rupam Mahmood. Continual Backprop: Stochastic Gradient Descent with Persistent Randomness. *arXiv:2108.06325*, 2021. <https://arxiv.org/abs/2108.06325>
- [11] Ben Goertzel, Matthew Iklé, Izabela Freire Goertzel, and Ari Heljakka. *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*. Springer, New York, 2009. doi:10.1007/978-0-387-76872-4. ISBN 978-0-387-76871-7.
- [12] Ben Goertzel. Metagoals Endowing Self-Modifying AGI Systems with Goal Stability or Moderated Goal Evolution: Toward a Formally Sound and Practical Approach. arXiv:2412.16559 [cs.AI], 2024. <https://arxiv.org/abs/2412.16559>
- [13] Eric B. Baum, *What Is Thought?*, MIT Press, Cambridge, MA, 2004.

- [14] David I. Spivak and Nelson Niu, *Polynomial Functors: A General Theory of Interaction*, Cambridge University Press, 2022.